

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Quách Ngọc Quang

**THIẾT KẾ HỆ THỐNG AIOT PHÁT HIỆN VÀ
CẢNH BÁO BẤT THƯỜNG CHO MÁY GIẶT
DÂN DỤNG SỬ DỤNG AUTOENCODER
KHÔNG GIÁM SÁT TRÊN THIẾT BỊ BIÊN**

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Kỹ thuật Máy tính

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Quách Ngọc Quang

THIẾT KẾ HỆ THỐNG AIOT PHÁT HIỆN VÀ
CẢNH BÁO BẤT THƯỜNG CHO MÁY GIẶT
DÂN DỤNG SỬ DỤNG AUTOENCODER
KHÔNG GIÁM SÁT TRÊN THIẾT BỊ BIÊN

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Kỹ thuật Máy tính

Cán bộ hướng dẫn: TS. Nguyễn Kiêm Hùng

Cán bộ đồng hướng dẫn: TS. Mai Linh

HÀ NỘI - 2026

TÓM TẮT

Đồ án với đề tài "*Thiết kế hệ thống AIoT phát hiện và cảnh báo bất thường cho máy giặt dân dụng sử dụng Autoencoder không giám sát trên thiết bị biên*" tập trung xây dựng hệ thống AIoT phát hiện và cảnh báo bất thường cho máy giặt dân dụng. Hệ thống tích hợp cảm biến âm thanh INMP441 và cảm biến gia tốc ADXL345, xử lý dữ liệu trực tiếp trên vi điều khiển Seeed Studio XIAO ESP32-S3 theo kiến trúc ba trạng thái cho ba pha vận hành GENTLE, STRONG và SPIN.

Về mặt thuật toán, đồ án xây dựng mô hình tự mã hóa đối xứng, kết hợp trích xuất đặc trưng tín hiệu số, lượng tử hóa INT8 và cơ chế phân tách mô hình theo từng pha vận hành. Cách tiếp cận này giúp hệ thống học quy luật của trạng thái bình thường, sau đó phát hiện bất thường thông qua sai số tái tạo mà không cần tập dữ liệu lỗi được gán nhãn đầy đủ.

Về mặt triển khai, hệ thống được hoàn thiện theo mô hình từ biên đến ứng dụng. Kết quả suy luận tại vi điều khiển Seeed Studio XIAO ESP32-S3 được truyền qua MQTT trên HiveMQ Cloud, lưu lịch sử bằng Firebase Firestore và hiển thị trên ứng dụng di động Flutter; nhờ đó người dùng có thể theo dõi trạng thái máy giặt và nhận cảnh báo gần thời gian thực.

Kết quả thực nghiệm cho thấy mô hình đạt F1-Score lần lượt 0,9898, 0,9892 và 0,9967 cho ba pha GENTLE, STRONG và SPIN; AUC đều lớn hơn 0,98. Độ trễ suy luận trung bình trên vi điều khiển Seeed Studio XIAO ESP32-S3 đạt 6,4 ms, dữ liệu đo xa được gửi với tần suất 1 Hz, độ trễ từ biên đến ứng dụng dưới 2 giây, trong khi firmware hoàn chỉnh sử dụng khoảng 1,20 MB Flash và 62 KB SRAM. Các kết quả này cho thấy giải pháp có khả năng vận hành ổn định trên phần cứng tài nguyên thấp và phù hợp để tích hợp vào các hệ thống giám sát máy giặt dân dụng.

Từ khóa: *AIoT, TinyML, Mạng nơ-ron tự mã hóa, Phát hiện bất thường, ESP32-S3.*

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án tốt nghiệp, em đã nhận được sự quan tâm, hướng dẫn và hỗ trợ tận tình từ nhiều cá nhân và tập thể. Nhân dịp này, em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất.

Trước tiên, em xin gửi lời cảm ơn trân trọng đến TS. Nguyễn Kiêm Hùng — cán bộ hướng dẫn chính — và TS. Mai Linh — cán bộ đồng hướng dẫn — đã tận tình định hướng đề tài, dành thời gian trao đổi chuyên môn và đưa ra những nhận xét xác đáng trong suốt quá trình nghiên cứu. Những chỉ dẫn của thầy cô về xử lý tín hiệu số, tối ưu hóa mô hình trên nền tảng TinyML và quản lý tài nguyên bộ nhớ trên vi điều khiển là nền tảng quan trọng giúp em hoàn thành đồ án này.

Em xin trân trọng cảm ơn toàn thể quý thầy cô giảng viên thuộc Khoa Điện tử Viễn thông, Ngành Kỹ thuật Máy tính, Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội đã truyền đạt nền tảng kiến thức vững chắc trong suốt bốn năm đào tạo. Những kiến thức về kiến trúc máy tính, hệ điều hành thời gian thực và trí tuệ nhân tạo chính là cơ sở lý luận và thực tiễn không thể thiếu để triển khai đề tài này.

Cuối cùng, em xin gửi lời tri ân sâu sắc đến gia đình và bạn bè đã luôn đồng hành, động viên và tạo điều kiện thuận lợi trong suốt thời gian thực hiện đồ án.

Do giới hạn về thời gian và kinh nghiệm thực tiễn, đồ án không tránh khỏi những thiếu sót nhất định. em kính mong nhận được sự góp ý, chỉ bảo của quý thầy cô trong Hội đồng chấm đồ án để công trình được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Quách Ngọc Quang

LỜI CAM ĐOAN

Tôi tên là Quách Ngọc Quang, sinh viên năm tư ngành Kỹ thuật máy tính, Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội. Tôi xin cam đoan đồ án tốt nghiệp với đề tài "*Thiết kế hệ thống AIoT phát hiện và cảnh báo bất thường cho máy giặt dân dụng sử dụng Autoencoder không giám sát trên thiết bị biên*" là công trình nghiên cứu của riêng tôi, được thực hiện dưới sự hướng dẫn khoa học của TS. Nguyễn Kiêm Hùng và TS. Mai Linh.

Các số liệu, kết quả thực nghiệm trình bày trong đồ án về độ trễ suy luận, sai số tái tạo (MAE) và các chỉ số lượng tử hóa mô hình là hoàn toàn trung thực và chưa từng được công bố trong bất kỳ công trình nào khác. Hệ thống phần cứng dựa trên vi điều khiển Seeed Studio XIAO ESP32-S3, mã nguồn firmware vận hành đa lõi trên FreeRTOS và ứng dụng giám sát đi kèm đều do tôi tự nghiên cứu, phát triển và tối ưu hóa. Các tài liệu tham khảo và mã nguồn mở sử dụng trong dự án đều được trích dẫn nguồn gốc rõ ràng.

Nếu có bất kỳ sự gian lận hoặc vi phạm quy chế nào về tính nguyên bản của công trình, tôi xin chịu hoàn toàn trách nhiệm trước Hội đồng bảo vệ và Ban Giám hiệu nhà trường.

Hà Nội, ngày ... tháng ... năm 2026

Tác giả

Quách Ngọc Quang

PHÊ DUYỆT CỦA CÁN BỘ HƯỚNG DẪN

...

Hà Nội, ngày ... tháng ... năm 2026

Cán bộ hướng dẫn

Cán bộ đồng hướng dẫn

TS. Nguyễn Kiêm Hùng

TS. Mai Linh

Mục lục

TÓM TẮT	i
LỜI CẢM ƠN	ii
LỜI CAM ĐOAN	iii
PHÊ DUYỆT CỦA CÁN BỘ HƯỚNG DẪN	iv
MỞ ĐẦU	xii
1 TỔNG QUAN LÝ THUYẾT	1
1.1 Nền tảng AIoT và xử lý tại biên	1
1.1.1 Điện toán biên và suy luận tại thiết bị	2
1.2 Bài toán phát hiện bất thường	2
1.2.1 Khái niệm và phân loại bất thường	3
1.2.2 Các hướng tiếp cận học máy và thách thức khi thiếu hụt dữ liệu lỗi	3
1.2.3 Phương pháp đánh giá bất thường dựa trên sai số tái tạo . .	4
1.3 Mô hình mạng nơ-ron tự mã hóa và các biến thể	5
1.3.1 Kiến trúc mạng Dense	5
1.3.2 Kiến trúc 1D-CNN và các giới hạn trên nền tảng nhúng . . .	6
1.3.3 Lựa chọn kiến trúc tối ưu cho bài toán giám sát thiết bị gia dụng	7
1.4 Công nghệ TinyML trên vi điều khiển	8

1.4.1	Ràng buộc về tài nguyên phần cứng	8
1.4.2	Kỹ thuật lượng tử hóa mô hình (Quantization INT8)	8
1.4.3	Framework TensorFlow Lite for Microcontrollers	9
1.5	Cơ sở xử lý tín hiệu cảm biến	10
1.5.1	Trích xuất đặc trưng âm thanh	10
1.5.2	Trích xuất đặc trưng rung động cơ học	12
1.5.3	Kỹ thuật kết hợp dữ liệu đa phương thức	12
1.6	Các công trình liên quan và khoảng trống nghiên cứu	13
1.6.1	Các công trình liên quan trong lĩnh vực giám sát và phát hiện bất thường	13
1.6.2	Khoảng trống nghiên cứu và đóng góp của đề án	15
2	THIẾT KẾ HỆ THỐNG	16
2.1	Kiến trúc tổng thể hệ thống	16
2.1.1	Sơ đồ khối chức năng	16
2.1.2	Đường ống xử lý luồng dữ liệu	18
2.1.3	Cơ chế điều phối đa nhiệm trên FreeRTOS	18
2.2	Thiết kế phần cứng	19
2.2.1	Nền tảng vi điều khiển lõi kép XIAO ESP32-S3	19
2.2.2	Tích hợp microphone kỹ thuật số INMP441 qua chuẩn giao tiếp I2S	20
2.2.3	Tích hợp cảm biến gia tốc ADXL345 qua chuẩn giao tiếp I2C	20
2.2.4	Sơ đồ kết nối và bố trí thiết bị trên đối tượng nghiên cứu . .	21
2.3	Kiến trúc phần mềm nhúng dựa trên hệ điều hành FreeRTOS . . .	22
2.3.1	Thiết kế luồng xử lý tín hiệu âm thanh trên lõi số 0	22
2.3.2	Thiết kế luồng thu thập rung động và suy luận AI trên lõi số 1	23
2.3.3	Cơ chế đồng bộ hóa đa tác vụ	24
2.4	Đề xuất kiến trúc phân mảnh trạng thái	24

2.4.1	Cơ sở phân tách chu trình hoạt động	24
2.4.2	Thiết kế mạng tự mã hóa Dense chuyên biệt	25
2.4.3	Kỹ thuật cân bằng đặc trưng với hàm mất mát MAE có trọng số	26
2.4.4	Đánh giá tương thích hàm kích hoạt trong môi trường lượng tử hóa	26
2.5	Thiết kế hạ tầng truyền thông Internet of Things	27
2.5.1	Mô hình xuất bản - đăng ký (Publish/Subscribe) và Broker	27
2.5.2	Cấu trúc bản tin chuẩn hóa JSON	27
2.5.3	Kỹ thuật truyền tin hỗn hợp: Telemetry định kỳ và Hướng sự kiện	28
2.5.4	Bảo mật truyền tải và Đánh đổi tài nguyên (Trade-offs) . . .	28
3	XÂY DỰNG DỮ LIỆU VÀ HUẤN LUYỆN MÔ HÌNH	29
3.1	Quá trình thu thập dữ liệu thực tế	29
3.1.1	Xây dựng giao thức truyền nhận dữ liệu tốc độ cao định dạng nhị phân	29
3.1.2	Tự động hóa quy trình thu thập và kiểm soát chất lượng dữ liệu	30
3.1.3	Thống kê phân phối và xử lý sự mất cân bằng dữ liệu (Data Imbalance)	30
3.2	Trích xuất đặc trưng và tiền xử lý dữ liệu	32
3.2.1	Thiết kế bộ lọc Mel tối ưu cho tín hiệu cơ khí	32
3.2.2	Kỹ thuật đồng bộ hóa thuật toán xử lý giữa Python và Firmware C++	33
3.2.3	Xử lý dữ liệu ngoại lai cho tín hiệu rung động	34
3.3	Cấu hình huấn luyện và lượng tử hóa mô hình	34
3.3.1	Phân chia tập huấn luyện, kiểm định và kiểm thử	35
3.3.2	Kỹ thuật tăng cường dữ liệu	35

3.3.3	Quá trình huấn luyện mô hình số thực (Float32)	35
3.3.4	Lượng tử hóa nhận thức huấn luyện	36
3.3.5	Xác định ngưỡng cảnh báo dựa trên quy tắc 3σ	37
3.3.6	Đánh giá cấu hình và hiệu năng tài nguyên	38
3.4	Kiểm chứng tính nhất quán của hệ thống	38
3.4.1	Đối chiếu phân phối đặc trưng giữa môi trường giả lập và phần cứng	38
3.4.2	Xác thực tính toàn vẹn của đồ thị suy luận	39
4	TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ	41
4.1	Triển khai phần mềm trên vi điều khiển Seeed Studio XIAO ESP32-S3	41
4.1.1	Chiến lược quản lý bộ nhớ cho hệ thống đa mô hình	41
4.1.2	Thực thi tiền xử lý tín hiệu trực tiếp trên Firmware	42
4.1.3	Tính toán sai số tái tạo trên miền lượng tử hóa (INT8-domain)	42
4.1.4	Cơ chế lọc trễ thời gian	42
4.2	Đánh giá hiệu năng hệ thống nhúng	43
4.2.1	Phân tích độ trễ suy luận	44
4.2.2	Đánh giá mức sử dụng tài nguyên bộ nhớ và độ ổn định hệ thống	45
4.3	Đánh giá khả năng phát hiện bất thường	47
4.3.1	Thiết lập kịch bản kiểm thử với giả lập lỗi (Anomaly Injection)	47
4.3.2	Phân tích phân phối sai số tái tạo	47
4.3.3	Đánh giá độ nhạy và hiệu quả qua các chỉ số học máy	48
4.3.4	Cơ chế lọc trễ thời gian và sự ổn định dài hạn	48
4.4	Phân tích thực nghiệm các quyết định thiết kế mô hình	49
4.4.1	Lựa chọn kiến trúc mạng qua các giai đoạn thử nghiệm	49
4.4.2	Nghiên cứu thành phần các biến thể mô hình	50
4.4.3	Phân tích ngưỡng τ bằng đường cong ROC và chỉ số Youden's J	51

4.4.4	Phân tích sâu nguyên nhân sai số pha GENTLE	52
4.5	Triển khai ứng dụng di động giám sát và cảnh báo thời gian thực .	52
4.5.1	Kiến trúc phần mềm và kết nối đám mây	53
4.5.2	Logic xử lý dữ liệu và thông báo đẩy	53
4.5.3	Giao diện người dùng và trực quan hóa dữ liệu	55
4.6	So sánh với các nghiên cứu liên quan	57
4.6.1	Giới thiệu hệ thống mốc đối sánh của Mansoureh Lord . . .	57
4.6.2	Cải tiến về kiến trúc mạng và thực thi phần cứng	57
4.6.3	Khả năng thích nghi đa trạng thái và độ tin cậy	58
4.6.4	Đối chiếu với các hướng nghiên cứu gần đây	58
4.7	Thảo luận và phân tích các mặt hạn chế	59
4.7.1	Hiện tượng Concept Drift và sự lão hóa cơ khí	59
4.7.2	Độ tin cậy hạ tầng và điều kiện vật lý của cảm biến	60
4.7.3	Giới hạn của dữ liệu lỗi giả lập	60
5	KẾT LUẬN	61
	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
5.1	Những kết quả đạt được	61
5.2	Hạn chế của đề tài	62
5.3	Hướng phát triển tiếp theo	62
	TÀI LIỆU THAM KHẢO	64

Danh sách hình vẽ

1.1	Kiến trúc mạng nơ-ron tự mã hóa	5
1.2	Đường ống trích xuất đặc trưng âm thanh MFE.	10
2.1	Sơ đồ kiến trúc tổng thể hệ thống	17
2.2	Sơ đồ kết nối phần cứng	21
2.3	Kiến trúc mạng tự mã hóa Dense đối xứng dùng cho mỗi pha vận hành.	25
3.1	Phân phối phương sai trục Z	31
3.2	Phân phối dữ liệu trước và sau tăng cường	32
3.3	Cấu trúc 13 dải lọc Mel	33
3.4	Đối chiếu phân phối năng lượng âm thanh	39
4.1	Cơ chế lọc nhiễu bằng cửa sổ trượt	43
4.2	Phân phối độ trễ suy luận	45
4.3	Kết quả biên dịch firmware	46
4.4	Biểu đồ phân tách sai số MAE	48
4.5	Đường cong ROC ba pha vận hành	51
4.6	Kiến trúc kết nối đám mây của ứng dụng di động	53
4.7	Minh họa thông báo đẩy trên Android	55
4.8	Giao diện ứng dụng di động Flutter	56

Danh sách bảng

3.1	Cách chia dữ liệu huấn luyện và kiểm định theo từng pha vận hành.	35
3.2	Cấu hình huấn luyện được sử dụng trong mã nguồn <code>training.py</code> .	36
3.3	Chi tiết tham số mạng tự mã hóa	38
4.1	Thống kê độ trễ suy luận	44
4.2	Chỉ số hiệu năng mô hình Ba trạng thái	48
4.3	So sánh ba kiến trúc mạng sau nén INT8 trên Kaggle	49
4.4	So sánh hiệu năng các biến thể mô hình	50
4.5	Kết quả khảo sát lớp thất cổ chai	51
4.6	Đối chiếu hiệu năng với mốc đối sánh	59
5.1	Tổng hợp các kết quả định lượng chính của đề án.	62

MỞ ĐẦU

1. Tính cấp thiết của đề tài

Giai đoạn 2026–2030 được xem là thời kỳ bứt tốc của hệ sinh thái Internet vạn vật (IoT) và quá trình chuyển dịch sang Trí tuệ nhân tạo vạn vật (AIoT). Quy mô kết nối toàn cầu được dự báo đạt khoảng 21,9 tỷ thiết bị trong năm 2026 [26] và có thể tăng lên khoảng 39 tỷ thiết bị kết nối vào năm 2030 [27]. Sự gia tăng này không chỉ mở rộng số lượng nút cảm biến trong môi trường gia đình và công nghiệp, mà còn tạo áp lực lớn lên băng thông truyền tải, chi phí lưu trữ, độ trễ phản hồi và khả năng bảo vệ dữ liệu riêng tư. Vì vậy, kiến trúc hệ thống đang dịch chuyển từ mô hình xử lý tập trung trên đám mây sang mô hình xử lý phân tán tại thiết bị biên, nơi dữ liệu được phân tích gần nguồn phát sinh hơn [1].

Trong chu kỳ phát triển 2026–2031, AIoT được dự báo tăng trưởng mạnh khi các mô hình AI được tích hợp trực tiếp với các điểm cuối IoT giàu cảm biến, chuyển hệ thống từ giám sát phản ứng sang trí tuệ tự trị tại biên [28]. Đối với các hệ thống nhúng giám sát thiết bị gia dụng, trọng tâm không chỉ là bảo trì dự đoán dài hạn mà còn là phát hiện bất thường và cảnh báo thời gian thực ngay trên vi điều khiển. Các kỹ thuật TinyML cho phép thiết bị liên tục theo dõi chuỗi dữ liệu đa biến từ cảm biến âm thanh và rung động, sau đó đưa ra phản hồi tức thời mà không cần gửi toàn bộ dữ liệu thô lên máy chủ. Cách tiếp cận này phù hợp với xu hướng thiết kế AI bền vững, trong đó hiệu quả năng lượng và giảm tải hạ tầng tính toán được xem là tiêu chí quan trọng của các hệ thống AI hiện đại [29].

Bên cạnh đó, từ năm 2027 trở đi, quá trình chuẩn bị cho mạng 6G và Massive IoT đặt ra yêu cầu mới về kết nối quy mô lớn, độ trễ thấp, khả năng vận hành tin cậy và bảo mật đầu cuối [30], [31]. Sự xuất hiện của Trí tuệ nhân tạo tác nhân (Agentic AI) trong hạ tầng viễn thông cho thấy mạng kết nối trong giai đoạn tới sẽ ngày càng tự động hóa, tự tối ưu và phụ thuộc nhiều hơn vào các luồng dữ liệu từ thiết bị biên [32]. Đồng thời, khi số lượng thiết bị và dịch vụ tự động tăng nhanh,

định danh máy móc và bảo mật thiết bị trở thành lớp phòng vệ quan trọng để bảo vệ toàn vẹn dữ liệu AIoT [33].

Trong bối cảnh đó, các thiết bị gia dụng có hệ thống truyền động cơ khí phức tạp như máy giặt là một đối tượng phù hợp để nghiên cứu AIoT tại biên. Các sự cố như mất cân bằng tải, lệch lồng giặt hay hư hỏng vòng bi thường biểu hiện qua rung động và âm thanh đặc trưng [2]. Nếu không được phát hiện và cảnh báo kịp thời, các lỗi này có thể gây hư hại phần cứng, làm giảm tuổi thọ thiết bị và tiềm ẩn nguy cơ mất an toàn cho người sử dụng [2]. Tuy nhiên, việc truyền liên tục dữ liệu âm thanh và rung động lên đám mây gây tốn băng thông, tăng độ trễ và tạo rủi ro riêng tư; trong khi dữ liệu lỗi thực tế lại khan hiếm, khiến các thuật toán học có giám sát dễ gặp vấn đề mất cân bằng dữ liệu [3], [4]. Do đó, sự kết hợp giữa TinyML và mô hình học không giám sát trở thành hướng tiếp cận phù hợp, cho phép thiết bị học quy luật vận hành bình thường và phát hiện bất thường thông qua sai số tái tạo ngay tại nguồn dữ liệu [3, 1].

Xuất phát từ những đòi hỏi thực tế và tiềm năng công nghệ trên, đề tài: *"Thiết kế hệ thống AIoT phát hiện và cảnh báo bất thường cho máy giặt dân dụng sử dụng Autoencoder không giám sát trên thiết bị biên"* được lựa chọn nghiên cứu nhằm cung cấp một giải pháp giám sát độc lập, thời gian thực và bảo mật.

2. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, dựa trên nền tảng lý thuyết về học sâu và các khung tối ưu hóa cho nền tảng biên, đề tài trực tiếp đề xuất kiến trúc mạng tự mã hóa đối xứng kết hợp kỹ thuật lượng tử hóa INT8 chuyên biệt để có thể hoạt động tối ưu trên dòng vi điều khiển ESP32-S3 [5, 6]. Đồng thời, nghiên cứu cũng đề xuất cơ chế phân mảnh trạng thái vận hành đa mô hình, tiến hành thực nghiệm và chứng minh hiệu quả vượt trội của cơ chế này trong việc nâng cao độ chính xác phân tách lỗi so với cấu trúc đơn mô hình [6].

Về mặt thực tiễn, đồ án cung cấp một hệ thống kết nối đầu cuối hoàn chỉnh, bao gồm từ nút mạng AI tại biên xử lý đa lỗi tại thiết bị cho đến ứng dụng di động thông minh cảnh báo tức thời cho người dùng. Giải pháp có tính linh hoạt cao, dễ dàng triển khai tích hợp trên các dòng máy giặt dân dụng hiện hành nhằm giảm thiểu rủi ro hỏng hóc lớn mà không cần can thiệp sâu vào cấu trúc phần cứng của nhà sản xuất.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề án tập trung vào bài toán phát hiện bất thường trên chuỗi thời gian đa phương thức thông qua việc ứng dụng các kiến trúc mạng nơ-ron học không giám sát và các phương pháp xử lý tín hiệu số trong nền tảng công nghệ TinyML.

Phạm vi nghiên cứu được xác định rõ trên ba khía cạnh. Về nền tảng phần cứng, hệ thống sử dụng vi điều khiển lõi kép Seeed Studio XIAO ESP32-S3 làm thiết bị biên, thực hiện thu thập dữ liệu qua cảm biến âm thanh kỹ thuật số INMP441 và cảm biến gia tốc ADXL345. Về nền tảng phần mềm, đề tài ứng dụng hệ điều hành thời gian thực để quản lý đa tác vụ, sử dụng giao thức MQTT cho hạ tầng truyền thông và phát triển ứng dụng di động giám sát trên nền tảng Flutter. Về môi trường thực nghiệm, các đánh giá được giới hạn trên các pha vận hành đặc trưng của máy giặt cửa trước dân dụng bao gồm các chế độ giặt nhẹ (GENTLE), giặt mạnh (STRONG) và vắt (SPIN).

4. Phương pháp nghiên cứu

Đề tài áp dụng kết hợp hai nhóm phương pháp chính là nghiên cứu lý thuyết và kiểm thử thực nghiệm. Phương pháp nghiên cứu lý thuyết được thực hiện thông qua việc tổng hợp và phân tích hệ thống các tài liệu học thuật liên quan đến phân tích chuỗi thời gian, lý thuyết nén mô hình học sâu, công nghệ AIoT và các tiêu chuẩn truyền thông bảo mật hiện đại [3, 4, 5].

Trong khi đó, phương pháp thực nghiệm đi theo hướng tiếp cận lấy dữ liệu làm trung tâm [7]. Quá trình này bao gồm việc tự xây dựng trạm thu thập để lấy dữ liệu thực tế tại hiện trường, đồng thời kết hợp các kỹ thuật tăng cường dữ liệu vật lý và giả lập lỗi để đánh giá giới hạn chịu đựng của mô hình. Hệ thống sau đó được triển khai và kiểm thử trên phần cứng thực tế bằng cách lập trình đa tác vụ song song trên vi điều khiển nhằm tối ưu hóa hiệu năng tính toán. Các chỉ số đánh giá học máy (AUC, F1-Score) và thông số sử dụng tài nguyên phần cứng (RAM/Flash, độ trễ) được tiến hành đo lường và thu thập trực tiếp từ các phiên chạy thực địa dài hạn [7].

5. Cấu trúc của đề án

Để giải quyết các mục tiêu và nhiệm vụ đã đặt ra, ngoài phần Mở đầu, Kết luận và Tài liệu tham khảo, nội dung chính của đề án được tổ chức thành 4 chương như sau:

- **Chương 1: Tổng quan lý thuyết.** Trình bày cơ sở lý luận về bài toán phát hiện bất thường, mạng nơ-ron tự mã hóa và các giới hạn kỹ thuật.
- **Chương 2: Thiết kế hệ thống.** Mô tả toàn diện kiến trúc phần cứng, thiết kế phần mềm nhúng đa nhân và đề xuất lý thuyết về kiến trúc mạng tự mã hóa ba trạng thái.
- **Chương 3: Xây dựng dữ liệu và Huấn luyện mô hình.** Trình bày chi tiết quy trình xây dựng tập dữ liệu thực tế, kỹ thuật trích xuất đặc trưng và quá trình lượng tử hóa mô hình học máy.
- **Chương 4: Triển khai và Đánh giá kết quả.** Trình bày các kết quả thực thi mô hình trên vi điều khiển, đánh giá khả năng phát hiện lỗi thực tế, đồng thời thảo luận và phân tích các mặt hạn chế của hệ thống.

TỔNG QUAN LÝ THUYẾT

1.1 Nền tảng AIoT và xử lý tại biên

Internet vạn vật (Internet of Things - IoT) là mô hình kết nối các đối tượng vật lý được nhúng cảm biến, bộ chấp hành và công nghệ truyền thông nhằm thu thập, trao đổi dữ liệu qua Internet [34]. Một hệ thống IoT truyền thống thường gồm ba lớp chính: lớp nhận thức thu thập dữ liệu từ môi trường vật lý, lớp mạng truyền dẫn dữ liệu, và lớp ứng dụng hoặc đám mây đảm nhiệm lưu trữ, phân tích và cung cấp dịch vụ cho người dùng. Trong bối cảnh hiện nay, quy mô kết nối toàn cầu tiếp tục tăng nhanh, với số thiết bị IoT được dự báo đạt khoảng 21,9 tỷ trong năm 2026 [26].

Sự mở rộng này làm bộc lộ hạn chế của mô hình IoT phụ thuộc hoàn toàn vào đám mây. Khi toàn bộ dữ liệu thô, đặc biệt là các tín hiệu có tần số lấy mẫu cao như rung động và âm thanh, đều được truyền lên máy chủ trung tâm, hệ thống phải đối mặt với áp lực băng thông, chi phí lưu trữ, độ trễ phản hồi và rủi ro riêng tư dữ liệu [1]. Các vấn đề này trở nên rõ rệt hơn trong bài toán giám sát tình trạng thiết bị, nơi hệ thống cần phản ứng nhanh trước các dấu hiệu bất thường về cơ khí.

Trí tuệ nhân tạo vạn vật (Artificial Intelligence of Things - AIoT) ra đời từ sự hội tụ giữa trí tuệ nhân tạo và IoT [35]. Thay vì chỉ thu thập và truyền dữ liệu thụ động, thiết bị AIoT có khả năng xử lý, phân tích và ra quyết định ngay tại hoặc gần nguồn phát sinh dữ liệu. Trong phạm vi đề án, hướng tiếp cận này cho phép nút cảm biến gắn ngoài máy giặt tự trích xuất đặc trưng rung động và âm thanh, thực hiện suy luận bất thường tại chỗ, sau đó chỉ truyền kết quả cảnh báo hoặc dữ liệu đã rút gọn lên ứng dụng giám sát. Cách thiết kế này phù hợp với yêu cầu giảm tải đường truyền, giảm độ trễ và tăng tính độc lập của hệ thống khi giám

sát các lỗi như mất cân bằng tải, lệch lồng giặt hoặc hư hỏng vòng bi [2].

1.1.1 Điện toán biên và suy luận tại thiết bị

Điện toán biên (Edge Computing) là mô hình xử lý dữ liệu phân tán, trong đó dữ liệu được xử lý gần nguồn phát sinh thay vì phụ thuộc hoàn toàn vào hạ tầng đám mây [1]. So với xử lý tập trung, điện toán biên giúp giảm độ trễ truyền dẫn, tiết kiệm băng thông và hạn chế việc đưa dữ liệu nhạy cảm của người dùng ra khỏi thiết bị. Đây là cơ sở quan trọng cho các hệ thống giám sát thời gian thực trong môi trường gia dụng, nơi tín hiệu cảm biến có thể được sinh ra liên tục nhưng người dùng chỉ cần nhận các thông tin trạng thái hoặc cảnh báo đã được xử lý.

Kế thừa triết lý của điện toán biên, TinyML tập trung vào việc triển khai các mô hình học máy trên hệ thống nhúng và vi điều khiển có năng lượng thấp, bộ nhớ hạn chế và năng lực tính toán nhỏ hơn nhiều so với máy tính thông thường [3, 6]. Điểm khác biệt cốt lõi của TinyML nằm ở khả năng suy luận tại thiết bị. Mô hình sau huấn luyện được nén, lượng tử hóa và chuyển đổi sang định dạng phù hợp để thực thi trực tiếp trên vi điều khiển, qua đó giảm phụ thuộc vào kết nối Internet và máy chủ từ xa.

Trong đồ án này, vi điều khiển Seeed Studio XIAO ESP32-S3 kết hợp với TensorFlow Lite Micro được sử dụng làm nền tảng suy luận tại biên [36]. Mô hình mạng nơ-ron tự mã hóa được lượng tử hóa về INT8 bằng kỹ thuật lượng tử hóa nhận thức huấn luyện, giúp giảm dung lượng lưu trữ và tăng hiệu quả tính toán trên phần cứng nhúng. Theo kết quả đo thực tế của firmware, tổng dung lượng ba mô hình cho ba pha vận hành là khoảng 95,3 KB, trong khi độ trễ suy luận trung bình đạt 6,4 ms. Các giá trị này cho thấy hệ thống có khả năng xử lý tại thiết bị với chi phí tài nguyên thấp, đồng thời vẫn duy trì được khả năng phản ứng thời gian thực.

1.2 Bài toán phát hiện bất thường

Phát hiện bất thường là bước nền tảng trong các hệ thống giám sát tình trạng thiết bị dựa trên dữ liệu. Trong bối cảnh Internet vạn vật (IoT) và giám sát thiết bị gia dụng, việc nhận diện sớm các dị thường trong tín hiệu cảm biến chuỗi thời gian cho phép phát hiện các sai lệch cơ khí trước khi chúng dẫn đến hỏng hóc

nghiêm trọng. Bất thường, hay Điểm dị thường (Outlier), được hiểu là các mẫu dữ liệu lệch khỏi hành vi bình thường mà hệ thống đã học được [5].

1.2.1 Khái niệm và phân loại bất thường

Dựa trên mối quan hệ không gian và thời gian giữa các điểm dữ liệu, các dị thường thường được chia thành ba dạng: Bất thường điểm (Point anomalies), Bất thường ngữ cảnh (Contextual anomalies) và Bất thường tập hợp (Collective anomalies) [5, 6].

Bất thường điểm xảy ra khi một mẫu dữ liệu đơn lẻ có giá trị sai lệch đột biến so với phân phối chung, ví dụ như một xung gia tốc biên độ lớn xuất hiện tức thời do va đập vật lý bất ngờ vào vỏ máy. Bất thường ngữ cảnh là dạng phức tạp hơn — một điểm dữ liệu chỉ được xem là bất thường khi đặt trong một bối cảnh cụ thể; chẳng hạn, rung lắc mạnh và tiếng ồn lớn là hành vi bình thường ở pha Vắt (Spin), nhưng lại là dấu hiệu hư hỏng nếu xuất hiện trong pha Giặt nhẹ (Gentle). Bất thường tập hợp phản ánh một chuỗi điểm dữ liệu liên tiếp có hành vi sai lệch so với quy luật tổng thể, dù từng điểm đơn lẻ có thể không vượt ngưỡng cảnh báo thông thường. Do máy giặt vận hành theo chu kỳ nhiều pha với đặc trưng tín hiệu khác nhau, đồ án tập trung vào hai dạng bất thường ngữ cảnh và tập hợp thông qua cơ chế phân mảnh trạng thái vận hành [4].

1.2.2 Các hướng tiếp cận học máy và thách thức khi thiếu hụt dữ liệu lỗi

Khi ứng dụng học máy vào bài toán chẩn đoán trạng thái thiết bị, hai hướng tiếp cận phổ biến nhất là học có giám sát (Supervised) và học không giám sát (Unsupervised) [4]. Phương pháp có giám sát đòi hỏi tập dữ liệu (Dataset) phải được gán nhãn đầy đủ cho cả trạng thái bình thường (Healthy state) và các dạng lỗi để thiết lập ranh giới quyết định. Tuy nhiên, hướng đi này gặp một hạn chế lớn trong thực tế: dữ liệu hỏng hóc cơ khí thường rất khan hiếm. Việc chủ động gây ra sự cố trên thiết bị thật để thu thập dữ liệu không chỉ tốn kém mà còn tiềm ẩn rủi ro mất an toàn [2].

Ngược lại, phương pháp học không giám sát chỉ yêu cầu hệ thống học các đặc trưng của trạng thái bình thường — loại dữ liệu rất sẵn có và dễ thu thập trong giai đoạn đầu thiết bị vận hành. Trong quá trình hoạt động, các tín hiệu lệch khỏi phân phối bình thường đã học sẽ được hệ thống nhận diện là bất thường [1]. Do

giới hạn về khả năng mô phỏng đầy đủ các lỗi vật lý trên máy giặt, đồ án này lựa chọn mạng nơ-ron tự mã hóa thuộc nhóm học không giám sát. Cách tiếp cận này giúp mô hình trích xuất hiệu quả các đặc trưng phi tuyến từ dữ liệu đa phương thức mà không phụ thuộc vào tập dữ liệu nhãn lỗi [5].

1.2.3 Phương pháp đánh giá bất thường dựa trên sai số tái tạo

Mạng tự mã hóa phát hiện bất thường thông qua cơ chế nén và khôi phục tín hiệu. Về cấu trúc, bộ mã hóa (Encoder) nén dữ liệu đầu vào X xuống một không gian ẩn (Latent space) có số chiều thấp hơn để trích xuất các đặc trưng cốt lõi. Sau đó, bộ giải mã (Decoder) tái tạo lại dữ liệu từ không gian ẩn này thành bản sao $\hat{X}$ [5]. Vì mô hình chỉ được huấn luyện trên dữ liệu bình thường, nó sẽ tái tạo rất chính xác các tín hiệu cơ học quen thuộc. Ngược lại, khi máy giặt gặp sự cố, tín hiệu cảm biến sẽ chứa các thành phần tần số hoặc biên độ lạ, khiến bộ giải mã không thể khôi phục chính xác và tạo ra sai số lớn.

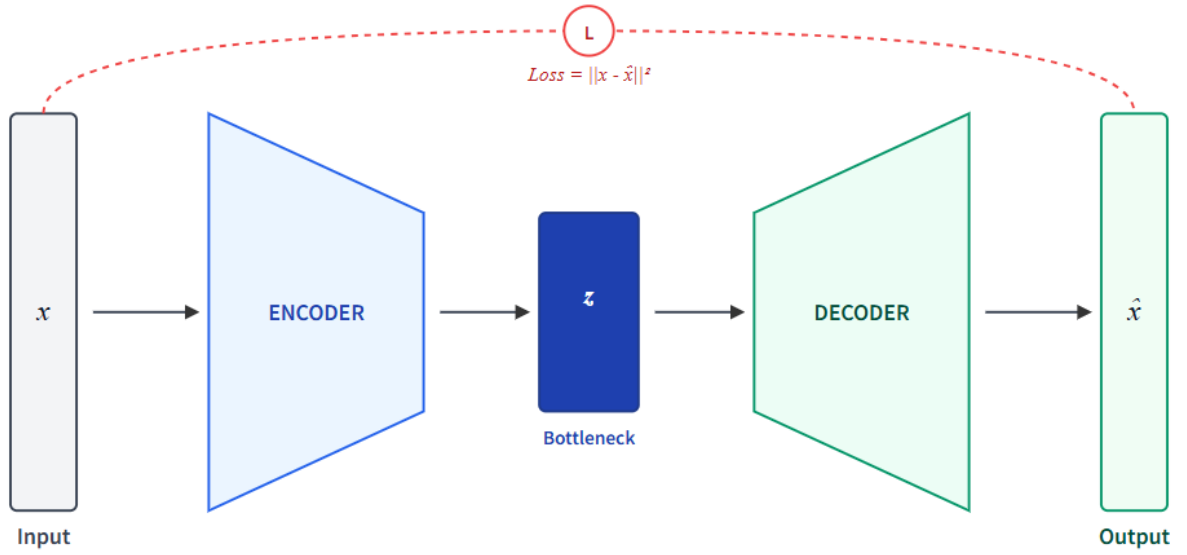
Sự chênh lệch giữa đầu vào gốc X và đầu ra tái tạo $\hat{X}$ chính là Sai số tái tạo (Reconstruction Error). Trong môi trường TinyML giới hạn tài nguyên như vi điều khiển Seeed Studio XIAO ESP32-S3, đồ án ưu tiên sử dụng hàm Sai số tuyệt đối trung bình (Mean Absolute Error - MAE) thay vì MSE. Lựa chọn này giúp giảm thiểu số chu kỳ tính toán của CPU trên dữ liệu số nguyên 8-bit (INT8), đồng thời tránh hiện tượng bùng nổ gradient khi tín hiệu đầu vào lẫn các Xung nhiễu (Spikes) cực đại [3, 1]. Công thức tính MAE cho một mẫu đầu vào có N chiều đặc trưng được biểu diễn ở Phương trình 1.1:

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i| \quad (1.1)$$

Hệ thống ra quyết định bằng cách so sánh MAE tức thời với một ngưỡng dị thường θ (được rút ra từ phân phối thống kê của tập huấn luyện). Nếu $MAE > \theta$, thiết bị bị coi là có bất thường. Tuy nhiên, để tránh các False Positives do nhiễu môi trường nhất thời, hệ thống được tích hợp thêm bộ lọc Lọc nhiễu thời gian (Temporal Smoothing). Thay vì cảnh báo ngay lập tức, hệ thống quan sát cửa sổ 10 mẫu gần nhất và chỉ kích hoạt báo động khi có ít nhất 5 mẫu vượt ngưỡng θ .

1.3 Mô hình mạng nơ-ron tự mã hóa và các biến thể

Mạng tự mã hóa là một kiến trúc mạng nơ-ron học sâu hoạt động theo cơ chế học không giám sát. Cấu trúc cốt lõi của mạng thường được thiết kế đối xứng, bao gồm hai thành phần chính: bộ mã hóa nén dữ liệu đầu vào thành một không gian ẩn có số chiều thấp hơn, và bộ giải mã khôi phục lại dữ liệu từ không gian ẩn đó sao cho đầu ra khớp nhất với tín hiệu ban đầu [5]. Trong bài toán phát hiện bất thường, cơ chế này ràng buộc mạng nơ-ron phải học cách trích xuất các đặc trưng cốt lõi và quy luật vận hành của trạng thái bình thường, đồng thời tự động loại bỏ các thành phần nhiễu không quan trọng.



Hình 1.1

Kiến trúc tổng quát của mạng nơ-ron tự mã hóa.

1.3.1 Kiến trúc mạng Dense

Mạng nơ-ron tự mã hóa Dense là cấu trúc cơ bản nhất của mạng tự mã hóa, trong đó mỗi nơ-ron đều được kết nối trực tiếp với toàn bộ nơ-ron ở tầng kế tiếp. Quá trình hoạt động của mạng Dense bao gồm hai giai đoạn chính. Đầu tiên, bộ mã hóa sử dụng hàm f_ϕ để ánh xạ vectơ đầu vào $x \in \mathbb{R}^d$ sang không gian ẩn $h \in \mathbb{R}^k$ (với $k < d$), như trong Phương trình 1.2:

$$h = \sigma(W_e x + b_e) \quad (1.2)$$

Sau đó, bộ giải mã dùng hàm g_ψ để ánh xạ ngược từ không gian ẩn h về lại không gian ban đầu, tạo ra tín hiệu phục hồi $\hat{x} \in \mathbb{R}^d$ theo Phương trình 1.3:

$$\hat{x} = \sigma'(W_d h + b_d) \quad (1.3)$$

Trong đó, W_e, W_d là các ma trận trọng số; b_e, b_d là các vectơ độ lệch (bias); và σ, σ' là các hàm kích hoạt phi tuyến. Quá trình huấn luyện sẽ tối ưu hóa tập tham số ϕ và ψ nhằm cực tiểu hóa sai số tái tạo trên dữ liệu bình thường. Với dữ liệu tín hiệu cảm biến chuỗi thời gian, hàm mất mát thường được sử dụng nhất là Sai số bình phương trung bình (Mean Squared Error - MSE):

$$\mathcal{L}(\phi, \psi) = \frac{1}{n} \sum_{i=1}^n |x_i - g_\psi(f_\phi(x_i))|^2 \quad (1.4)$$

Trong bài toán của đề án, do vectơ đầu vào chỉ có kích thước 19 chiều (gồm các đặc trưng thống kê rung động và Năng lượng dải Mel - MFE¹), cấu trúc Dense AE cực kỳ phù hợp. Kiến trúc nông và nhỏ gọn này giúp giải quyết hiệu quả bài toán tiết kiệm dung lượng bộ nhớ Flash và SRAM vốn rất hạn hẹp trên vi điều khiển [1].

1.3.2 Kiến trúc 1D-CNN và các giới hạn trên nền tảng nhúng

Khác với Dense AE, mạng nơ-ron tích chập một chiều (1D-CNN) được thiết kế chuyên biệt để khai thác tính tương quan cục bộ trên dữ liệu chuỗi thời gian. Kiến trúc này sử dụng các bộ lọc (kernels) trượt dọc theo trục thời gian để trích xuất đặc trưng. Phép toán tích chập tại một vị trí t với bộ lọc w có kích thước k được biểu diễn như Phương trình 1.5:

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-i} + b \quad (1.5)$$

Về mặt lý thuyết, 1D-CNN AE rất hiệu quả trong việc nhận diện các mẫu rung động tuần hoàn hoặc các xung dị thường. Tuy nhiên, khi triển khai thực tế trên vi điều khiển, kiến trúc này bộc lộ những hạn chế lớn về mặt hiệu năng. Phép toán tích chập đòi hỏi số lượng lớn các phép nhân-cộng tích lũy (MACs), làm tăng

¹ Mel-Filterbank Energy (MFE) là kỹ thuật xử lý tín hiệu âm thanh mô phỏng cách cảm nhận tần số của tai người, bằng cách nhóm các phổ tần số thu được từ biến đổi Fourier thành các dải năng lượng logarit. Chi tiết về phương pháp này sẽ được trình bày ở Mục 1.5.1.

độ trễ suy luận và tiêu tốn nhiều không gian bộ nhớ SRAM [3, 8]. Thêm vào đó, kết quả thực nghiệm của đề tài (sẽ được phân tích chi tiết tại Chương 4) còn chỉ ra một thách thức khác: sai số tái tạo của 1D-CNN AE biến động rất mạnh khi ép kiểu trọng số từ Float32 sang INT8. Điều này khiến hệ thống khó duy trì được độ tin cậy sau quá trình lượng tử hóa [3].

1.3.3 Lựa chọn kiến trúc tối ưu cho bài toán giám sát thiết bị gia dụng

Thiết kế mô hình Trí tuệ nhân tạo tại biên (Edge AI) luôn là bài toán đánh đổi giữa độ nhạy phát hiện lỗi, độ trễ suy luận và tài nguyên bộ nhớ. Vì vậy, ở giai đoạn đầu, đồ án tiến hành so sánh ba kiến trúc mạng tự mã hóa trên môi trường mô phỏng Kaggle gồm Vanilla AE, 1D-CNN AE và LSTM AE. Các mô hình được huấn luyện trên cùng tập dữ liệu cảm biến đã tiền xử lý, sau đó được đánh giá theo sai số tái tạo, dung lượng sau lượng tử hóa INT8, độ trễ suy luận dự kiến tính toán trên Kaggle và mức ổn định khi thêm nhiễu.

Kết quả mô phỏng ban đầu xếp hạng hiệu quả theo thứ tự 1D-CNN AE, Vanilla AE và LSTM AE. Cần nhấn mạnh rằng các giá trị dung lượng và độ trễ trong giai đoạn này là số liệu dự kiến được tính toán trên Kaggle sau khi quy đổi/lượng tử hóa mô hình, chưa phải kết quả đo trực tiếp trên phần cứng. LSTM AE bị loại sớm do độ trễ suy luận dự kiến cao nhất, khoảng 129,50 ms, cùng dung lượng sau nén INT8 khoảng 368,66 KB. Vanilla AE có cấu trúc đơn giản hơn nhưng sai số tái tạo lớn nhất, với MSE đạt 0,004947, dung lượng INT8 dự kiến khoảng 375,01 KB và chỉ số bất ổn với nhiễu lên tới 793,63%. Trong khi đó, 1D-CNN AE cho kết quả tốt nhất ở giai đoạn mô phỏng, với MSE 0,003698 và kích thước mô hình dự kiến khoảng 30,52 KB, nên ban đầu được xem là ứng viên chính cho hệ thống.

Tuy nhiên, khi chuyển sang giai đoạn triển khai trên TensorFlow Lite Micro, 1D-CNN AE bộc lộ các rào cản kỹ thuật nghiêm trọng. Kết quả suy luận giữa môi trường Python và C++ không khớp do khác biệt trong triển khai toán tử Conv1D; đầu vào dạng chuỗi làm tăng chi phí quản lý bộ đệm trên RAM; đồng thời quá trình lượng tử hóa INT8 gây rủi ro mất ổn định tham số. Từ kết quả này, đồ án chuyển sang hướng thiết kế Symmetric Dense AE kết hợp khối trích xuất đặc trưng DSP 19 chiều. Kiến trúc đối xứng giúp giảm độ phức tạp đồ thị suy luận, phù hợp hơn với giới hạn bộ nhớ của vi điều khiển Seeed Studio XIAO ESP32-S3 và trở thành kiến trúc cốt lõi cho hệ thống cuối cùng.

1.4 Công nghệ TinyML trên vi điều khiển

TinyML (Tiny Machine Learning) là hướng tiếp cận đưa các mô hình học máy xuống thực thi trực tiếp trên các hệ thống nhúng và vi điều khiển với mức tiêu thụ điện năng chỉ vài miliwatt [3]. Việc tích hợp xử lý tín hiệu số (DSP) và mạng nơ-ron sâu ngay trên các thiết bị có tài nguyên hạn chế giúp dịch chuyển năng lực tính toán ra sát nguồn phát sinh dữ liệu. Phương pháp này giúp khắc phục hiệu quả các hạn chế về độ trễ, băng thông truyền tải và rủi ro bảo mật dữ liệu so với mô hình điện toán đám mây truyền thống [1].

1.4.1 Ràng buộc về tài nguyên phần cứng

Triển khai mạng nơ-ron trên thiết bị biên như Seeed Studio XIAO ESP32-S3 yêu cầu tính toán kỹ lưỡng các giới hạn vật lý của phần cứng. Khác với môi trường đám mây, vi điều khiển phải đối mặt với ba rào cản thiết kế chính. Thứ nhất là giới hạn nghiêm ngặt về bộ nhớ, bao gồm bộ nhớ Flash để lưu trữ trọng số và bộ nhớ SRAM dành cho việc cấp phát tensor. Mặc dù ESP32-S3 sở hữu vi xử lý lõi kép Xtensa LX7 mạnh mẽ, dung lượng SRAM nội (internal SRAM) chỉ ở mức 512 KB [9]. Mức dung lượng này phải được chia sẻ đồng thời cho hệ điều hành, các ngăn xếp tác vụ (task stacks), bộ đệm truyền thông không dây và vùng nhớ cho mạng nơ-ron. Do đó, các mô hình học sâu buộc phải được nén tối ưu để tránh lỗi tràn bộ nhớ (Out of Memory) [8].

Thứ hai là ràng buộc về độ trễ thực thi (Inference Latency). Trong bài toán giám sát máy giặt, tín hiệu rung động được lấy mẫu với tần số cao lên tới 1000 Hz. Điều này đòi hỏi mô hình phải hoàn thành quá trình suy luận (forward pass) trong một cửa sổ thời gian hẹp để đảm bảo tính thời gian thực, tránh gây thất cổ chai dữ liệu. Cuối cùng là ràng buộc về năng lượng và năng lực xử lý; mô hình AI không được chiếm dụng quá nhiều chu kỳ CPU để đảm bảo hệ thống có thể duy trì đồng thời các tác vụ quản lý cảm biến và giao tiếp Wi-Fi/MQTT [3].

1.4.2 Kỹ thuật lượng tử hóa mô hình (Quantization INT8)

Để vượt qua các giới hạn khắt khe về bộ nhớ và năng lực tính toán, lượng tử hóa nguyên 8-bit (INT8 Quantization) là kỹ thuật nén mô hình cốt lõi được áp dụng. Kỹ thuật này chuyển đổi các trọng số (weights) và giá trị kích hoạt

(activations) từ định dạng số thực dấu phẩy động 32-bit (Float32) sang số nguyên 8-bit. Quá trình ánh xạ tuyến tính từ miền số thực r sang miền số nguyên q được mô tả qua Phương trình 1.6:

$$r = S(q - Z) \quad (1.6)$$

Trong đó, S (Scale) là số thực dương đại diện cho tỷ lệ thu phóng, và Z (Zero-point) là giá trị số nguyên dùng để dịch tâm, tương ứng với giá trị 0 trong miền số thực [10]. Lượng tử hóa INT8 giúp giảm kích thước lưu trữ của mô hình đi khoảng 4 lần so với Float32, đồng thời giảm lượng RAM tiêu thụ trong quá trình suy luận (inference) theo tỷ lệ tương ứng [3, 10]. Các nghiên cứu đã chỉ ra rằng khi kết hợp với phương pháp lượng tử hóa sau huấn luyện (Post-Training Quantization), mức suy giảm độ chính xác của mạng thường được kiểm soát dưới 1% [10]. Hơn nữa, với kiến trúc lõi LX7 của ESP32-S3, dữ liệu kiểu INT8 cho phép vi điều khiển khai thác tối đa tập lệnh vector (Vector Instructions) chuyên dụng. Điều này giúp thực thi nhiều phép toán nhân-cộng tích lũy (MACs) song song, từ đó hệ thống có thể đạt được độ trễ suy luận khoảng 6,4 ms trên kiến trúc XIAO ESP32-S3 [1, 9].

1.4.3 Framework TensorFlow Lite for Microcontrollers

Hệ thống sử dụng TensorFlow Lite for Microcontrollers (TFLite Micro) để nhúng trực tiếp mô hình tự mã hóa lượng tử hóa xuống vi điều khiển. TFLite Micro được thiết kế tối ưu cho các thiết bị biên, loại bỏ sự phụ thuộc vào hệ điều hành tiêu chuẩn và không yêu cầu cơ chế cấp phát bộ nhớ động. Việc không sử dụng các lệnh `malloc()` hay `new` trong quá trình chạy mô hình giúp ngăn ngừa rủi ro phân mảnh bộ nhớ, đảm bảo độ tin cậy cho hệ thống khi vận hành liên tục dài ngày [11].

Framework này hoạt động bằng cách biên dịch mô hình thành một mảng byte không đổi (const byte array) lưu trên bộ nhớ Flash. Sau đó, một bộ thông dịch (Interpreter) sẽ điều phối quá trình tính toán thông qua một vùng đệm cấp phát sẵn duy nhất gọi là vùng đệm tensor (Tensor Arena) [11]. Việc tích hợp TFLite Micro trên hệ điều hành thời gian thực FreeRTOS cho phép hệ thống thực thi đa luồng độc lập. Dù tác vụ suy luận AI được gán mức ưu tiên thấp hơn các tác vụ thu thập dữ liệu phần cứng, kiến trúc đa nhân vẫn đảm bảo quá trình này đáp

ứng tốt các yêu cầu về thời gian thực. Để mô hình TFLite Micro hoạt động chính xác, dữ liệu đầu vào cần đi qua một luồng xử lý tín hiệu số tối ưu, nội dung này sẽ được trình bày trong mục tiếp theo.

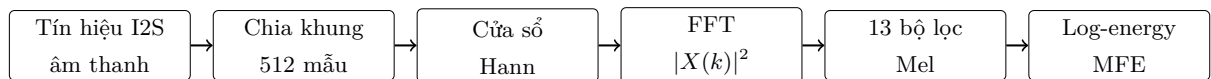
1.5 Cơ sở xử lý tín hiệu cảm biến

Trong các hệ thống giám sát tại biên, tiền xử lý và trích xuất đặc trưng là bước quan trọng ảnh hưởng trực tiếp đến hiệu suất của mô hình. Thay vì đẩy trực tiếp dữ liệu thô với số chiều lớn vào mạng nơ-ron, quy trình trích xuất đặc trưng giúp giữ lại các thông tin cốt lõi, giảm nhiễu và hạ thấp đáng kể yêu cầu tính toán. Đối với bài toán giám sát máy giặt, hệ thống khai thác đồng thời hai nguồn dữ liệu: tín hiệu âm thanh để nhận diện biến động tần số cao và tín hiệu rung động để theo dõi trạng thái cơ khí ở tần số thấp.

1.5.1 Trích xuất đặc trưng âm thanh

Tín hiệu âm thanh thu từ microphone INMP441 qua bus I2S có dạng chuỗi thời gian liên tục. Để mô hình tự mã hóa nhận diện được các âm thanh bất thường như lỗi ma sát hay tiếng rít động cơ, kỹ thuật năng lượng dải lọc Mel được áp dụng. Phương pháp này mô phỏng thính giác con người — nhạy cảm với các thay đổi ở dải tần số thấp hơn là tần số cao. Phép biến đổi từ tần số Hz sang thang đo Mel được tính theo Phương trình 1.7:

$$M(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right) \quad (1.7)$$



Hình 1.2

Đường ống trích xuất đặc trưng âm thanh MFE.

Quy trình xử lý trong Hình 1.2 bắt đầu bằng việc chia tín hiệu thành các khung dài 64 ms, tương đương 512 mẫu ở tần số lấy mẫu 8 kHz, trượt với bước nhảy 32 ms. Các tham số này được lựa chọn dựa trên sự cân bằng giữa khả năng giữ lại thông tin âm học và giới hạn tài nguyên của vi điều khiển. Tần số lấy mẫu 8 kHz cho phép giữ lại dải âm thanh 0–4 kHz theo định lý Nyquist, đủ bao phủ các

thành phần tiếng ồn cơ khí chính của động cơ, vòng bi và lồng giặt, trong khi giảm đáng kể dung lượng bộ đệm và số phép tính so với các mức 16 kHz hoặc 44,1 kHz [12, 13]. Với 8 kHz, khung 512 mẫu tương ứng 64 ms và đồng thời là kích thước lũy thừa của 2, phù hợp để thực hiện FFT hiệu quả. Cấu hình này tạo độ phân giải tần số khoảng $8000/512 \approx 15,6$ Hz, đủ chi tiết để quan sát sự thay đổi phổ của tiếng ma sát hoặc tiếng rít cơ khí; nếu khung ngắn hơn, độ phân giải tần số giảm, còn nếu khung dài hơn, độ trễ và bộ nhớ đệm tăng không phù hợp với xử lý tại biên.

Bước nhảy 32 ms tạo mức chồng lấn 50% giữa hai khung liên tiếp. Mức chồng lấn này giúp đặc trưng phổ thay đổi mượt hơn theo thời gian, hạn chế bỏ sót các bất thường ngắn nhưng vẫn không làm tăng gấp đôi tải tính toán như các mức chồng lấn dày hơn. Trước khi biến đổi, cửa sổ Hann được nhân với từng khung để làm mềm hai biên của đoạn tín hiệu, qua đó giảm hiện tượng rò rỉ phổ khi FFT giả định tín hiệu trong mỗi khung có tính tuần hoàn [13]. Nhờ vậy, các đặc trưng năng lượng Mel ổn định hơn và ít bị ảnh hưởng bởi nhiễu biên khung.

Sau đó, phép biến đổi Fourier nhanh (FFT) chuyển tín hiệu sang miền tần số, và phần năng lượng phổ được đưa qua dải 13 bộ lọc hình tam giác xếp chồng theo thang Mel [14]. Số lượng 13 bộ lọc được lựa chọn nhằm cân bằng giữa khả năng phân giải phổ và kích thước vectơ đầu vào. Giá trị này đủ để mô tả các vùng năng lượng chính của tiếng ồn cơ khí, đồng thời giữ không gian đặc trưng âm thanh ở mức nhỏ để khi kết hợp với 6 đặc trưng rung động, vectơ đầu vào cuối cùng chỉ có 19 chiều và phù hợp với vùng nhớ hạn chế của TensorFlow Lite Micro trên vi điều khiển.

Đầu ra của mỗi bộ lọc là một giá trị năng lượng logarit. Với bộ lọc Mel thứ m , đặc trưng năng lượng được tính theo Phương trình 1.8:

$$E_m = \log \left(\sum_{k=0}^{K-1} |X(k)|^2 H_m(k) + \epsilon \right) \quad (1.8)$$

Trong đó, $X(k)$ là phổ sau FFT, $H_m(k)$ là đáp ứng của bộ lọc Mel thứ m , K là số điểm tần số được xét và ϵ là hằng số nhỏ để tránh phép tính $\log(0)$. Đồ án chủ đích chọn MFE thay vì MFCC (Mel-Frequency Cepstral Coefficients) nhằm loại bỏ bước biến đổi Cosin ngược (DCT). Lựa chọn này giúp tiết kiệm chu kỳ CPU trên vi điều khiển trong khi vẫn bảo toàn đủ thông tin không gian phổ để phát hiện bất thường [3, 14].

1.5.2 Trích xuất đặc trưng rung động cơ học

Tín hiệu rung động từ cảm biến gia tốc ADXL345 phản ánh trực tiếp các sự cố cơ học như mất cân bằng tải hay lệch trục lồng giạt. Vì các lỗi này thường bộc lộ rõ qua cường độ và độ phân tán của dao động trong miền thời gian, đồ án tiến hành trích xuất hai đặc trưng thống kê là Giá trị hiệu dụng (RMS) và Phương sai (Variance) cho từng trục (X, Y, Z).

Giá trị RMS đo lường mức năng lượng trung bình của rung động, được tính theo Phương trình 1.9:

$$x_{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (1.9)$$

Bên cạnh đó, Phương sai đánh giá mức độ biến thiên của tín hiệu quanh giá trị trung bình. Việc kết hợp phương sai giúp mô hình nhạy hơn với các xung và đập bất thường do mất cân bằng tải gây ra [2]. Công thức tính phương sai được mô tả tại Phương trình 1.10:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2 \quad (1.10)$$

Chỉ sử dụng các đặc trưng miền thời gian giúp hệ thống không phải tính toán các phép biến đổi miền phức tạp (như FFT), qua đó rút ngắn tối đa độ trễ suy luận và tiết kiệm dung lượng SRAM trên vi điều khiển Seeed Studio XIAO ESP32-S3 [8].

1.5.3 Kỹ thuật kết hợp dữ liệu đa phương thức

Để đánh giá đầy đủ trạng thái vận hành của thiết bị, đồ án áp dụng kỹ thuật kết hợp dữ liệu ở mức đặc trưng. Phương pháp này giúp mô hình bắt được mối tương quan chéo giữa âm thanh và rung động; chẳng hạn, tiếng ồn tần số cao đi kèm với biên độ rung lớn thường là dấu hiệu rõ ràng của lỗi ở pha vát.

Vectơ đầu vào $V_{input} \in \mathbb{R}^{19}$ được tạo ra bằng cách ghép các đặc trưng theo Phương trình 1.11:

$$V_{input} = [MFE_{1...13} \oplus RMS_{x,y,z} \oplus \sigma_{x,y,z}^2] \quad (1.11)$$

Trong đó, toán tử $\oplus$ biểu diễn phép ghép nối, gộp không gian 13 chiều MFE và 6 chiều rung động thành một vectơ đặc trưng 19 chiều duy nhất.

Thao tác quan trọng nhất ở bước này là sử dụng **Chuẩn hóa hỗn hợp** để tinh chỉnh từng loại dữ liệu trước khi nạp vào mạng tự mã hóa. Với đặc trưng âm thanh MFE, do cường độ luôn bị giới hạn vật lý bởi độ nhạy của microphone, hệ thống dùng **Min-Max Scaling** để ánh xạ dải $[-80, 0]$ dB về khoảng $[0, 1]$. Ngược lại, dữ liệu rung động (RMS và phương sai) thường xuyên xuất hiện các đỉnh nhiễu cao đột biến khi có va đập. Để xử lý, hệ thống dùng **Robust Scaling** — phương pháp chuẩn hóa dựa trên trung vị và dải phân vị. Cách này đưa dữ liệu về cùng thang đo chuẩn mà không làm lu mờ thông tin của các đỉnh giá trị bất thường [7].

Đường ống tiền xử lý và chuẩn hóa hỗn hợp này không chỉ cung cấp dữ liệu sạch cho mô hình trí tuệ nhân tạo mà còn là nền tảng để hệ thống chạy đa nhân song song ổn định trên FreeRTOS. Các nghiên cứu liên quan và những hạn chế mà hệ thống này khắc phục sẽ được tổng kết ở cuối chương.

1.6 Các công trình liên quan và khoảng trống nghiên cứu

1.6.1 Các công trình liên quan trong lĩnh vực giám sát và phát hiện bất thường

Giám sát tình trạng thiết bị và phát hiện bất thường cho các hệ thống máy móc là một bài toán quan trọng trong công nghiệp. Hầu hết các hệ thống giám sát Internet vạn vật hiện nay đều vận hành dựa trên kiến trúc điện toán đám mây. Trong đó, thiết bị biên đóng vai trò thu thập và truyền dữ liệu thô qua Internet về máy chủ để phân tích. Dù tận dụng được năng lực tính toán lớn, phương pháp này bộc lộ nhiều hạn chế khi áp dụng vào thiết bị gia dụng như: độ trễ phản hồi, sự phụ thuộc vào băng thông mạng, rủi ro rò rỉ dữ liệu cá nhân và chi phí hạ tầng [3, 1].

Gần đây, sự phát triển của công nghệ TinyML đã thúc đẩy xu hướng dịch chuyển trí tuệ nhân tạo ra vùng biên. Nghiên cứu của Mansoureh Lord (2021) [15] đã chứng minh tính khả thi của việc chạy trực tiếp các thuật toán học máy trên vi điều khiển nhằm phát hiện dị thường. Tác giả đã thử nghiệm kiến trúc mạng nơ-ron tự mã hóa và biến phân tự mã hóa (Variational Autoencoder - VAE), trong đó mô hình tự mã hóa đạt độ chính xác 92% khi nhận diện lỗi qua sai số tái tạo.

Bên cạnh đó, một số nghiên cứu cũng đã triển khai các thuật toán máy học truyền thống như máy véc-tơ hỗ trợ (Support Vector Machine - SVM) hay rừng ngẫu nhiên trên thiết bị biên để giám sát cơ khí thời gian thực [6].

Ở hướng học sâu không giám sát trên thiết bị gia dụng, Castangia và cộng sự (2022) [16] đề xuất sử dụng VAE để phát hiện bất thường điện dựa trên chữ ký công suất của máy rửa bát, máy giặt và máy sấy. Nhóm tác giả tạo các điểm bất thường ngẫu nhiên nhằm mô phỏng lỗi điện thực tế, sau đó so sánh VAE với One-Class SVM sử dụng hai đặc trưng thủ công là năng lượng tiêu thụ và thời lượng chu kỳ. Kết quả cho thấy VAE đạt F1-Score lớn hơn 0,90 trên toàn bộ các thiết bị thử nghiệm, chứng minh lợi thế của học sâu trong việc mô hình hóa chu kỳ vận hành bình thường so với các đặc trưng thủ công.

Đối với riêng máy giặt, Shul và cộng sự (2023) [12] khai thác phổ tiếng ồn vận hành để phát hiện bất thường bằng mạng DNN tự giám sát. Công trình này nhấn mạnh rằng tiếng ồn của máy giặt thay đổi mạnh theo khối lượng đồ giặt và độ mất cân bằng, khiến bài toán phát hiện bất thường khó hơn trong điều kiện thực tế. Để xử lý bối cảnh vận hành, nhóm tác giả sử dụng thêm thông tin tốc độ quay của lồng và khối lượng đồ giặt, huấn luyện mô hình dự đoán phổ tiếng ồn tương lai từ các phổ quá khứ và kiểm chứng chéo trên nhiều loại vải, khối lượng khác nhau. Mô hình đề xuất cải thiện ROC-AUC tối đa khoảng 1,4% trên toàn bộ tập kiểm thử, cho thấy tiềm năng của dữ liệu âm thanh nhưng vẫn đòi hỏi mô hình DNN phức tạp và thông tin vận hành nội bộ từ máy giặt.

Riêng với đối tượng máy giặt, các công trình hiện hành, tiêu biểu như của Mohammadi (2020) [2], chủ yếu phân tích phổ rung động để phát hiện lỗi mất cân bằng tải. Tuy nhiên, các nghiên cứu này đa phần dựa trên học có giám sát – phương pháp đòi hỏi tập dữ liệu có nhãn lớn về các trạng thái lỗi. Điều này gây tốn kém chi phí và khó thực hiện trong điều kiện vận hành thiết bị gia dụng thực tế.

Hơn nữa, việc đồng bộ hóa dữ liệu từ nhiều cảm biến (âm thanh, rung động) đòi hỏi năng lực xử lý tín hiệu cao. Trong các vi điều khiển đơn nhân, việc luân phiên giữa tác vụ đọc cảm biến liên tục và suy luận AI thường gây ra hiện tượng trôi thời gian (time drift), dẫn đến mất mát dữ liệu đầu vào khi hệ thống xử lý các tác vụ có mức ưu tiên xung đột nhau [8].

1.6.2 Khoảng trống nghiên cứu và đóng góp của đề án

Từ các phân tích trên, đề án nhận diện ba khoảng trống nghiên cứu khi áp dụng TinyML vào bài toán Phát hiện bất thường cho máy giặt, qua đó đề xuất các giải pháp kỹ thuật cụ thể:

Thứ nhất, rào cản về cảnh báo giả do tính chất đa trạng thái của thiết bị. Máy giặt vận hành qua nhiều pha có đặc tính vật lý thay đổi liên tục. Do đó, đề án đề xuất cơ chế Phân luồng đa trạng thái tự động (Chuyển đổi Ba trạng thái), giúp nhận diện pha vận hành theo thời gian thực từ dữ liệu chấn động, từ đó định tuyến dòng dữ liệu vào các mô hình học máy chuyên biệt (GENTLE, STRONG, SPIN). Hướng tiếp cận này giúp giảm thiểu tình trạng cảnh báo giả khi thiết bị chuyển pha.

Thứ hai, giới hạn về dữ liệu lỗi và không gian bộ nhớ của vi điều khiển. Đề án áp dụng kiến trúc mạng nơ-ron đối xứng không giám sát [4], chỉ yêu cầu học từ phân phối dữ liệu bình thường. Để tối ưu hóa cho vi điều khiển S32K3, kỹ thuật lượng tử hóa nhận thức huấn luyện được tích hợp, giúp nén dung lượng mô hình nhưng vẫn duy trì độ chính xác trên miền số nguyên INT8.

Thứ ba, thách thức về độ trễ và mất đồng bộ khi sử dụng đa cảm biến. Việc đồng thời trích xuất đặc trưng phổ âm thanh (I2S, Mel-filterbank) và đọc dữ liệu gia tốc (I2C) dễ làm gián đoạn luồng suy luận. Đề án giải quyết vấn đề này bằng cách thiết kế hệ thống phần mềm xử lý song song thực sự trên nền tảng FreeRTOS. Tác vụ xử lý âm thanh được phân luồng vào lõi số 0, trong khi việc thu thập rung động và suy luận AI chạy trên lõi số 1. Dữ liệu giữa các luồng được đồng bộ hóa an toàn qua Nhóm sự kiện và Hàng đợi, ngăn ngừa hiện tượng thất cổ chai phần cứng.

Thông qua việc giải quyết đồng thời ba vấn đề trên, hệ thống được đề xuất hướng tới việc đảm bảo tính chính xác và độ phản hồi theo thời gian thực, cung cấp một phương pháp tiếp cận phù hợp cho các ứng dụng trí tuệ nhân tạo tại biên đa phương thức.

THIẾT KẾ HỆ THỐNG

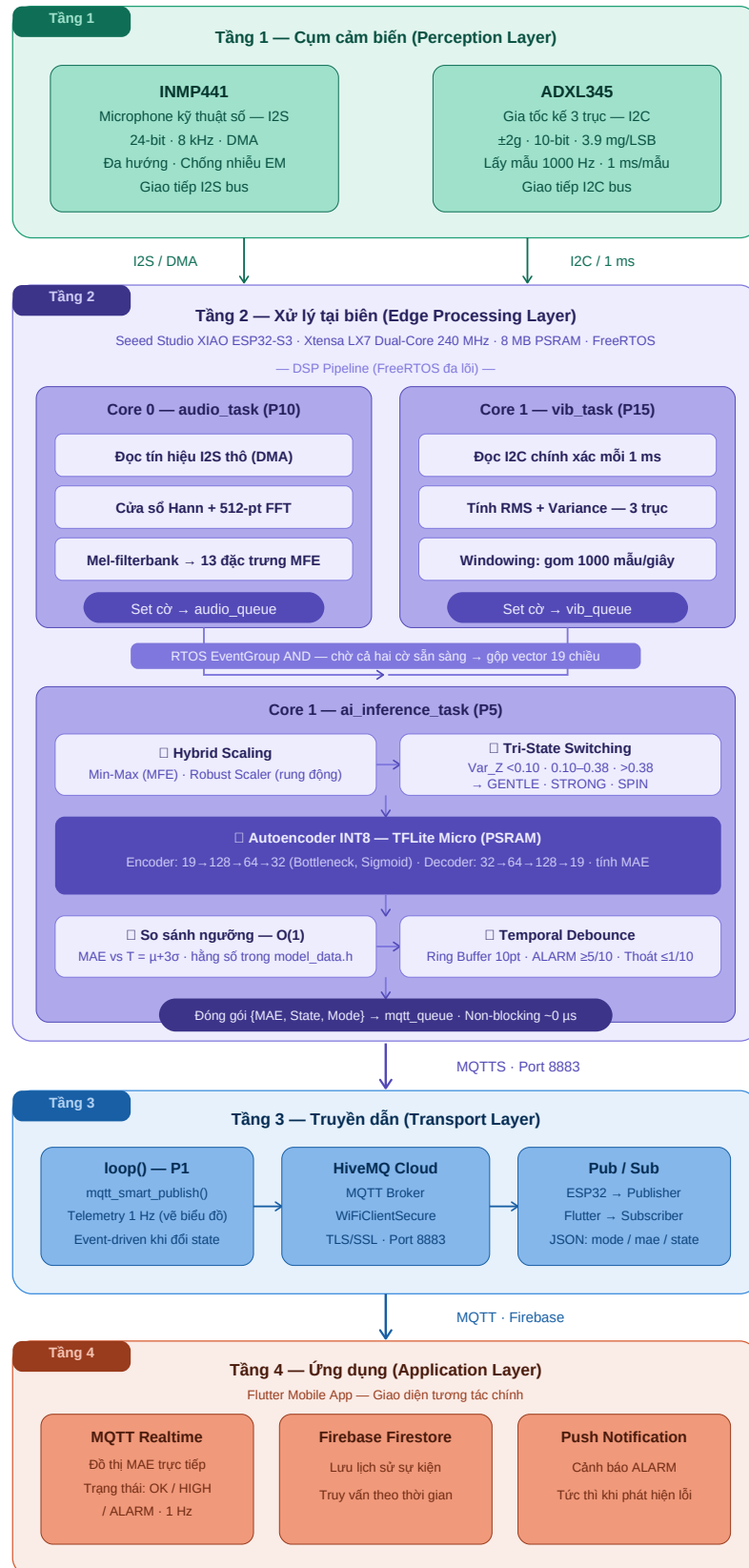
2.1 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc AIoT định hướng biên, tập trung vào khả năng xử lý tại chỗ nhằm đảm bảo tính thời gian thực. Cấu trúc hệ thống được chia thành bốn tầng chức năng chính để thuận tiện cho việc quản lý và mở rộng (Hình 2.1).

2.1.1 Sơ đồ khối chức năng

Các khối thành phần trong hệ thống tương tác theo luồng dữ liệu một chiều để tối ưu hóa thời gian phản hồi:

- **Tầng cảm biến:** Gồm microphone kỹ thuật số INMP441 (I2S) thu tín hiệu âm thanh và cảm biến gia tốc ADXL345 (I2C) đo rung động cơ học của lồng giặt.
- **Tầng xử lý tại biên:** Sử dụng vi điều khiển Seeed Studio XIAO ESP32-S3 làm đơn vị xử lý trung tâm. Tại đây, dữ liệu thô được đưa qua đường ống xử lý tín hiệu số, trích xuất thành vector đặc trưng 19 chiều và nạp vào mô hình mạng nơ-ron tự mã hóa lượng tử hóa INT8 để suy luận [15]. Hệ thống tận dụng kiến trúc lõi kép để thực hiện song song các tác vụ thu thập dữ liệu và tính toán mạng nơ-ron.
- **Tầng truyền dẫn:** Sử dụng giao thức MQTTs qua cổng 8883 để kết nối với HiveMQ Cloud. Đây là lớp trung gian điều phối thông tin từ thiết bị biên đến ứng dụng người dùng một cách bảo mật qua lớp mã hóa TLS/SSL.
- **Tầng ứng dụng:** Ứng dụng di động phát triển trên nền tảng Flutter đóng



Hình 2.1

Sơ đồ kiến trúc tổng thể hệ thống bao gồm 4 tầng chức năng: Tầng cảm biến, Tầng xử lý tại biên, Tầng truyền dẫn và Tầng ứng dụng.

vai trò giao diện tương tác chính. Ứng dụng tích hợp đồng bộ dữ liệu thời gian thực qua MQTT và dữ liệu lịch sử qua Firebase Firestore. Cơ chế thông báo đẩy được sử dụng để cảnh báo người dùng kịp thời về các sự cố hỏng hóc.

2.1.2 Đường ống xử lý luồng dữ liệu

Quy trình xử lý dữ liệu được thiết kế theo cấu trúc luồng tuyến tính, đảm bảo tính toàn vẹn từ dữ liệu vật lý đến quyết định logic:

1. Thu thập và Tiền xử lý: Tín hiệu âm thanh đi qua bộ lọc cửa sổ Hann và phép biến đổi Fourier nhanh 512 điểm để trích xuất 13 đặc trưng bộ lọc Mel. Song song đó, tín hiệu rung động (lấy mẫu ở tốc độ 1000 mẫu/giây) được tính toán giá trị hiệu dụng và phương sai để làm cơ sở phân tích trạng thái.
2. Định tuyến trạng thái: Dựa trên phương sai trục Z của rung động, hệ thống tự động xác định máy đang vận hành ở pha nào (GENTLE, STRONG, hoặc SPIN). Từ đó, vi điều khiển thực hiện nạp và kích hoạt mô hình AI chuyên biệt tương ứng lên vùng đệm tensor.
3. Suy luận AI và so sánh ngưỡng: Vector đặc trưng được đưa qua lớp chuẩn hóa kép trước khi vào mô hình mạng nơ-ron tự mã hóa để tính toán lỗi tái tạo (MAE). Hệ thống sử dụng các ngưỡng phát hiện dị thường đã được tối ưu hóa riêng biệt cho từng pha vận hành. Các giá trị này được nhúng trực tiếp dưới dạng hằng số tĩnh nhằm triệt tiêu độ trễ tính toán, cho phép nhận diện lỗi với độ phức tạp $\mathcal{O}(1)$.
4. Hậu xử lý: Để triệt tiêu báo động giả do nhiễu cơ học tức thời, hệ thống áp dụng cơ chế bộ lọc cửa sổ trượt có nhớ và bộ đệm vòng lưu giữ 10 trạng thái gần nhất. Thuật toán trễ được tích hợp để thiết lập bộ quy tắc bất đối xứng: hệ thống chỉ chuyển sang trạng thái ALARM khi có ít nhất 5/10 mẫu vượt ngưỡng, và chỉ khôi phục về trạng thái bình thường khi tỷ lệ lỗi giảm xuống mức tối thiểu ($\leq 1/10$). Cơ chế có trạng thái này đảm bảo độ tin cậy tuyệt đối cho hệ thống.

2.1.3 Cơ chế điều phối đa nhiệm trên FreeRTOS

Đồ án triển khai kiến trúc song song thực sự trên lõi kép của FreeRTOS nhằm triệt tiêu hiện tượng mất đồng bộ thời gian trong kỹ thuật kết hợp dữ liệu

đa phương thức:

- Lỗi số 0: Chuyên trách tiền xử lý tín hiệu. Tác vụ `audio_task` với mức ưu tiên 10 thực hiện đọc I2S và liên tục thực thi khối lệnh FFT/bộ lọc Mel nặng về tính toán dấu phẩy động.
- Lỗi số 1: Đảm nhiệm hai tác vụ quan trọng. Tác vụ `vib_task` được cấp đặc quyền cao nhất trong nhóm tác vụ người dùng với mức ưu tiên 15 nhằm đảm bảo chu kỳ lấy mẫu phần cứng I2C chính xác ở mức 1 ms. Tác vụ `ai_inference_task` với mức ưu tiên 5 chịu trách nhiệm vận hành mô hình TFLite Micro và chạy thuật toán phân tích cảnh báo.

Việc đồng bộ hóa giữa các lỗi được thực thi qua cơ chế nhóm sự kiện (đảm bảo logic cổng AND - AI chỉ chạy khi cả hai cảm biến đã sẵn sàng) và hàng đợi luồng an toàn. Đặc biệt, hệ thống cô lập hoàn toàn khâu truyền dẫn mạng khỏi tiến trình AI. Tác vụ AI chỉ làm nhiệm vụ đóng gói cảnh báo và đẩy vào `mqtt_queue` qua thao tác không chặn (mất $\sim 0 \mu s$). Hàm `loop()` nội tại của vi điều khiển, chạy ngầm ở mức ưu tiên 1, đóng vai trò tiêu thụ dữ liệu từ hàng đợi này và xử lý các gián đoạn I/O mạng. Thiết kế này giúp triệt tiêu hoàn toàn rủi ro độ trễ mạng làm treo cứng tiến trình giám sát AI tại biên.

2.2 Thiết kế phần cứng

Hệ thống phần cứng được thiết kế hướng tới sự nhỏ gọn và tiết kiệm năng lượng, đồng thời cần đảm bảo năng lực tính toán đủ để thực thi song song các luồng xử lý tín hiệu và mạng nơ-ron.

2.2.1 Nền tảng vi điều khiển lõi kép XIAO ESP32-S3

Trung tâm điều khiển và xử lý suy luận của hệ thống là mô-đun Seeed Studio XIAO ESP32-S3. Việc lựa chọn nền tảng này thay vì các dòng vi điều khiển truyền thống được dựa trên ba yếu tố kỹ thuật chính:

- Kiến trúc đa nhân và tập lệnh gia tốc Trí tuệ nhân tạo (AI): Chip được trang bị vi xử lý Xtensa® 32-bit LX7 lõi kép hoạt động ở xung nhịp 240 MHz. Dòng S3 có hỗ trợ các tập lệnh mở rộng vector, giúp thực thi các phép toán nhân-

tích lũy (MAC) hiệu quả hơn, phù hợp cho việc gia tốc suy luận với mô hình TensorFlow Lite Micro [9].

- Hệ thống bộ nhớ phân tầng: Hệ thống sử dụng phiên bản có tích hợp sẵn 8 MB PSRAM (Pseudo-Static RAM) và 8 MB bộ nhớ Flash bên cạnh 512 KB SRAM nội bộ [37]. Dung lượng PSRAM mở rộng đóng vai trò là vùng đệm cho phép hệ thống nạp cùng lúc ba mô hình mạng nơ-ron tự mã hóa (GENTLE, STRONG, SPIN), hỗ trợ thiết bị chuyển đổi mô hình linh hoạt mà không gây quá tải bộ nhớ.
- Kết nối Internet vạn vật (IoT): Tích hợp sẵn Wi-Fi 2.4 GHz, cung cấp băng thông đủ để triển khai các giao thức truyền tải mã hóa (WiFiClientSecure) kết nối với máy chủ đám mây.

2.2.2 Tích hợp microphone kỹ thuật số INMP441 qua chuẩn giao tiếp I2S

Để thu thập đặc trưng âm thanh, hệ thống sử dụng microphone kỹ thuật số đa hướng INMP441. Đây là cảm biến MEMS có các đặc điểm phù hợp với môi trường dễ nhiễu động như máy giặt:

- Giao tiếp I2S (Inter-IC Sound): Tín hiệu âm thanh được số hóa trực tiếp tại cảm biến với độ phân giải 24-bit và truyền về vi điều khiển Seeed Studio XIAO ESP32-S3 qua bus I2S [17]. Cách tiếp cận này giúp hạn chế nhiễu điện từ môi trường — một vấn đề thường gặp ở các microphone analog khi hoạt động gần động cơ công suất lớn.
- Cơ chế truy cập bộ nhớ trực tiếp (DMA): Luồng dữ liệu âm thanh lấy mẫu ở tần số 8 kHz được đẩy thẳng vào vùng nhớ SRAM thông qua bộ điều khiển DMA. Kỹ thuật này giúp hệ thống thu thập dữ liệu liên tục mà không làm tăng đáng kể tải xử lý của CPU, cho phép lõi số 0 tập trung tài nguyên vào khâu tính toán đặc trưng bộ lọc Mel.

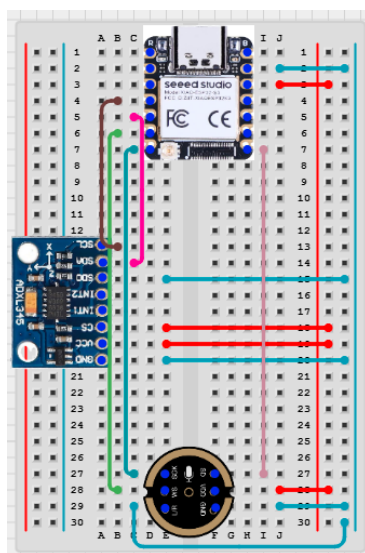
2.2.3 Tích hợp cảm biến gia tốc ADXL345 qua chuẩn giao tiếp I2C

Sự rung động vật lý của lồng giặt là dữ liệu quan trọng để nhận diện các trạng thái bất thường cơ học. Cảm biến gia tốc ba trục ADXL345 được tích hợp với cấu hình như sau:

- Độ phân giải và dải đo: Cảm biến được thiết lập ở dải đo $\pm 2g$. Với độ phân giải 10-bit (tương đương mức độ nhạy 3,9 mg/LSB) [18], thiết bị có thể ghi nhận được các biến động nhỏ của lồng giặt, hỗ trợ thu thập dữ liệu đầu vào ổn định.
- Tốc độ lấy mẫu 1000 Hz: Cảm biến giao tiếp qua bus I2C và được gán mức ưu tiên cao nhất trong hệ điều hành FreeRTOS (mức ưu tiên 15). Việc duy trì chu kỳ lấy mẫu 1 ms/mẫu giúp hệ thống tính toán chính xác giá trị hiệu dụng và phương sai của trục Z, làm cơ sở dữ liệu cho thuật toán phân luồng trạng thái tự động.

2.2.4 Sơ đồ kết nối và bố trí thiết bị trên đối tượng nghiên cứu

Việc bố trí thiết bị giám sát được thiết kế dựa trên thực nghiệm, tuân thủ sơ đồ nối dây phần cứng chi tiết (Hình 2.2).



Hình 2.2

Sơ đồ kết nối phần cứng và vị trí lắp đặt cụm cảm biến

Toàn bộ cụm mạch được đặt trong vỏ bảo vệ cách điện và gắn cố định tại mặt bên ngoài lồng giặt. Vị trí này được lựa chọn dựa trên hai yếu tố:

1. Về mặt âm học: Hỗ trợ thu nhận tín hiệu âm thanh trực tiếp từ lồng máy, đồng thời giúp microphone giảm thiểu một phần các tạp âm nhiễu từ môi trường xung quanh.
2. Về mặt cơ học: Các dao động do mất cân bằng tải hoặc lỗi trục sẽ được cảm

biến gia tốc ghi nhận với biên độ rõ rệt nhất, qua đó cải thiện tỷ số tín hiệu trên nhiễu (SNR) của vector đặc trưng đầu vào.

Với phần cứng và sơ đồ kết nối đã thiết lập, phần tiếp theo sẽ trình bày chi tiết về thiết kế phần mềm nhúng để điều phối các tác vụ và triển khai thuật toán Trí tuệ nhân tạo (AI) trên hệ thống.

2.3 Kiến trúc phần mềm nhúng dựa trên hệ điều hành FreeRTOS

Phần mềm nhúng của hệ thống được phát triển trên FreeRTOS để đáp ứng yêu cầu xử lý thời gian thực [19]. Trong các hệ thống nhúng đơn giản, cấu trúc vòng lặp tuần tự có thể đủ cho việc đọc cảm biến và điều khiển cơ bản. Tuy nhiên, hệ thống AIoT của đề án phải thực hiện đồng thời nhiều tác vụ có đặc tính thời gian khác nhau, bao gồm thu thập dữ liệu âm thanh, lấy mẫu rung động, xử lý tín hiệu số, suy luận TensorFlow Lite Micro, duy trì kết nối Wi-Fi và truyền bản tin MQTT. Nếu toàn bộ luồng xử lý được đặt trong một vòng lặp duy nhất, độ trễ mạng hoặc tác vụ tính toán nặng có thể làm mất mẫu cảm biến và ảnh hưởng trực tiếp đến sai số tái tạo.

FreeRTOS cho phép chia hệ thống thành các tác vụ độc lập, mỗi tác vụ có mức ưu tiên và chu kỳ thực thi riêng. Thay vì sử dụng mô hình lập trình tuần tự, đề án triển khai kiến trúc *song song thực sự trên lõi kép*, khai thác hai lõi vật lý của Sseed Studio XIAO ESP32-S3 để phân tách độc lập luồng thu thập dữ liệu và luồng suy luận AI. Các cơ chế nhóm sự kiện, hàng đợi và ghim lõi được sử dụng nhằm đồng bộ dữ liệu, giảm tranh chấp tài nguyên và bảo vệ luồng suy luận khỏi độ trễ bất định của truyền thông mạng.

2.3.1 Thiết kế luồng xử lý tín hiệu âm thanh trên lõi số 0

Tác vụ xử lý âm thanh được gán vào lõi số 0 với mức ưu tiên 10, đảm bảo dòng dữ liệu liên tục và tránh mất mẫu. Đường ống xử lý bao gồm:

- **Thu thập dữ liệu I2S-DMA:** Cơ chế DMA được sử dụng để đọc liên tục tín hiệu âm thanh từ microphone INMP441. Tần số lấy mẫu được thiết lập ở mức 8 kHz. Việc giới hạn tần số lấy mẫu này giúp tiết kiệm dung lượng bộ nhớ đệm và giảm tải khối lượng tính toán FFT, đồng thời vẫn giữ lại được các dải tần số âm học đặc trưng của động cơ (0-4 kHz).

- **Tiền xử lý DSP:** Tác vụ áp dụng hàm cửa sổ Hann và phép biến đổi Fourier nhanh 512 điểm thông qua thư viện `esp-dsp` [20].
- **Trích xuất đặc trưng Mel:** Năng lượng trên 13 dải lọc Mel được tính toán theo thang logarit để tạo thành vector đặc trưng âm thanh.

Sau khi hoàn tất, tác vụ sẽ đẩy dữ liệu vào `audio_queue` và bật cờ hiệu `AUDIO_READY_BIT` trong nhóm sự kiện.

2.3.2 Thiết kế luồng thu thập rung động và suy luận AI trên lõi số 1

Lõi số 1 quản lý hai tác vụ với mức ưu tiên khác nhau, đảm nhiệm thu thập rung động và ra quyết định cảnh báo.

Tác vụ thu thập rung động (`vib_task`). Tác vụ này được cấp mức ưu tiên cao nhất trong ứng dụng người dùng (mức ưu tiên 15) nhằm đảm bảo độ chính xác cho chu kỳ lấy mẫu. Thông qua hàm định thời `vTaskDelayUntil`, hệ thống duy trì chu kỳ 1 ms (1000 Hz) cho mỗi lần đọc cảm biến ADXL345 qua bus I2C. Dữ liệu sau đó được xử lý để lấy giá trị RMS và phương sai trục Z, đồng thời bật cờ hiệu `VIB_READY_BIT`.

Tác vụ suy luận AI (`ai_task`). Được cấu hình ở mức ưu tiên 5, tác vụ này đảm nhiệm quá trình phân tích và ra quyết định với luồng thực thi như sau:

1. **Đồng bộ hóa:** Đợi nhóm sự kiện (`AUDIO_READY_BIT` & `VIB_READY_BIT`).
2. **Chuẩn hóa hỗn hợp:** Áp dụng *MinMax Scaling* cho 13 đặc trưng âm thanh và *RobustScaler* cho 6 đặc trưng rung động để hạn chế ảnh hưởng của nhiễu ngoại lai trước khi suy luận.
3. **Chuyển đổi bộ thông dịch:** Hệ thống đã cấp phát sẵn 3 vùng đệm tensor trên PSRAM. Dựa vào chỉ số phương sai rung động, hệ thống sẽ trở đến bộ thông dịch của mô hình tương ứng (GENTLE, STRONG, hoặc SPIN).
4. **Suy luận:** Gọi hàm `Invoke()` của TensorFlow Lite Micro để tính toán sai số tái tạo (MAE).
5. **Ra quyết định:** Đối chiếu giá trị MAE với các ngưỡng tĩnh đã được định nghĩa trong `model_data.h` để xác định trạng thái bất thường.

2.3.3 Cơ chế đồng bộ hóa đa tác vụ

Để quản lý luồng dữ liệu, hệ thống sử dụng ba cơ chế đồng bộ hóa của FreeRTOS:

- **Nhóm sự kiện:** Đường ống xử lý AI chỉ được kích hoạt khi cả hai luồng dữ liệu (âm thanh và rung động) đều đã thu thập đủ mẫu, giúp loại bỏ tình trạng lệch pha thời gian giữa các cảm biến.
- **Hàng đợi:** Sử dụng `audio_queue` để truyền vector đặc trưng và `mqtt_queue` để truyền thông báo cảnh báo. Cơ chế hàng đợi giúp các tác vụ giao tiếp an toàn mà không chặn lẫn nhau.
- **Cô lập tác vụ truyền dẫn:** Quá trình gửi dữ liệu MQTTS được tách biệt khỏi luồng suy luận AI và đưa vào hàm `loop()` nội tại (mức ưu tiên 1). Hệ thống sử dụng phương thức `mqtt_smart_publish()` để gửi bản tin nhip định kỳ (30 giây) hoặc gửi ngay lập tức khi có sự kiện cảnh báo. Cách thiết kế này tách biệt độ trễ mạng khỏi vòng lặp xử lý AI tại vùng biên, tránh gián đoạn quá trình suy luận.

2.4 Đề xuất kiến trúc phân mảnh trạng thái

Máy giặt là một hệ thống động lực học có tính chu kỳ, trong đó các pha vận hành mang đặc tính vật lý (âm thanh và biên độ rung) thay đổi rõ rệt. Việc sử dụng một mô hình mạng nơ-ron tự mã hóa duy nhất để bao quát toàn bộ chu kỳ sẽ làm tăng độ phức tạp của không gian phân phối dữ liệu, dẫn đến tỷ lệ cảnh báo giả (False Positive) cao tại các thời điểm thiết bị chuyển pha. Các nghiên cứu trước đây thường chỉ tập trung giám sát trên một trạng thái tĩnh nhất định, bỏ qua vấn đề này [15]. Để giải quyết bài toán này, đề án đề xuất kiến trúc mạng nơ-ron tự mã hóa Ba trạng thái.

2.4.1 Cơ sở phân tách chu trình hoạt động

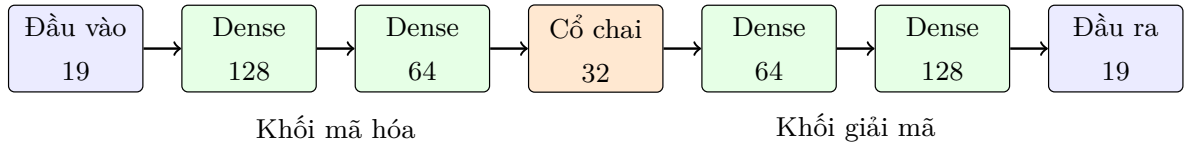
Để hệ thống có thể tự xác định trạng thái vận hành cục bộ mà không cần can thiệp vào mạch điều khiển trung tâm của máy giặt, một thuật toán phân luồng trạng thái dựa trên dữ liệu chấn động đã được thiết kế.

Thông qua phân tích tương quan thực nghiệm trên tập dữ liệu thu thập, nghiên cứu xác định được **phương sai trực Z** (đại diện cho cường độ chấn động thẳng đứng) của cảm biến ADXL345 là chỉ báo ổn định để phân loại trạng thái. Logic định tuyến được xây dựng bằng cách so sánh với các ngưỡng phân hoạch tĩnh, chia chu kỳ thành ba giai đoạn chính: **GENTLE** (Giật/Ngâm), **STRONG** (Xả/Đảo lòng) và **SPIN** (Vất tốc độ cao).

Các ngưỡng này được trích xuất từ phân tích phân phối thống kê của tập dữ liệu huấn luyện (trình bày chi tiết tại Chương 3). Cơ chế định tuyến này cho phép tác vụ `ai_task` tự động trở con trở bộ thông dịch (`Interpreter`) tới vùng nhớ chứa trọng số mô hình tương ứng. Do cả ba mô hình đã được nạp sẵn trên PSRAM, quá trình chuyển đổi diễn ra tức thời, đáp ứng yêu cầu thời gian thực của hệ thống.

2.4.2 Thiết kế mạng tự mã hóa Dense chuyên biệt

Mỗi mô hình con được xây dựng theo kiến trúc mạng nơ-ron tự mã hóa Dense với cấu trúc đối xứng, giúp đơn giản hóa tính toán và cho phép tái sử dụng bộ đệm hiệu quả trong môi trường nhúng:



Hình 2.3

Kiến trúc mạng tự mã hóa Dense đối xứng dùng cho mỗi pha vận hành.

- **Lớp đầu vào:** Tiếp nhận vector đặc trưng 19 chiều (13 đặc trưng Mel-filterbank và 6 đặc trưng rung động).
- **Khối mã hóa:** Thực hiện nén dữ liệu qua các lớp ẩn ($19 \rightarrow 128 \rightarrow 64 \rightarrow 32$). Lớp ẩn đầu tiên được thiết kế theo hướng biểu diễn vượt quá nhằm tăng cường khả năng trích xuất các tương quan phi tuyến từ luồng tín hiệu đa phương thức [21].
- **Khối giải mã:** Có cấu trúc phản chiếu đối xứng ($32 \rightarrow 64 \rightarrow 128 \rightarrow 19$), làm nhiệm vụ tái tạo lại vector đầu vào ban đầu.

2.4.3 Kỹ thuật cân bằng đặc trưng với hàm mất mát MAE có trọng số

Trong kỹ thuật kết hợp dữ liệu đa phương thức, sự chênh lệch về số lượng đặc trưng (13 chiều âm thanh so với 6 chiều rung động) có thể khiến thuật toán học máy ưu tiên giảm thiểu sai số của luồng âm học và bỏ qua các biến động cơ học nhỏ. Để khắc phục, đề án áp dụng kỹ thuật hàm mất mát MAE có trọng số trong giai đoạn huấn luyện. Hàm mất mát được định nghĩa như sau:

$$L_{WMAE} = \frac{1}{N} \sum_{i=1}^N w_i \cdot |x_i - \hat{x}_i| \quad (2.1)$$

Trong đó, hệ số w_i được đặt bằng 1 đối với các đặc trưng âm thanh và bằng 5 đối với các đặc trưng rung động. Giá trị 5 được lựa chọn theo hướng thực nghiệm có kiểm soát: nó bù lại bất lợi về số chiều của nhánh rung động (6 chiều so với 13 chiều âm thanh) nhưng vẫn không làm triệt tiêu hoàn toàn đóng góp của phổ âm thanh. Bằng cách tăng trọng số phạt trên sai số tái tạo của rung động, mô hình được định hướng biểu diễn chính xác hơn các đặc trưng cơ học, từ đó cải thiện độ nhạy với các sự cố bất thường vật lý. Hiệu quả của lựa chọn này được kiểm chứng trong nghiên cứu loại bỏ thành phần ở Chương 4, khi biến thể không dùng MAE có trọng số cho kết quả thấp hơn rõ rệt so với mô hình đề xuất.

2.4.4 Đánh giá tương thích hàm kích hoạt trong môi trường lượng tử hóa

Để triển khai hiệu quả trên vi điều khiển Seeed Studio XIAO ESP32-S3, phương pháp lượng tử hóa nhận thức huấn luyện được tích hợp. Kỹ thuật này ánh xạ toàn bộ trọng số mạng nơ-ron từ định dạng số thực Float32 sang định dạng số nguyên INT8.

Đặc biệt, tại lớp thắt cổ chai, đề án sử dụng hàm kích hoạt Sigmoid thay cho hàm ReLU truyền thống. Quyết định thiết kế này dựa trên đặc tính hữu hạn của Sigmoid (giới hạn đầu ra trong khoảng $[0, 1]$). Khoảng giá trị này giúp quá trình lượng tử hóa ánh xạ đồng đều hơn vào 256 mức rời rạc của chuẩn INT8, qua đó giảm sai số mất mát thông tin so với hàm ReLU có dải giá trị dương không giới hạn [10].

2.5 Thiết kế hạ tầng truyền thông Internet of Things

Hệ thống yêu cầu một hạ tầng mạng ổn định và tiết kiệm tài nguyên để truyền kết quả suy luận từ thiết bị biên đến người dùng. Đồ án sử dụng giao thức MQTT làm nền tảng giao tiếp, kết hợp với các cơ chế điều phối luồng tin phù hợp với giới hạn của vi điều khiển Sseeed Studio XIAO ESP32-S3.

MQTT là giao thức truyền thông điệp gọn nhẹ, hoạt động theo mô hình xuất bản - đăng ký và được chuẩn hóa bởi OASIS cho các hệ thống IoT [38]. Thay vì yêu cầu thiết bị biên kết nối trực tiếp đến từng ứng dụng người dùng, MQTT sử dụng một máy chủ trung gian để tiếp nhận, định tuyến và phân phối bản tin theo chủ đề. Cơ chế này giúp giảm độ phụ thuộc giữa thiết bị gửi và thiết bị nhận, đồng thời phù hợp với các nút mạng tài nguyên hạn chế nhờ kích thước bản tin nhỏ và chi phí duy trì kết nối thấp [39].

2.5.1 Mô hình xuất bản - đăng ký (Publish/Subscribe) và Broker

Thay vì truyền tải dữ liệu cảm biến thô lên đám mây, kiến trúc trí tuệ nhân tạo tại biên cho phép thiết bị biên giữ lại vai trò xử lý tính toán. Nút mạng Sseeed Studio XIAO ESP32-S3 đóng vai trò là một *Publisher*, chỉ xuất bản các kết quả đã qua suy luận.

Hệ thống sử dụng HiveMQ Cloud làm dịch vụ máy chủ MQTT trung gian. Ứng dụng di động Flutter đóng vai trò là *Subscriber*, đăng ký các chủ đề (Topic) cụ thể để nhận thông tin từ máy giặt. Mô hình xuất bản - đăng ký tách biệt logic hoạt động giữa thiết bị vật lý và phần mềm giám sát, giúp hai thành phần có thể phát triển độc lập.

2.5.2 Cấu trúc bản tin chuẩn hóa JSON

Dữ liệu giám sát được đóng gói theo định dạng JSON để tương thích đa nền tảng. Bản tin được thiết kế tối giản nhằm giảm overhead khi phân giải chuỗi (parsing) trên vi điều khiển, bao gồm 3 trường thông tin:

- **mode**: Trạng thái vận hành hiện tại của máy giặt (GENTLE, STRONG, hoặc SPIN).
- **mae**: Giá trị sai số tái tạo do mô hình tự mã hóa xuất ra.

- **state:** Trạng thái đánh giá của hệ thống sau bộ lọc thời gian (OK, HIGH, hoặc ALARM).

2.5.3 Kỹ thuật truyền tin hỗn hợp: Telemetry định kỳ và Hướng sự kiện

Để cân bằng giữa yêu cầu trực quan hóa dữ liệu trên ứng dụng và giới hạn tài nguyên mạng, hệ thống triển khai hàm `mqtt_smart_publish()` tích hợp hai cơ chế truyền tin:

1. **Truyền tin định kỳ (Dữ liệu đo xa 1 Hz):** Hệ thống thiết lập chu kỳ xuất bản bản tin 1 giây/lần. Tần số này đủ để cung cấp đồ thị sai số tái tạo (MAE) mượt mà trên ứng dụng giám sát thời gian thực, đồng thời không gây quá tải cho các tác vụ xử lý mạng của vi điều khiển.
2. **Truyền tin hướng sự kiện:** Khi thuật toán nhận diện có sự thay đổi về trạng thái **state** (ví dụ: chuyển từ OK sang ALARM), một bản tin ngắt sẽ được xuất bản ngay lập tức mà không chờ đến chu kỳ tiếp theo, giữ độ trễ cảnh báo ở mức thấp nhất có thể.

2.5.4 Bảo mật truyền tải và Đánh đổi tài nguyên (Trade-offs)

Hệ thống kết nối với Broker thông qua cổng 8883 để sử dụng giao thức MQTTS (MQTT qua TLS/SSL). Tuy nhiên, do phần lớn RAM của vi điều khiển Seeed Studio XIAO ESP32-S3 đã được cấp phát cho Tensor Arena của các mô hình AI, hệ thống cấu hình lớp `WiFiClientSecure` ở chế độ `setInsecure()`.

Phương pháp này vẫn mã hóa tải trọng (khối dữ liệu) trên đường truyền, nhưng bỏ qua bước xác thực chứng chỉ máy chủ Broker [22]. Đây là sự đánh đổi phù hợp với điều kiện tài nguyên hạn chế của phần cứng hiện tại; việc tích hợp xác thực chứng chỉ đầy đủ được xác định là hướng mở rộng cho các phiên bản phần cứng sau.

XÂY DỰNG DỮ LIỆU VÀ HUẤN LUYỆN MÔ HÌNH

Trong các hệ thống học máy tại biên (TinyML), tập dữ liệu huấn luyện chất lượng cao là yếu tố nền tảng để mô hình hoạt động tốt trên thực tế. Do bài toán phát hiện bất thường gặp vấn đề khan hiếm dữ liệu lỗi, đề tài tiếp cận theo hướng học không giám sát, yêu cầu một tập dữ liệu nền (Background Dataset) chuẩn xác phản ánh các trạng thái vận hành bình thường của thiết bị. Chương này trình bày quy trình xây dựng tập dữ liệu thực tế, các kỹ thuật tăng cường dữ liệu và quá trình huấn luyện mô hình tự mã hóa ba trạng thái.

3.1 Quá trình thu thập dữ liệu thực tế

Để xây dựng tập dữ liệu phản ánh đặc tính vật lý của máy giặt, một trạm thu thập dữ liệu chuyên dụng dựa trên vi điều khiển Seeed Studio XIAO ESP32-S3 đã được thiết kế và gắn trực tiếp lên lồng giặt để đo đạc thực địa.

3.1.1 Xây dựng giao thức truyền nhận dữ liệu tốc độ cao định dạng nhị phân

Thu thập dữ liệu đa cảm biến (âm thanh 8 kHz + gia tốc 1 kHz) qua cổng Serial bằng văn bản ASCII sẽ gây độ trễ phân giải cao, dẫn đến tràn bộ đệm và mất mẫu. Để khắc phục, đồ án triển khai giao thức truyền dẫn nhị phân (Binary Protocol) tùy chỉnh qua UART ở tốc độ baud 921600. ESP32 trích xuất vector đặc trưng 19 chiều tại thiết bị biên, sau đó đóng gói vào một cấu trúc chuẩn:

- **Cấu trúc Payload:** Mỗi gói tin bắt đầu bằng hai byte đồng bộ (Sync word) 0xAA 0xBB, tiếp theo là 19 giá trị float (76 bytes), và kết thúc bằng một byte kiểm tra lỗi (Checksum).

- **Hiệu năng:** Truyền trực tiếp khối nhớ nhị phân thay vì chuyển đổi sang chuỗi tiết kiệm khoảng 60% băng thông kênh truyền, đảm bảo thu thập liên tục mà không quá tải phần cứng.

3.1.2 Tự động hóa quy trình thu thập và kiểm soát chất lượng dữ liệu

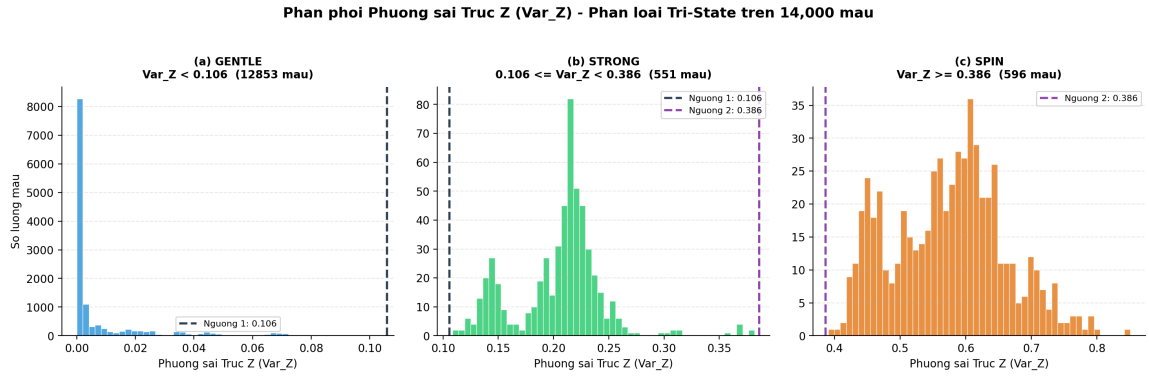
Một kịch bản Python (`dataCollection.py`) được phát triển để tự động hóa quy trình thu thập cho các chu kỳ kéo dài 45 - 90 phút.

- **Tự động hóa phân mảnh (Windowing):** Kịch bản liên tục đọc luồng Serial, căn lề theo byte đồng bộ để giải mã mảng nhị phân. Mỗi tệp ghi 5 giây được tách thành 5 cửa sổ 1 giây, tương ứng 5 mẫu đặc trưng độc lập, rồi lưu trữ dưới định dạng ma trận NumPy (`.npy`).
- **Kiểm soát chất lượng (Data Quality Control):** Quá trình thu nhận thực hiện hai bước kiểm tra:
 1. *Xử lý nhiễu:* Tự động loại bỏ các mẫu chứa giá trị NaN hoặc Inf (do xung đột bus I2C hoặc nhiễu điện từ).
 2. *Đồng bộ hóa dải năng lượng:* Áp dụng kỹ thuật *Audio Volume Shifting*, giới hạn năng lượng phổ Mel trong dải chuẩn $[-80, 0]$ dB. Thao tác này giúp khắc phục sai lệch biểu diễn dữ liệu giữa môi trường huấn luyện (Python) và môi trường thực thi (C++).

3.1.3 Thống kê phân phối và xử lý sự mất cân bằng dữ liệu (Data Imbalance)

Sau nhiều phiên đo đạc các chu trình giặt thực tế, hệ thống có tập dữ liệu gốc (Raw Dataset) gồm 14.000 mẫu (khoảng 20 giờ vận hành). Dữ liệu được phân nhóm tự động thành ba cụm trạng thái dựa trên phương sai trục Z (Hình 3.1), phản ánh mất cân bằng tự nhiên của chu trình giặt:

- **Tập GENTLE (Pha giặt/ngâm):** 12.853 mẫu (khoảng 91,8%). Tỷ trọng lớn phù hợp vì đây là pha chiếm thời gian dài nhất.
- **Tập STRONG (Pha xả/đảo lồng):** 551 mẫu (3,9%). Pha này diễn ra trong thời gian ngắn khi máy đảo lồng để xả nước.
- **Tập SPIN (Pha vắt):** 596 mẫu (4,3%). Pha vắt tốc độ cao kéo dài vài phút ở cuối chu trình.



Hình 3.1

Phân phối thực nghiệm của phương sai trục Z (Var_Z) trên tập dữ liệu 14.000 mẫu. Hai ngưỡng phân hoạch tại 0.106 và 0.386 tạo ra ba vùng phân bố tách biệt rõ rệt.

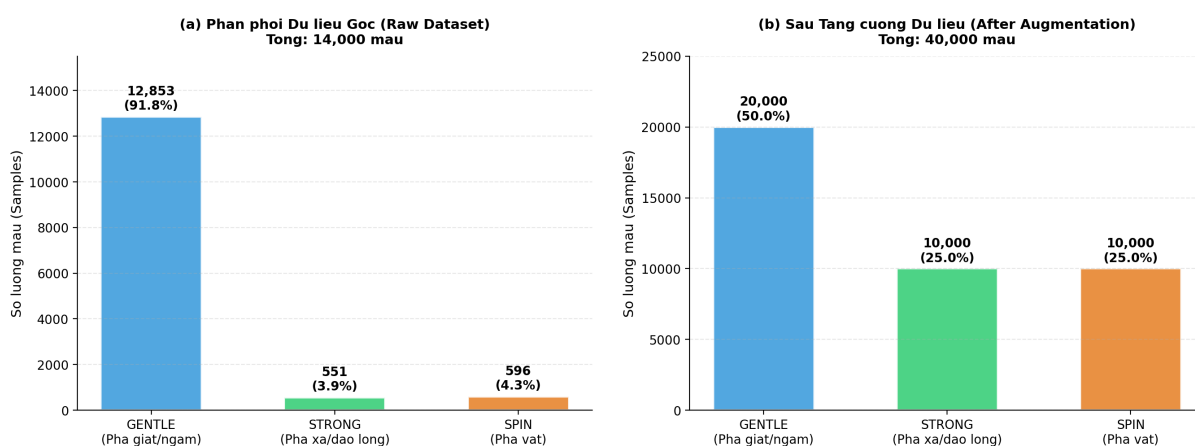
Các ngưỡng phân hoạch trạng thái được xác định tự động thông qua thuật toán **K-Means Clustering**. Thuật toán gom cụm phương sai trục Z thành 3 nhóm, từ đó tính toán các ranh giới τ_1 và τ_2 tại điểm trung vị giữa hai cụm kế cận. Cách tiếp cận này cho phép hệ thống thích nghi với mức rung động khác nhau của từng loại lồng giặt.

Tuy nhiên, chênh lệch phân phối ở tỷ lệ 92:4:4 tạo vấn đề cho huấn luyện. Nếu dùng trực tiếp tập dữ liệu thô, mạng nơ-ron tự mã hóa sẽ thiên lệch về phía tập GENTLE, giảm độ chính xác tái tạo ở các pha STRONG và SPIN. Để khắc phục, thay vì giảm lấy mẫu (Undersampling), đề án áp dụng chiến lược tăng cường dữ liệu (Data Augmentation) [7]. Ba kỹ thuật được kết hợp:

1. **Magnitude Scaling:** Nhân biên độ rung với hệ số ngẫu nhiên trong khoảng $[0.8, 1.2]$ để giả lập sự thay đổi tải trọng đồ giặt.
2. **Audio Volume Shifting:** Tịnh tiến năng lượng phổ Mel ngẫu nhiên từ -5 dB đến $+2$ dB nhằm giả lập sai số về khoảng cách thu âm.
3. **Per-axis Jittering:** Thêm nhiễu Gauss độc lập vào từng trục gia tốc để tăng khả năng chống rung động vi mô.

Sau khi tăng cường dữ liệu, các tập GENTLE, STRONG và SPIN lần lượt có 20.000, 10.000 và 10.000 mẫu, với tỷ lệ cân bằng hơn là 2:1:1 (Hình 3.2).

Phân tích Phân phối Dữ liệu Gốc và sau Tăng cường Dữ liệu



Hình 3.2

Phân phối dữ liệu ba trạng thái trước và sau quá trình tăng cường. Tỷ lệ chênh lệch ban đầu (92:4:4) được cân bằng về mức 2:1:1.

3.2 Trích xuất đặc trưng và tiền xử lý dữ liệu

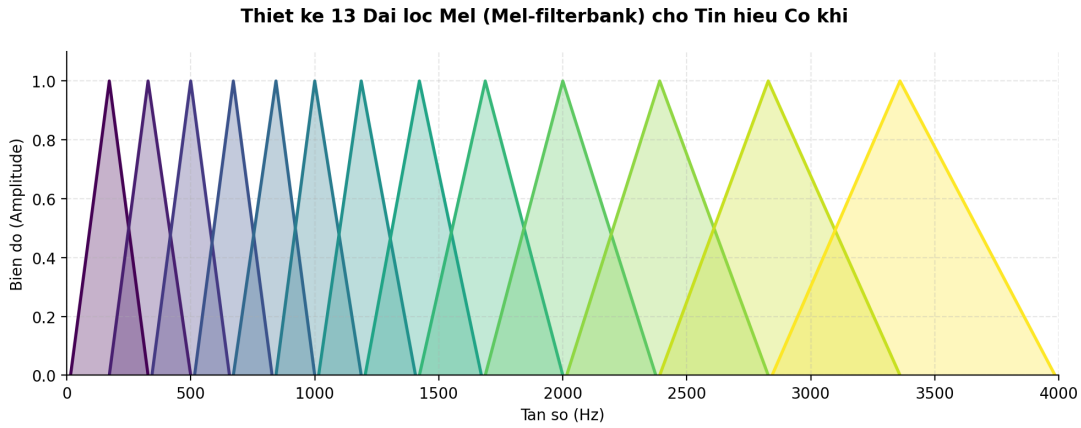
Dữ liệu thô từ các cảm biến ở dạng chuỗi thời gian chứa nhiều nhiễu nền và có số chiều lớn. Bước Trích xuất đặc trưng (Feature Extraction) giữ lại các đặc trưng tiêu biểu nhất về động lực học của máy giặt, tối ưu hóa không gian dữ liệu đầu vào cho mạng nơ-ron trên vi điều khiển.

3.2.1 Thiết kế bộ lọc Mel tối ưu cho tín hiệu cơ khí

Đối với dữ liệu âm thanh, đồ án chuyển đổi sang biểu diễn Năng lượng dải Mel (Mel-Frequency Energy - MFE). Mặc dù phổ Mel được dùng rộng rãi trong xử lý giọng nói, hệ thống tùy chỉnh cấu trúc này cho phân tích tiếng ồn cơ khí:

- Tín hiệu âm thanh lấy mẫu ở tần số 8 kHz được phân khung kết hợp cửa sổ Hann để giảm hiện tượng rò rỉ phổ, sau đó áp dụng phép biến đổi Fourier nhanh 512 điểm với bước trượt 256 mẫu.
- Thay vì sử dụng 40 hoặc 128 dải Mel như các bài toán phân loại âm thanh tổng quát, hệ thống được tối ưu hóa xuống còn 13 dải lọc tam giác (13 Mel-bins). Các dải lọc tập trung mật độ cao ở vùng tần số thấp và trung — nơi phát ra tiếng ồn vận hành của động cơ và luồng nước — đồng thời giảm độ phân giải ở vùng tần số cao để loại bỏ nhiễu môi trường không mong muốn.

- Năng lượng tại mỗi dải Mel được tính theo thang logarit tự nhiên để mô phỏng đặc tính nén dải động của thính giác. Cách tiếp cận này làm nổi bật các dấu hiệu bất thường có biên độ nhỏ vốn dễ bị lấn át bởi nhiễu nền. Kết quả đầu ra là một vector 13 chiều cho mỗi khung âm thanh.



Hình 3.3

Trực quan hóa cấu trúc 13 dải lọc Mel tùy chỉnh được sử dụng trong hệ thống. Các bộ lọc hình tam giác được thiết kế phi tuyến, tập trung ở vùng tần số thấp (dưới 1000 Hz) để bóc tách tối đa âm thanh cơ học của máy giặt, đồng thời giảm phân giải ở dải tần cao để hạn chế nhiễu môi trường.

3.2.2 Kỹ thuật đồng bộ hóa thuật toán xử lý giữa Python và Firmware C++

Khi phát triển TinyML, sai lệch biểu diễn (Representation Mismatch) là vấn đề thường gặp. Trong giai đoạn thử nghiệm đầu, mô hình mạng nơ-ron tự mã hóa đạt độ chính xác cao trên Python nhưng sinh ra cảnh báo giả khi triển khai lên vi điều khiển Seeed Studio XIAO ESP32-S3.

Nguyên nhân là sự khác biệt trong tính toán dấu phẩy động: Python sử dụng thư viện `librosa` với độ chính xác kép (Float64), trong khi firmware C++ thực thi các hàm FFT bằng tập lệnh vector trên kiến trúc Xtensa LX7 chỉ hỗ trợ Float32, kết hợp với mảng lọc tĩnh `MEL_FB` tự định nghĩa. Sai số tích lũy qua 13 dải tần đã làm thay đổi phân phối của vector đặc trưng đầu vào so với lúc huấn luyện.

Để khắc phục, đề án thiết kế lại luồng trích xuất đặc trưng theo cơ chế nhất quán:

1. Loại bỏ sự phụ thuộc vào `librosa`. Mã nguồn Python được viết lại để tính toán FFT và Mel-filterbank bằng các phép toán ma trận NumPy cơ bản, mô

phồng chính xác logic làm tròn số và mảng MEL_FB của mã nguồn C++.

2. Áp dụng hệ số nhân tĩnh (Static Scaling Factor) và giới hạn dải năng lượng trong khoảng $[-80, 0]$ dB một cách đồng nhất trên cả hai môi trường.

Phương pháp định hướng firmware (Firmware-Driven Data Pipeline) này đảm bảo nguyên tắc "Train as you deploy", giúp phân phối dữ liệu tại máy chủ huấn luyện và thiết bị biên trùng khớp nhau.

3.2.3 Xử lý dữ liệu ngoại lai cho tín hiệu rung động

Đối với cảm biến gia tốc ADXL345, 6 đặc trưng thống kê được trích xuất bao gồm giá trị hiệu dụng (RMS) và phương sai của ba trục X, Y, Z. Dữ liệu chấn động của máy giặt chứa nhiều ngoại lai (Outliers) mang tính nhất thời, chẳng hạn như tiếng cốc áo va đập vào lồng giặt hoặc sự xô lệch đột ngột của tải trọng.

Nếu áp dụng MinMax Scaling tiêu chuẩn, một gai nhiễu đơn lẻ có thể làm thay đổi toàn bộ dải cực trị, nén 99% dữ liệu bình thường vào một dải biên độ hẹp và mất khả năng phân biệt các biến động nhỏ. Do đó, đề án sử dụng kỹ thuật xử lý hỗn hợp (Hybrid Scaling):

1. **Cắt xén ngoại lai (Clipping):** Đối với phương sai trục X và trục Y (nhạy cảm với va đập), thuật toán xác định ngưỡng cắt tại phân vị thứ 99 của tập huấn luyện và giới hạn các giá trị vượt ngưỡng về mức biên này (99th Percentile Clipping). Riêng phương sai trục Z được giữ nguyên dải giá trị ban đầu, vì đây là đặc trưng quyết định để phân loại trạng thái ba trạng thái.
2. **Chuẩn hóa (Scaling):** Dữ liệu sau khi qua bộ lọc Clipping được chuẩn hóa bằng RobustScaler. Đề án mở rộng khoảng phân vị tính toán lên (10, 90) thay vì IQR tiêu chuẩn (25, 75) để bao quát tốt hơn dải động của cảm biến [23]. Đồng thời, một hệ số bù tối thiểu 0.002 được cộng thêm vào mẫu số để tránh lỗi chia cho 0 khi máy giặt ở pha tĩnh (GENTLE), đảm bảo tính ổn định số học của hệ thống.

3.3 Cấu hình huấn luyện và lượng tử hóa mô hình

Sau khi dữ liệu được tiền xử lý và chuẩn hóa về dải $[0, 1]$, bước tiếp theo là xây dựng quy trình huấn luyện và nén mô hình nhằm đáp ứng các giới hạn về bộ

nhớ và năng lực tính toán của Seeed Studio XIAO ESP32-S3.

3.3.1 Phân chia tập huấn luyện, kiểm định và kiểm thử

Quá trình huấn luyện được thực hiện riêng cho từng pha vận hành nhằm tránh việc mô hình bị chi phối bởi trạng thái GENTLE vốn chiếm tỷ trọng lớn trong dữ liệu thô. Từ tập `train_features_v6.csv` gồm 14.000 vectơ đặc trưng 19 chiều, dữ liệu được phân hoạch theo phương sai trục Z thành ba tập GENTLE, STRONG và SPIN như đã trình bày ở Mục 3.1.3. Sau đó, mỗi tập trạng thái được tăng cường độc lập trước khi đưa vào mô hình tự mã hóa tương ứng.

Trong mã huấn luyện, tập kiểm định được tách trực tiếp từ dữ liệu sau tăng cường bằng tham số `validation_split=0.15`. Như vậy, 85% dữ liệu tăng cường được dùng để cập nhật trọng số mô hình, còn 15% được giữ lại để theo dõi sai số kiểm định và kích hoạt cơ chế dừng sớm. Tập kiểm thử cuối cùng không được dùng trong quá trình tối ưu trọng số; các đánh giá định lượng trên tập kiểm thử độc lập và dữ liệu lỗi giả lập được trình bày ở Chương 4.

Bảng 3.1

Cách chia dữ liệu huấn luyện và kiểm định theo từng pha vận hành.

Pha	Mẫu gốc	Sau tăng cường	Huấn luyện 85%	Kiểm định 15%
GENTLE	12.853	20.000	17.000	3.000
STRONG	551	10.000	8.500	1.500
SPIN	596	10.000	8.500	1.500

3.3.2 Kỹ thuật tăng cường dữ liệu

Phương pháp học không giám sát yêu cầu tập dữ liệu huấn luyện bao quát được các biến động tự nhiên của môi trường vận hành bình thường. Hệ thống áp dụng các phương pháp tăng cường dữ liệu trực tiếp trên tín hiệu vật lý: Điều chỉnh rung (Vibration Scaling), Dịch chuyển biên độ âm thanh (Audio Volume Shifting) và Rung động từng trục (Per-axis Jittering) (chi tiết tại Mục 3.1.3).

3.3.3 Quá trình huấn luyện mô hình số thực (Float32)

Mô hình đối xứng được thiết kế theo cấu trúc thắt cổ chai sâu (Deep Bottleneck) với số lượng nơ-ron qua các lớp lần lượt là: $19 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow$

19. Việc mở rộng không gian đặc trưng ở lớp ẩn đầu tiên (từ 19 lên 128 chiều) tạo cấu trúc biểu diễn dư thừa, cung cấp đủ không gian để trích xuất các mối tương quan phi tuyến giữa tín hiệu âm thanh và rung động trước khi nén tại lớp cổ chai 32 chiều.

- **Hàm kích hoạt Sigmoid:** Mặc dù ReLU là hàm kích hoạt phổ biến, kiến trúc này sử dụng Sigmoid cho toàn bộ các lớp ẩn. Sigmoid giới hạn giá trị đầu ra trong khoảng $[0, 1]$, thu hẹp dải động của dữ liệu và giảm sai số khi thực hiện lượng tử hóa về định dạng số nguyên 8-bit (INT8).
- **Hàm mất mát có trọng số:** Trong vector đầu vào, 13 đặc trưng âm thanh chiếm ưu thế về số lượng, nhưng 6 đặc trưng rung động lại là chỉ báo vật lý quan trọng nhất để xác định lỗi cơ khí. Để ấn định nghĩa hàm `weighted_mae()`, đặt trọng số hệ số 5 cho các sai số thuộc nhóm gia tốc, nhằm ưu tiên tối ưu hóa khả năng tái tạo động hình cơ học của lồng giặt.
- **Thuật toán tối ưu và khử nhiễu:** Mô hình sử dụng thuật toán tối ưu Adam [24] với tốc độ học khởi tạo 0,001 trong giai đoạn huấn luyện số thực. Để tăng tính bền vững, dữ liệu đầu vào được cộng thêm nhiễu Gaussian nhẹ có độ lệch chuẩn 0,02 trong miền đã chuẩn hóa $[0, 1]$, trong khi hàm mục tiêu yêu cầu mô hình tái tạo lại tín hiệu sạch. Cách tiếp cận này giúp mô hình học được các đặc trưng cốt lõi và tránh hội tụ về hàm đồng nhất (identity function).

Bảng 3.2

Cấu hình huấn luyện được sử dụng trong mã nguồn `training.py`.

Thông số	Tiền huấn luyện Float32	Tinh chỉnh QAT INT8
Thuật toán tối ưu	Adam	Adam
Tốc độ học	1×10^{-3}	1×10^{-4}
Số epoch tối đa	200	80
Batch size	64	64
Tỷ lệ kiểm định	15%	15%
Dừng sớm	Patience 20	Patience 15
Khôi phục trọng số tốt nhất	Có	Có

3.3.4 Lượng tử hóa nhận thức huấn luyện

Để triển khai mô hình trên vi điều khiển Seeed Studio XIAO ESP32-S3, kích thước trọng số phải được nén. Thay vì sử dụng lượng tử hóa sau huấn luyện (PTQ),

đồ án áp dụng lượng tử hóa nhận thức huấn luyện. Phương pháp này tích hợp các phép toán mô phỏng lượng tử hóa vào quá trình tinh chỉnh (Fine-tuning), giúp mô hình điều chỉnh trọng số để bù cho sai số do giảm độ phân giải số học.

Quá trình huấn luyện được chia thành hai giai đoạn:

1. **Giai đoạn 1 - Tiền huấn luyện:** Mô hình được huấn luyện với định dạng số thực trong tối đa 200 epochs, batch size 64 và cơ chế dừng sớm sau 20 epochs nếu sai số kiểm định không cải thiện. Sai số kiểm định (Val Loss) hội tụ ở mức xấp xỉ **0,0210** (đối với pha SPIN).
2. **Giai đoạn 2 - Tinh chỉnh:** Mô hình sau tiền huấn luyện được bọc bằng lớp lượng tử hóa nhận thức huấn luyện và tiếp tục tối ưu thêm tối đa 80 epochs với tốc độ học 1×10^{-4} . Cơ chế dừng sớm được đặt patience 15. Nhờ QAT, sai số kiểm định chỉ tăng không đáng kể lên mức **0,0225**.

Mức suy giảm hiệu năng dưới 7% chứng tỏ QAT bảo toàn độ chính xác của mô hình khi chuyển đổi sang môi trường nhúng.

Sau khi huấn luyện, các mô hình được xuất dưới dạng mảng C (Mảng C). Mỗi mô hình đơn lẻ (25.779 tham số) sau lượng tử hóa INT8 có kích thước xấp xỉ **31,7 KB**. Tổng dung lượng cho tổ hợp 3 mô hình (hỗn hợp Ba trạng thái) chỉ chiếm **95,3 KB**, giảm 4 lần so với mô hình Float32, cho phép triển khai thuật toán chuyển đổi trạng thái mà không tăng áp lực lên bộ nhớ Flash.

3.3.5 Xác định ngưỡng cảnh báo dựa trên quy tắc 3σ

Ngưỡng phát hiện dị thường (τ) cho từng pha được thiết lập từ phân phối sai số tái tạo của tập dữ liệu bình thường, tuân theo quy tắc ba sigma (3σ):

$$\tau = \mu_{MAE} + 3 \cdot \sigma_{MAE} \quad (3.1)$$

Lựa chọn hệ số $k = 3$ thay vì các ngưỡng tối ưu theo chỉ số Youden's J phản ánh yêu cầu thực tiễn: cảnh báo giả (False Positive) gây ảnh hưởng tiêu cực đến người dùng nghiêm trọng hơn phản hồi chậm đối với một số dị thường nhỏ (False Negative). Ngưỡng 3σ cung cấp dải biên an toàn lớn, đảm bảo độ chính xác (Precision) cao trước khi phát tín hiệu hỏng hóc.

Các ngưỡng cố định được xác lập tương ứng: GENTLE (0,1241), STRONG (0,0968), và SPIN (0,0672).

3.3.6 Đánh giá cấu hình và hiệu năng tài nguyên

Kiến trúc mạng nơ-ron sử dụng toàn bộ hàm kích hoạt Sigmoid để giới hạn dữ liệu trong dải $[0, 1]$, ngăn ngừa các lỗi tràn bộ đệm khi vi điều khiển thực thi các phép toán nhân ma trận INT8.

Bảng 3.3

Chi tiết cấu trúc các lớp và số lượng tham số của một mô hình tự mã hóa thành phần.

STT	Lớp	Kích thước	Tham số
1	Đầu vào	19	—
2	Dense (Encoder-1)	128	2.560
3	Dense (Encoder-2)	64	8.256
4	Dense (lớp thắt cổ chai)	32	2.080
5	Dense (Decoder-1)	64	2.112
6	Dense (Decoder-2)	128	8.320
7	Đầu ra	19	2.451
Tổng cộng:			25.779

Kích thước tối ưu này (95,3 KB cho 3 mô hình) giúp thời gian tính toán suy luận trên lõi Xtensa LX7 của vi điều khiển Seeed Studio XIAO ESP32-S3 đạt mức khoảng 6,4 ms (Bảng 4.1), chỉ chiếm 0,64% quỹ thời gian của một chu kỳ lấy mẫu 1 Hz, duy trì tần số lấy mẫu tốc độ cao mà không làm chậm các tiến trình khác trong hệ điều hành FreeRTOS.

3.4 Kiểm chứng tính nhất quán của hệ thống

Khi triển khai các hệ thống học máy tại biên, hiện tượng không khớp triển khai — sự sai lệch kết quả suy luận giữa môi trường huấn luyện (Python/TensorFlow) và môi trường thực thi (C++/TFLite Micro) — là vấn đề phổ biến. Phần này trình bày các phương pháp kiểm chứng tính nhất quán của toàn bộ đường ống xử lý, từ tiền xử lý đến suy luận cuối cùng.

3.4.1 Đối chiếu phân phối đặc trưng giữa môi trường giả lập và phần cứng

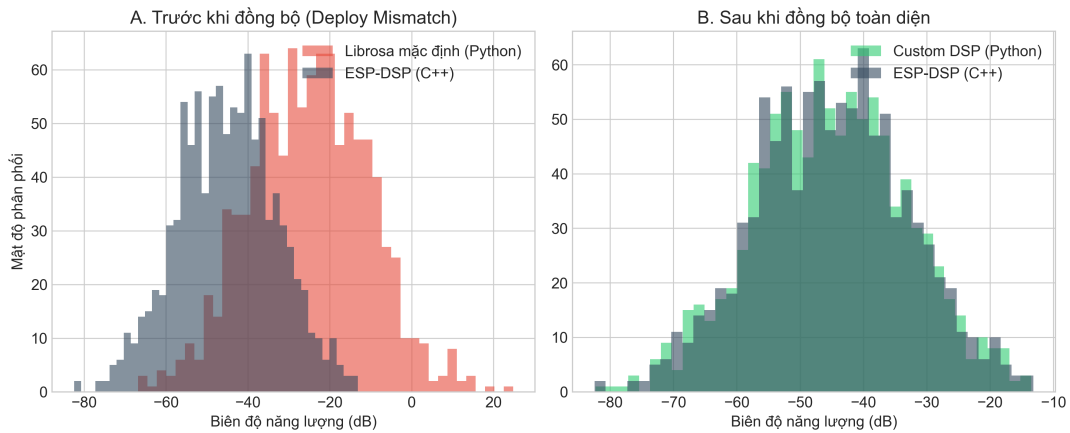
Trong giai đoạn đầu phát triển, mô hình đạt độ chính xác cao trên tập kiểm thử nhưng sinh ra nhiều cảnh báo giả khi nạp xuống vi điều khiển Seeed Studio XIAO ESP32-S3. Nguyên nhân bắt nguồn từ sự bất đồng bộ trong thuật toán trích

xuất đặc trưng âm thanh giữa hai môi trường.

Kịch bản Python ban đầu sử dụng thư viện `librosa` với độ chính xác kép (Float64), trong khi firmware C++ sử dụng thư viện `esp-dsp` với Float32. Sự khác biệt về thư viện và độ chính xác số học này làm thay đổi phân phối dữ liệu đầu vào.

Để khắc phục, hệ thống thực hiện đồng bộ hóa thuật toán:

- **Đồng nhất tham số DSP:** Loại bỏ sự phụ thuộc vào `librosa`. Mã nguồn Python được viết lại bằng NumPy để mô phỏng logic tính toán của vi điều khiển. Cấu hình cố định: cửa sổ Hann, FFT 512 điểm và bước trượt 256 mẫu. Với bộ đệm 1 giây ở tần số lấy mẫu 8 kHz, số khung hợp lệ được tính theo $\lfloor (8000 - 512)/256 \rfloor + 1 = 30$, đúng với cấu hình firmware.
- **Cố định dải động:** Năng lượng tín hiệu qua bộ lọc Mel được giới hạn và ánh xạ về khoảng $[-80, 0]$ dB trên cả hai môi trường. Xác thực thông qua đồ thị phân phối xác minh ma trận đặc trưng từ Python và dữ liệu từ bộ nhớ của vi điều khiển Seeed Studio XIAO ESP32-S3 trùng khớp ở cấp độ bit.



Hình 3.4

Đối chiếu phân phối năng lượng phổ âm thanh trước và sau khi đồng bộ thuật toán DSP giữa Python và C++.

3.4.2 Xác thực tính toàn vẹn của đồ thị suy luận

Sau khi xuất mô hình dưới dạng mảng byte, xác thực tính toàn vẹn bộ nhớ khi nạp qua TensorFlow Lite Micro là bước quan trọng. Hệ thống sử dụng xác thực chéo ở cấp độ tensor:

1. **Trích xuất vector kiểm thử:** 100 vector đặc trưng ngẫu nhiên từ Python được sử dụng làm dữ liệu nền kiểm chứng.
2. **Quản lý vùng nhớ an toàn:** Để tránh lỗi phân mảnh và tràn bộ nhớ cấp phát động, mảng bộ nhớ làm việc trung gian (`tensor_arena`) được cấp phát tĩnh. Kích thước vùng nhớ được tính toán tối ưu với cấu trúc mạng và đặt tại phân vùng SRAM nội bộ để tăng tốc độ truy xuất. Điều này cho phép thiết bị chuyển đổi linh hoạt giữa 3 mô hình (GENTLE, STRONG, SPIN) trên cùng một vùng nhớ trung gian mà không cần cấp phát lại bộ nhớ.
3. **Suy luận song song và đối chiếu:** Một vector đầu vào được đưa qua mô hình Float32 trên máy tính và mô hình INT8 trên vi điều khiển. Đầu ra `int8_t` từ vi điều khiển Seeed Studio XIAO ESP32-S3 được gửi qua cổng Serial để so sánh từng phần tử với mảng đầu ra trên máy chủ.

Bộ thông dịch TFLM thực thi đồ thị mạng ổn định, không ghi nhận tràn vùng nhớ đệm hay lỗi sai số vượt ngưỡng trong quá trình lượng tử hóa.

TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

4.1 Triển khai phần mềm trên vi điều khiển Seeed Studio XIAO ESP32-S3

Chuyển đổi từ mô hình học máy lý thuyết sang hệ thống nhúng ổn định yêu cầu tối ưu hóa sâu về mặt thực thi. Phần này trình bày các giải pháp kỹ thuật để triển khai kiến trúc mạng nơ-ron tự mã hóa Ba trạng thái trên phần cứng tài nguyên hạn chế.

4.1.1 Chiến lược quản lý bộ nhớ cho hệ thống đa mô hình

Hệ thống sử dụng ba mô hình mạng nơ-ron tự mã hóa độc lập cho ba trạng thái (GENTLE, STRONG, SPIN), với tổng dung lượng 95,3 KB. Trong kiến trúc TensorFlow Lite Micro, quản lý bộ nhớ được chia thành hai vùng riêng:

- Vùng nhớ trọng số tĩnh: Mảng C chứa cấu trúc và trọng số mô hình từ `model_data.h` được khai báo với từ khóa `const`, buộc trình biên dịch lưu trữ trên bộ nhớ Flash. Vi điều khiển truy xuất trực tiếp các trọng số thông qua cơ chế ánh xạ bộ nhớ đệm của ESP32-S3 mà không cần sao chép vào RAM.
- Vùng nhớ nội suy: Xử lý dữ liệu kích hoạt giữa các lớp nơ-ron theo thời gian thực bằng vùng đệm tensor. Do lớp thất cổ chai 32-chiều nhỏ gọn, vùng đệm tensor chỉ chiếm vài kilobyte trên SRAM nội bộ siêu tốc.

Chiến lược này loại bỏ độ trễ nạp mô hình, đảm bảo hệ thống không mất mẫu dữ liệu trong các pha chuyển tiếp của máy giặt.

4.1.2 Thực thi tiền xử lý tín hiệu trực tiếp trên Firmware

Để không gian suy luận thực tế trùng khớp với không gian huấn luyện, vì điều khiển thực thi trực tiếp các thuật toán xử lý tín hiệu số bằng các hàm tối ưu hóa phần cứng.

- Chuẩn hóa phổ âm thanh: Sau FFT 512-điểm và bộ lọc Mel, năng lượng phổ được tính dưới dạng Log-dB. Thư viện `esp-dsp` gia tốc các phép toán logarit bằng tập lệnh vector phần cứng. Các giá trị biên độ sau đó được ánh xạ tuyến tính về dải $[0, 1]$ dựa trên hệ số Min $(-80,0)$ và Scale $(80,0)$ từ quá trình huấn luyện.
- Chuẩn hóa rung động: Firmware thực thi công thức: $z = \frac{x - \text{median}}{\text{IQR}}$. Toàn bộ 36 tham số median và IQR cho 3 pha đã được tính sẵn từ Python và lưu dưới dạng hằng số `const float`. Để tránh chia cho không khi máy ở trạng thái tĩnh, hằng số epsilon $\epsilon = 0,002$ được cộng vào mẫu số.

4.1.3 Tính toán sai số tái tạo trên miền lượng tử hóa (INT8-domain)

Hệ thống thực hiện tính toán sai số tái tạo trực tiếp trên miền số nguyên thay vì miền số thực. Vì mô hình đầu ra và dữ liệu đầu vào đều được lượng tử hóa sang INT8, phép tính MAE được thực thi theo công thức:

$$\text{MAE}_{\text{int8}} = \frac{1}{N} \sum_{i=1}^N \left| x_{\text{quantized}}^{(i)} - \hat{x}_{\text{quantized}}^{(i)} \right| \quad (4.1)$$

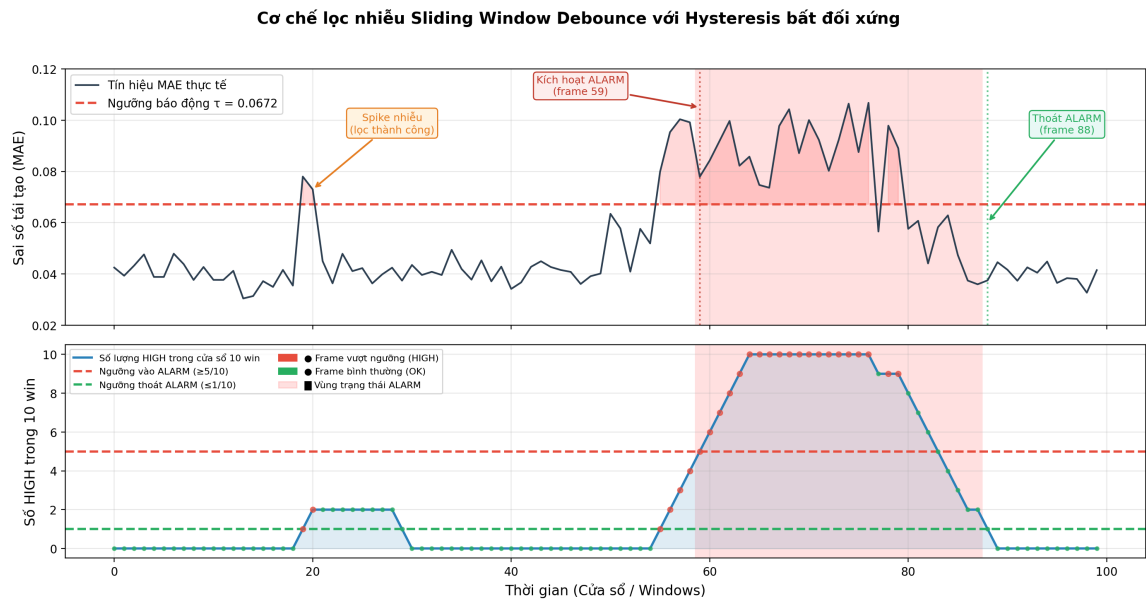
Các phép trừ và giá trị tuyệt đối trên kiểu `int8_t` tiêu tốn ít chu kỳ máy và tương thích tối ưu với tập lệnh SIMD của ESP32-S3 [9]. Sau khi có tổng MAE_{int8} , hệ thống thực hiện một phép nhân duy nhất với hệ số tỉ lệ để giải lượng tử hóa kết quả về Float32 và so sánh với ngưỡng cảnh báo. Phương pháp này giảm chu kỳ CPU hơn 4 lần so với giải lượng tử hóa từng phần tử đầu ra, góp phần duy trì tổng thời gian suy luận ở mức chỉ vài mili-giây.

4.1.4 Cơ chế lọc trễ thời gian

Để loại bỏ cảnh báo giả do nhiễu tức thời, hệ thống triển khai bộ lọc cửa sổ trượt kết hợp cơ chế trễ trong logic ra quyết định. Thay vì kích hoạt cảnh báo ngay khi một mẫu vượt ngưỡng, hệ thống quan sát chuỗi trạng thái của 10 cửa sổ liên tiếp (tương đương khoảng 10 giây vận hành):

- **Điều kiện kích hoạt:** Trạng thái cảnh báo (ALARM) chỉ được xác lập khi có ít nhất 5 trên 10 mẫu gần nhất vượt ngưỡng τ .
- **Cơ chế duy trì và thoát:** Khi đã ở trong trạng thái cảnh báo, yêu cầu độ tin cậy cao hơn để phục hồi. Cảnh báo chỉ được gỡ bỏ khi có ít nhất 9 trên 10 mẫu gần nhất trở về mức an toàn (tức là chỉ cho phép tối đa 1 mẫu vượt ngưỡng).

Cơ chế bất đối xứng này ngăn dao động trạng thái (chống nhiễu "chập chờn") tại vùng biên của ngưỡng cắt, đồng thời ưu tiên an toàn thiết bị bằng cách duy trì cảnh báo cho đến khi sự cố được khắc phục ổn định.



Hình 4.1

Mô phỏng cơ chế cửa sổ trượt kết hợp trễ bất đối xứng: kích hoạt cảnh báo khi $\geq 5/10$ mẫu vượt ngưỡng, phục hồi khi $\leq 1/10$ mẫu vượt ngưỡng. Xung nhiễu đơn lẻ (frame 19–21) được lọc hoàn toàn nhờ cơ chế biểu quyết đa số.

Kỹ thuật lọc trễ phần mềm này hoạt động như bộ lọc thông thấp cho logic quyết định, loại bỏ các đỉnh nhiễu ngẫu nhiên và giúp hệ thống đạt độ tin cậy cực cao (F1-Score > 0,98) trong các bài kiểm tra thực địa dài hạn.

4.2 Đánh giá hiệu năng hệ thống nhúng

Triển khai học máy tại biên (*TinyML*) vào sản xuất công nghiệp không chỉ đánh giá dựa trên độ chính xác thuật toán mà còn phụ thuộc vào hiệu năng thực

thi trên phần cứng giới hạn. Độ trễ, mức tiêu thụ bộ nhớ và độ ổn định của hệ điều hành thời gian thực (RTOS) là tiêu chí đánh giá tính khả thi của hệ thống [25].

4.2.1 Phân tích độ trễ suy luận

Độ trễ suy luận là thời gian từ khi vi điều khiển hoàn tất trích xuất đặc trưng đầu vào (vectơ 19 chiều) cho đến khi đưa ra sai số tái tạo MAE và nhãn phân loại. Duy trì độ trễ suy luận thấp giúp hệ thống phản ứng tức thời với các sự cố cơ khí đột ngột trước khi gây thiệt hại vật lý.

Nhờ lượng tử hóa nhận thức huấn luyện chuyển đổi trọng số sang INT8, hệ thống tối ưu hóa khối lượng tính toán. Bên cạnh đó, Seeed Studio XIAO ESP32-S3 sở hữu kiến trúc lõi kép Xtensa® LX7 tích hợp tập lệnh vector SIMD chuyên dụng cho trí tuệ nhân tạo, thực thi song song các phép toán nhân-tích lũy (MAC) trên mảng dữ liệu [9, 6].

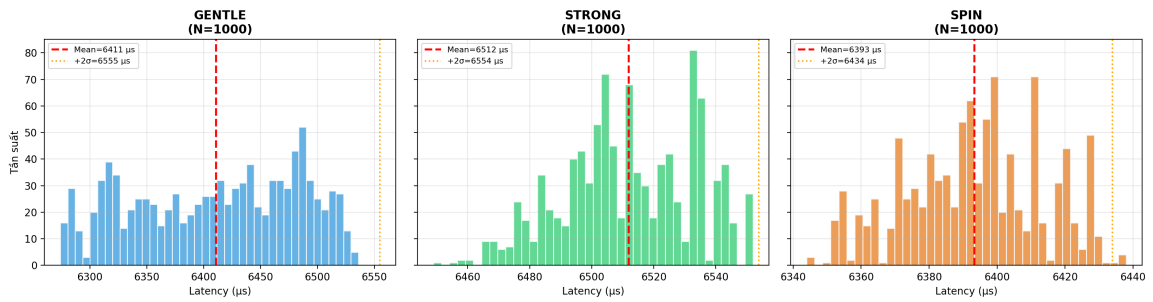
Để đo đạc chính xác, hệ thống thực hiện đánh giá hiệu năng tự động với $N=1\,000$ lần suy luận trên từng mô hình, sử dụng hàm `esp_timer_get_time()` đo trực tiếp thời gian thực thi của hàm `Invoke()` trong TensorFlow Lite Micro. Kết quả cho thấy thời gian suy luận trung bình tổng hợp đạt **6,44 ms** (Bảng 4.1).

Bảng 4.1

Thống kê độ trễ suy luận trên vi điều khiển Seeed Studio XIAO ESP32-S3 ($N=1\,000$ mẫu mỗi mô hình, tổng cộng 3 000 phép đo)

Mô hình	Mean (μs)	Std	Min	Max	P95	P99
GENTLE	6.411	71,9	6.274	6.536	6.516	6.526
STRONG	6.512	21,0	6.449	6.552	6.543	6.551
SPIN	6.393	20,3	6.344	6.438	6.427	6.429
Tổng hợp	6.439	68,8	6.274	6.552	6.535	6.545

Phân phối Độ trễ Suy luận trên ESP32-S3 (N=1000/model)



Hình 4.2

Biểu đồ phân phối thực nghiệm của độ trễ suy luận trên vi điều khiển Seeed Studio XIAO ESP32-S3 cho 3 mô hình ba trạng thái. Mỗi mô hình được đo 1000 lần suy luận liên tiếp. Độ trễ tập trung ổn định quanh 6,4 ms với độ lệch chuẩn thấp ($< 72\mu s$).

- Đánh giá đáp ứng thời gian thực: Với tần số lấy mẫu đặc trưng 1 Hz (chu kỳ 1000 ms), thời gian suy luận 6,4 ms chỉ chiếm 0,64% quỹ thời gian CPU, dư dả cho các tác vụ khác.
- Dư địa tài nguyên: Tác vụ Trí tuệ nhân tạo (AI) hoàn thành trong vài mili-giây, cho phép lõi CPU số 1 nhường tài nguyên cho các tác vụ giao tiếp mạng (MQTT, TLS/SSL) mà không gây nghẽn cổ chai.
- Biến động thời gian suy luận: Mô hình GENTLE ghi nhận độ lệch chuẩn cao hơn so với hai mô hình còn lại. Điều này xuất phát từ việc vùng đệm tensor được cấp phát trên PSRAM ngoại vi; các hiện tượng trượt bộ nhớ đệm ngẫu nhiên của vi điều khiển Seeed Studio XIAO ESP32-S3 khi truy xuất dữ liệu qua bus SPI làm tăng nhẹ thời gian của một số chu kỳ. Tuy nhiên, mức chênh lệch này vẫn nằm trong ngưỡng chấp nhận được của hệ thống.

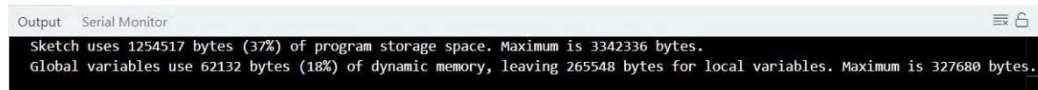
4.2.2 Đánh giá mức sử dụng tài nguyên bộ nhớ và độ ổn định hệ thống

Rào cản chính khi đưa các mô hình Học sâu vào vùng biên là dung lượng bộ nhớ tĩnh (Flash) và động (SRAM) eo hẹp [6, 1].

Chiếm dụng bộ nhớ Flash và số lượng tham số:

Hệ thống tối ưu hóa kiến trúc đối xứng với Lớp thắt cổ chai 32 chiều và lượng tử hóa INT8 triệt để. Toàn bộ 3 mô hình (GENTLE, STRONG, SPIN) sau lượng tử hóa INT8 có tổng dung lượng chỉ **95,3 KB**. Khi tích hợp vào firmware hoàn chỉnh (mã điều khiển, FreeRTOS, TensorFlow Lite for Microcontrollers, thư viện

MQTT/TLS, WiFi stack và các driver I2S/I2C), tổng dung lượng Flash được ghi nhận qua kết quả biên dịch thực tế là **1.254.517 bytes** (xấp xỉ **1,20 MB**), chiếm **37%** không gian chương trình (Hình 4.3).



Hình 4.3

Kết quả biên dịch firmware v11 trên Arduino IDE 2.3.5, xác nhận mức chiếm dụng bộ nhớ Flash (37%) và SRAM (18%) thực tế trên vi điều khiển Seeed Studio XIAO ESP32-S3.

Quản lý bộ nhớ động (SRAM/PSRAM):

TensorFlow Lite Micro đòi hỏi vùng nhớ đệm để lưu trữ các tensor trung gian. Nhờ kiến trúc gọn nhẹ, tổng không gian nhớ mà các mô hình yêu cầu rất thấp. Bộ nhớ động (Global Variables) chiếm **62.132 bytes (18%)** trên tổng 327.680 bytes SRAM, để dư 265.548 bytes cho các tác vụ thời gian thực. Hệ thống quy hoạch bộ nhớ trên kiến trúc lõi kép Seeed Studio XIAO ESP32-S3:

1. Tận dụng SRAM nội bộ (512 KB) cho các bộ đệm DMA tốc độ cao của cảm biến âm thanh và rung động để tránh mất mẫu.
2. Vùng nhớ (3×40 KB) được cấp phát trên PSRAM 8 MB thông qua hàm `heap_caps_aligned_alloc()`, đảm bảo dư dả tài nguyên cho các tác vụ giao tiếp mạng mà không gây xung đột hay Tràn ngăn xếp.
3. Hệ thống có thể duy trì hoạt động liên tục trong nhiều chu kỳ giặt mà không xảy ra Rò rỉ bộ nhớ hay Watchdog Reset.

Độ ổn định hệ thống trong thử nghiệm thực địa:

Hệ thống được đánh giá qua các bài kiểm tra thực địa kéo dài nhiều chu kỳ hoạt động đầy đủ của máy giặt. Nhờ kiến trúc FreeRTOS đa nhiệm:

- Tác vụ thu thập cảm biến trên lõi số 0 và tác vụ suy luận Trí tuệ nhân tạo trên lõi số 1 hoạt động độc lập thông qua Nhóm sự kiện và Hàng đợi.
- Tình trạng tác vụ không được cấp đủ thời gian CPU và xung đột tài nguyên được ngăn chặn nhờ gán mức độ ưu tiên (ví dụ: ngắt I2C đo rung động được cấp mức ưu tiên 15 - cao nhất để đảm bảo thời gian thực).

Trong các thử nghiệm dài hạn, hệ thống không ghi nhận Rò rỉ bộ nhớ hay Watchdog Reset. Dữ liệu đo xa được duy trì ổn định ở tần suất 1,0 Hz, đảm bảo luồng dữ liệu mượt mà cho cơ chế lọc nhiễu của sổ thời gian và kích hoạt cảnh báo tức thời trên ứng dụng di động.

4.3 Đánh giá khả năng phát hiện bất thường

Hệ thống học không giám sát nhằm nhận diện các mẫu dữ liệu chệch hướng so với phân phối chuẩn mà không cần gán nhãn trước [5]. Để xác thực năng lực này, đánh giá được tiến hành thông qua giả lập các kịch bản lỗi vật lý và phân tích sai số tái tạo trên mô hình TFLite thực tế.

4.3.1 Thiết lập kịch bản kiểm thử với giả lập lỗi (Anomaly Injection)

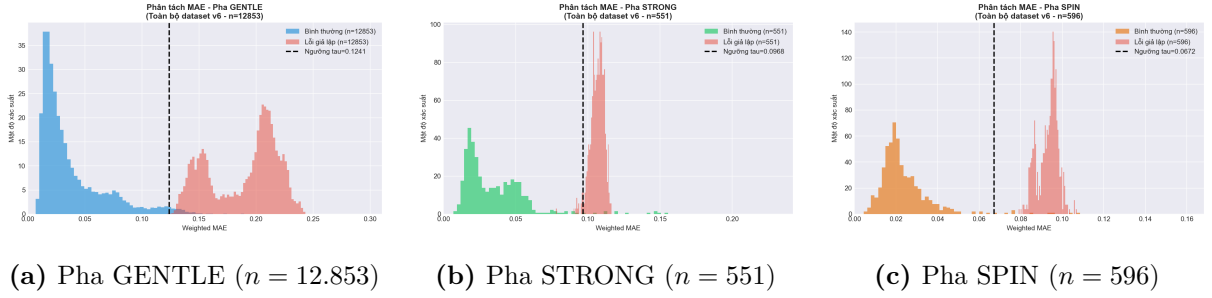
Hệ thống giả lập các lỗi vật lý trực tiếp trên đối tượng nghiên cứu để tạo ra các điểm bất thường có kiểm soát, tương tự các hư hỏng thường gặp:

- **Mất cân bằng tải trọng (Load Imbalance):** Mô phỏng lệch lồng giặt bằng cách bố trí vật nặng dồn về một phía. Thử nghiệm này buộc hệ thống xử lý các biên độ gia tốc văng lớn bất thường, đặc biệt trong pha SPIN.
- **Biến động âm thanh:** Tạo tiếng ồn ma sát kim loại hoặc va đập nhân tạo để kiểm chứng độ nhạy của bộ lọc Mel 13 dải trước các dấu hiệu hỏng hóc vòng bi hoặc động cơ.
- **Trường hợp biên:** Áp dụng dịch chuyển biên độ và nhiễu biến thiên để kiểm chứng độ bền bỉ của mô hình trước các nhiễu nền ngẫu nhiên trong môi trường dân dụng.

4.3.2 Phân tích phân phối sai số tái tạo

Mô hình được kiểm chứng trên *tập dữ liệu v6* — tập kiểm thử độc lập thu thập từ các phiên vận hành tách biệt hoàn toàn với dữ liệu huấn luyện, đảm bảo khả năng tổng quát hóa trước điều kiện môi trường thực tế. Phân tích biểu đồ MAE (Hình 4.4) cho thấy phân tách trạng thái rõ rệt:

- **Pha GENTLE** ($n = 12.853$): Mặc dù có cường độ tín hiệu thấp nhất và tập dữ liệu lớn nhất, mô hình duy trì sai số tái tạo bình thường ở mức cực thấp. Dữ liệu lỗi sau khi giả lập vượt hoàn toàn qua ngưỡng $\tau = 0,1241$, cho thấy độ nhạy của mô hình ngay cả ở trạng thái tĩnh.
- **Pha STRONG và SPIN**: Phân tách giữa hai cụm dữ liệu rõ nét. Đặc biệt ở pha SPIN, sai số MAE của dữ liệu bất thường tăng đáng kể so với ngưỡng $\tau = 0,0672$, cho phép hệ thống phát hiện lỗi vắng lòng với độ tin cậy cao.



Hình 4.4

Biểu đồ phân tách sai số tái tạo MAE thực nghiệm trên mô hình v11. Mỗi pha vận hành được kiểm thử với tập dữ liệu tương ứng và tập lỗi giả lập ($n_{anomaly} \approx 500$ cho mỗi pha). Đường đứt nét biểu thị ngưỡng quyết định (τ).

4.3.3 Đánh giá độ nhạy và hiệu quả qua các chỉ số học máy

Hiệu năng được đo lường qua chỉ số diện tích dưới đường cong ROC (AUC), F1-Score, Precision và Recall. Kết quả ghi nhận tại Bảng 4.2:

Bảng 4.2

Chỉ số hiệu năng của mô hình tự mã hóa Ba trạng thái trên tập kiểm thử

Trạng thái	AUC	F1-Score	Precision	Recall
GENTLE	0,9989	0,9898	0,9837	0,9960
STRONG	0,9837	0,9892	0,9804	0,9982
SPIN	0,9993	0,9967	0,9933	1,0000

4.3.4 Cơ chế lọc trễ thời gian và sự ổn định dài hạn

Trong môi trường thực tế, sai số tức thời có thể vượt ngưỡng do nhiễu ngẫu nhiên. Vì vậy, firmware sử dụng cơ chế biểu quyết bất đối xứng đã trình bày tại Hình 4.1: kích hoạt ALARM khi có ít nhất 5/10 mẫu vượt ngưỡng và chỉ phục hồi

về OK khi số mẫu vượt ngưỡng giảm xuống không quá 1/10. Sự kết hợp giữa mô hình học sâu và bộ lọc thời gian giúp hệ thống phản ứng chính xác với các xung nhiễu, đảm bảo chỉ phát cảnh báo khi sự cố thực sự xảy ra.

4.4 Phân tích thực nghiệm các quyết định thiết kế mô hình

Để xác nhận tính tối ưu của kiến trúc mạng nơ-ron tự mã hóa Ba trạng thái, hệ thống thực hiện các thí nghiệm bổ sung nhằm làm rõ vai trò của từng thành phần kỹ thuật.

4.4.1 Lựa chọn kiến trúc mạng qua các giai đoạn thử nghiệm

Trước khi chốt kiến trúc cuối cùng, đề án đã thử nghiệm ba cấu trúc mạng ở giai đoạn mô phỏng trên Kaggle: Vanilla AE làm mốc cơ sở, 1D-CNN AE cho dữ liệu chuỗi thời gian và LSTM AE cho phụ thuộc thời gian dài. Các mô hình được đánh giá trên cùng tập dữ liệu đã tiền xử lý, cùng quy trình huấn luyện và cùng các tiêu chí gồm sai số tái tạo, dung lượng mô hình sau khi nén INT8 trên Kaggle, độ trễ suy luận mô phỏng trên Kaggle và độ bất ổn khi thêm nhiễu. Kết quả ban đầu cho thấy thứ tự hiệu quả là 1D-CNN AE, Vanilla AE và LSTM AE.

Bảng 4.3

So sánh ba kiến trúc mạng ở giai đoạn mô phỏng sau khi nén INT8 trên Kaggle. Các giá trị trong bảng là số liệu mô phỏng, chưa phải kết quả đo trực tiếp trên phần cứng thực tế.

Chỉ số	Vanilla AE	1D-CNN AE	LSTM AE
RAE Loss (MSE)	0,004947	0,003698	0,004198
Dung lượng INT8 (KB)	375,01 KB	30,52 KB	368,66 KB
Độ trễ mô phỏng (ms/Kaggle)	76,56 ms	79,73 ms	129,50 ms
Bất ổn với nhiễu (%)	793,63%	706,43%	722,42%

Từ Bảng 4.3, 1D-CNN AE là ứng viên tốt nhất ở giai đoạn mô phỏng vì đạt sai số tái tạo thấp nhất, dung lượng sau nén INT8 nhỏ nhất và độ bất ổn với nhiễu thấp nhất trong ba mô hình. Vanilla AE có độ trễ mô phỏng thấp hơn 1D-CNN AE một chút nhưng lại có sai số tái tạo, dung lượng INT8 và độ bất ổn với nhiễu cao hơn đáng kể. LSTM AE không phù hợp với yêu cầu nhúng do độ trễ mô phỏng

cao nhất và dung lượng sau nén vẫn lớn.

Mặc dù 1D-CNN AE đứng đầu ở giai đoạn mô phỏng, quá trình triển khai thực tế trên TensorFlow Lite Micro cho thấy kiến trúc này không ổn định khi đưa xuống phần cứng. Nguyên nhân chính gồm sai lệch kết quả giữa triển khai Conv1D trên Python và C++, yêu cầu đầu vào dạng chuỗi làm tăng chi phí quản lý bộ đệm, và rủi ro mất ổn định khi chuyển toàn bộ mô hình sang INT8. Vì vậy, đề án chuyển hướng sang Symmetric Dense AE kết hợp khối trích xuất đặc trưng DSP 19 chiều. Phương án này không phải là mô hình đứng đầu trong vòng so sánh mô phỏng ban đầu, mà là quyết định thiết kế sau khi cân nhắc đầy đủ tính khả thi triển khai trên vi điều khiển Sseed Studio XIAO ESP32-S3.

Sau khi tối ưu, kiến trúc Symmetric Dense AE sử dụng cấu trúc đối xứng $19 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 19$ với lớp thất cổ chai Sigmoid, tương thích tốt với lượng tử hóa INT8. Tổng dung lượng ba mô hình cho ba pha vận hành là 95,3 KB, độ trễ suy luận trung bình đo thực tế đạt 6,4 ms, đáp ứng yêu cầu triển khai nhúng và được chọn cho hệ thống cuối cùng.

4.4.2 Nghiên cứu thành phần các biến thể mô hình

So sánh giữa mô hình ba trạng thái đề xuất với các biến thể cấu hình khác nhau giúp xác định hiệu quả của từng thành phần. Kết quả được tổng hợp trong Bảng 4.4.

Bảng 4.4

So sánh hiệu năng giữa các biến thể mô hình trong nghiên cứu thành phần. (*) Trễ trung bình đo thực tế trên vi điều khiển Sseed Studio XIAO ESP32-S3 với $N=1\,000$ lần.

Biến thể	MAE (SPIN)	F1-Score	Trễ (ms)	Model Flash (KB)
Mô hình đơn AE (Chung 3 pha)	0,0421	0,845	6,5	31,7
Ba trạng thái AE (Không dùng MAE có trọng số)	0,0291	0,912	6,4*	95,3
Ba trạng thái AE đề xuất	0,0242	0,9967	6,4*	95,3

Kiến trúc Ba trạng thái đem lại bước nhảy lớn nhất về hiệu năng. Biến thể mô hình đơn sử dụng chung một mô hình cho cả 3 pha chỉ đạt F1-Score 0,845, thấp hơn đáng kể so với Ba trạng thái, cho thấy mỗi pha có đặc tính thống kê riêng biệt. Bên cạnh đó, hàm mất mát MAE có trọng số (trọng số rung động gấp 5 lần âm thanh) nâng F1 từ 0,912 lên 0,9967 ở pha SPIN, khẳng định tầm quan trọng của việc ưu tiên kênh cảm biến mang tính vật lý cao hơn.

Kích thước lớp thất cổ chai ảnh hưởng trực tiếp đến khả năng nén đặc trưng và tài nguyên tính toán. Khảo sát các kích thước tiềm năng tìm ra kích thước tối ưu (Bảng 4.5).

Bảng 4.5

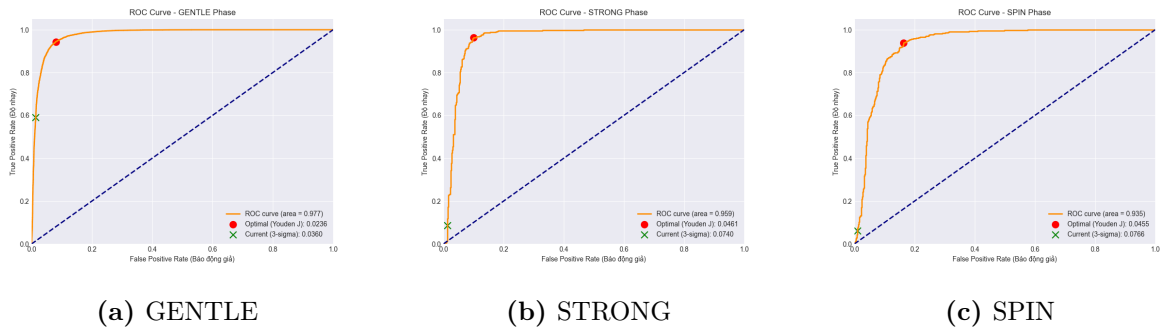
Kết quả khảo sát kích thước lớp thất cổ chai. *Lưu ý: Các giá trị độ trễ (ms) trong bảng là ước tính tương đối dùng để so sánh tỷ lệ tài nguyên.*

Số chiều	MAE (TB)	Tham số	Trễ (ms)	Đánh giá
8	0,0321	19.347	5,8	Underfitting
16	0,0265	22.563	6,1	Đạt yêu cầu
32	0,0242	25.779	6,4	Điểm tối ưu
64	0,0241	32.211	7,4	Bão hòa

Tại 32 chiều, mô hình đạt hiệu năng tối ưu. Tăng lên 64 chiều chỉ giảm sai số MAE không đáng kể nhưng tăng 15% độ trễ suy luận và dung lượng bộ nhớ. 32 chiều là lựa chọn tối ưu cho vi điều khiển Seeed Studio XIAO ESP32-S3.

4.4.3 Phân tích ngưỡng τ bằng đường cong ROC và chỉ số Youden's J

Phân tích đường cong ROC tối ưu hóa khả năng phân tách giữa dữ liệu bình thường và bất thường (Hình 4.5).



Hình 4.5

Đường cong ROC và điểm Youden's J Index tối ưu cho ba pha vận hành GENTLE, STRONG và SPIN.

Diện tích dưới đường cong (AUC) đạt: GENTLE (0,9989), STRONG (0,9837), SPIN (0,9993). Thông qua chỉ số Youden's J ($J = Sensitivity + Specificity - 1$), chúng tôi xác định ngưỡng $\tau_{optimal}$. Tuy nhiên, trong thiết kế thực tế, hệ thống ưu tiên ngưỡng thống kê 3σ (nghiêm ngặt hơn Youden) để giảm cảnh báo giả và tránh

ảnh hưởng tiêu cực đến người dùng trong môi trường dân dụng.

Tại ngưỡng 3σ , độ chính xác (Precision) đạt 0,99 ở cả 3 pha, nhưng chỉ số Recall ở pha GENTLE giảm xuống 0,78 so với 0,91 tại điểm Youden. Đây là sự đánh đổi kỹ thuật nhằm đảm bảo độ ổn định của hệ thống cảnh báo trước các nhiễu môi trường.

4.4.4 Phân tích sâu nguyên nhân sai số pha GENTLE

Pha STRONG có AUC thấp nhất (0,9837), trong khi pha GENTLE vẫn là chế độ dễ phát sinh nhiễu nền do cường độ vận hành thấp. Phân tích các mẫu bị nhận diện sai cho thấy hai vấn đề chính:

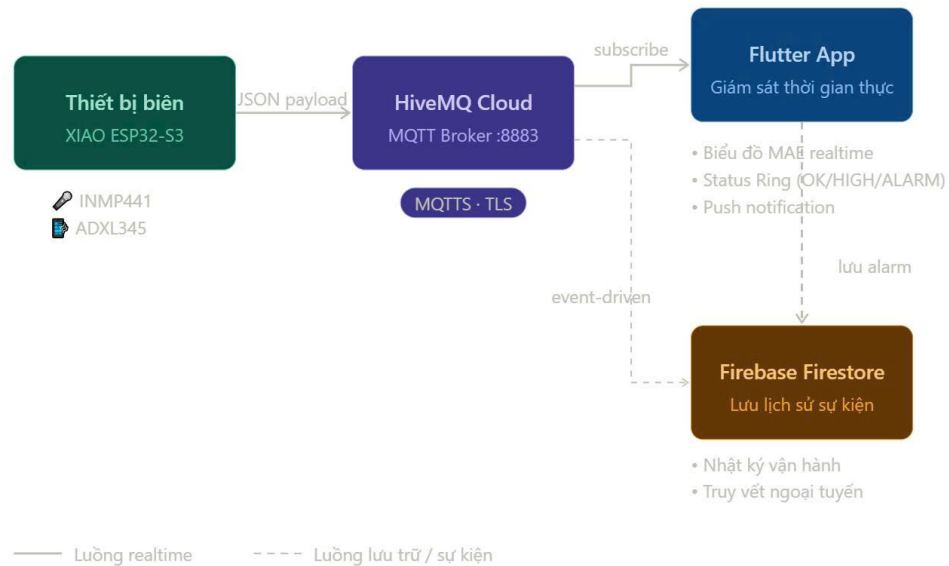
- **False Negatives:** Xảy ra khi biến động gia tốc trục Z cực nhỏ (MAE xấp xỉ ngưỡng bình thường). Ở mức năng lượng này, đặc trưng rung động của lỗi bị lấn át bởi nhiễu nền của động cơ Inverter, khiến mạng nơ-ron tự mã hóa tái tạo lỗi như tín hiệu bình thường.
- **False Positives:** Chủ yếu do các nhiễu âm thanh đột biến từ môi trường bên ngoài (tiếng va chạm vật lý vào vỏ máy) không thuộc chu kỳ vận hành của máy giặt nhưng trùng vào dải tần số bộ lọc Mel mà mô hình giám sát.

4.5 Triển khai ứng dụng di động giám sát và cảnh báo thời gian thực

Để hoàn thiện giải pháp AIoT từ biên đến đám mây, ứng dụng di động đa nền tảng được phát triển trên khung Flutter. Flutter là khung phát triển giao diện mã nguồn mở sử dụng mô hình giao diện khai báo, trong đó giao diện được xây dựng từ các thành phần tiện ích và tự cập nhật khi trạng thái dữ liệu thay đổi [40]. Đặc điểm này phù hợp với bài toán giám sát thời gian thực, nơi trạng thái máy giặt, sai số tái tạo và cảnh báo có thể thay đổi liên tục theo luồng MQTT. Ứng dụng hoạt động như trạm giám sát trung tâm, cho phép người dùng theo dõi tình trạng máy giặt từ xa và nhận cảnh báo tức thời.

4.5.1 Kiến trúc phần mềm và kết nối đám mây

Ứng dụng sử dụng kiến trúc hướng sự kiện, duy trì kết nối với hai nền tảng đám mây, như minh họa tại Hình 4.6.



Hình 4.6

Sơ đồ kiến trúc kết nối đám mây của ứng dụng di động. Dữ liệu từ thiết bị biên được phân phát theo hai nhánh độc lập: nhánh thời gian thực qua MQTT tới giao diện người dùng và nhánh lưu trữ lâu dài qua Firebase Firestore.

HiveMQ Cloud tiếp nhận luồng dữ liệu thời gian thực từ thiết bị biên với vai trò máy chủ trung gian MQTT. Dữ liệu được đóng gói theo định dạng JSON và truyền qua cổng 8883 (MQTTS), đảm bảo bảo mật và giữ độ trễ thấp.

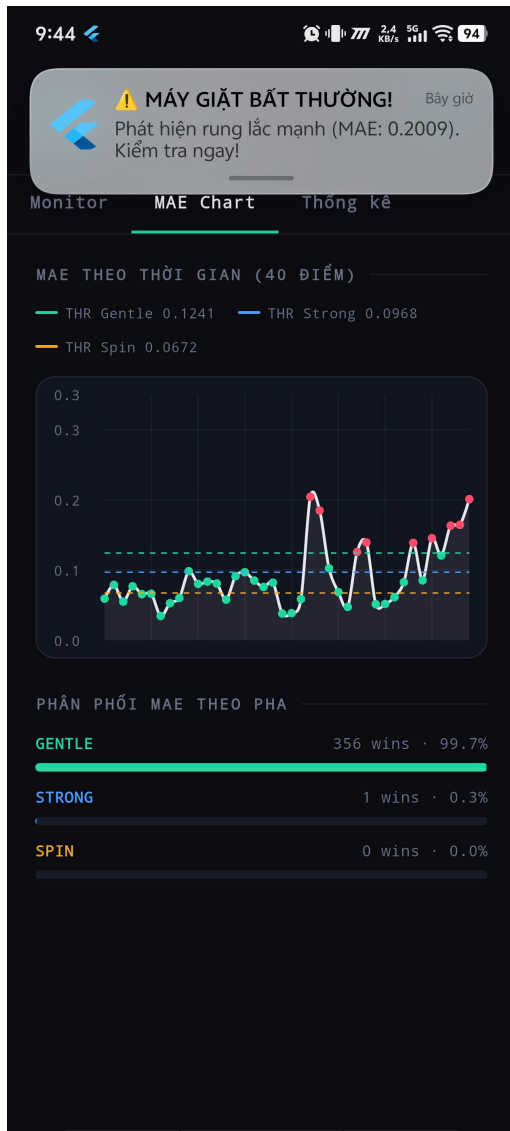
Firebase Firestore lưu trữ lịch sử các sự kiện bất thường. Firestore là cơ sở dữ liệu NoSQL tổ chức dữ liệu theo các bộ sưu tập và tài liệu, phù hợp với dữ liệu vận hành có cấu trúc linh hoạt như thời điểm cảnh báo, pha vận hành, giá trị MAE và trạng thái xử lý [41]. Người dùng có thể tra cứu nhật ký vận hành và mốc thời gian lỗi ngay cả khi thiết bị biên ngoại tuyến; điều này hỗ trợ bảo trì theo lịch.

4.5.2 Logic xử lý dữ liệu và thông báo đẩy

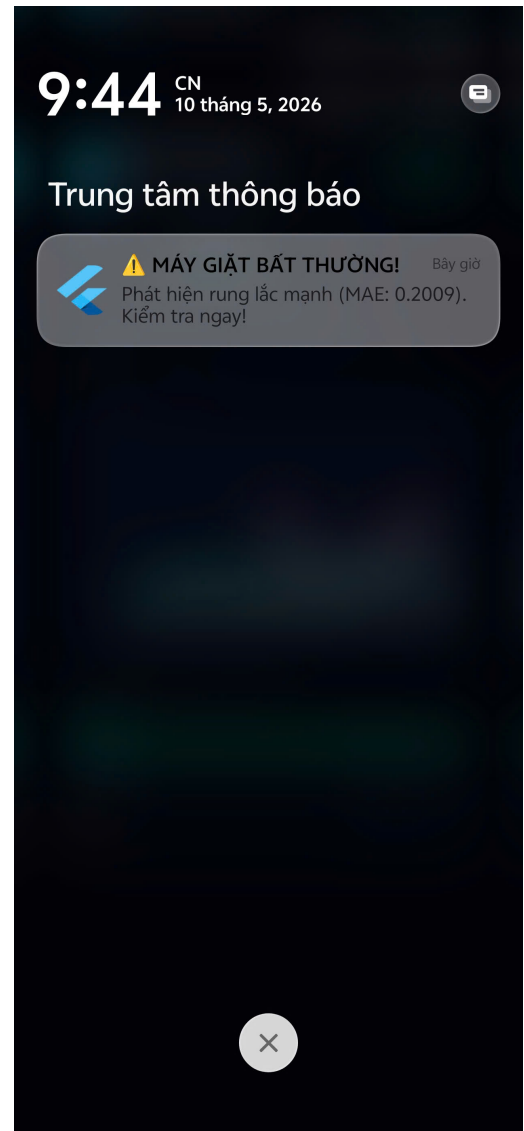
Ứng dụng sử dụng bộ lắng nghe MQTT để xử lý dữ liệu. Mỗi khi nhận gói tin từ chủ đề `tinym1/quang_wm_2026/status`, ứng dụng thực hiện ba bước: giải

mã tải tin JSON để trích xuất các trường `is_alarm`, `mae` và `mode`; kích hoạt thông báo đẩy ưu tiên cao nếu phát hiện cảnh báo, kèm đầy đủ giá trị MAE và pha vận hành; ghi nhận sự kiện vào Firestore. Song song đó, biểu đồ MAE trên giao diện được cập nhật liên tục.

Thông báo được tạo thông qua thư viện `flutter_local_notifications` với mức ưu tiên cao, bao gồm giá trị MAE và pha vận hành tại thời điểm xảy ra lỗi. Do đặc độ trễ từ đầu đến cuối — từ khi vi điều khiển phát hiện bất thường đến khi thông báo xuất hiện — ghi nhận mức xấp xỉ **2,0 giây**. Thời gian này bao gồm độ trễ MQTT qua máy chủ trung gian và phản hồi của hệ điều hành, đáp ứng yêu cầu giám sát thời gian thực cho thiết bị gia dụng. Hình 4.7 minh họa thông báo đẩy trên Android.



(a) Thông báo dạng banner xuất hiện phía trên màn hình khi ứng dụng đang mở.



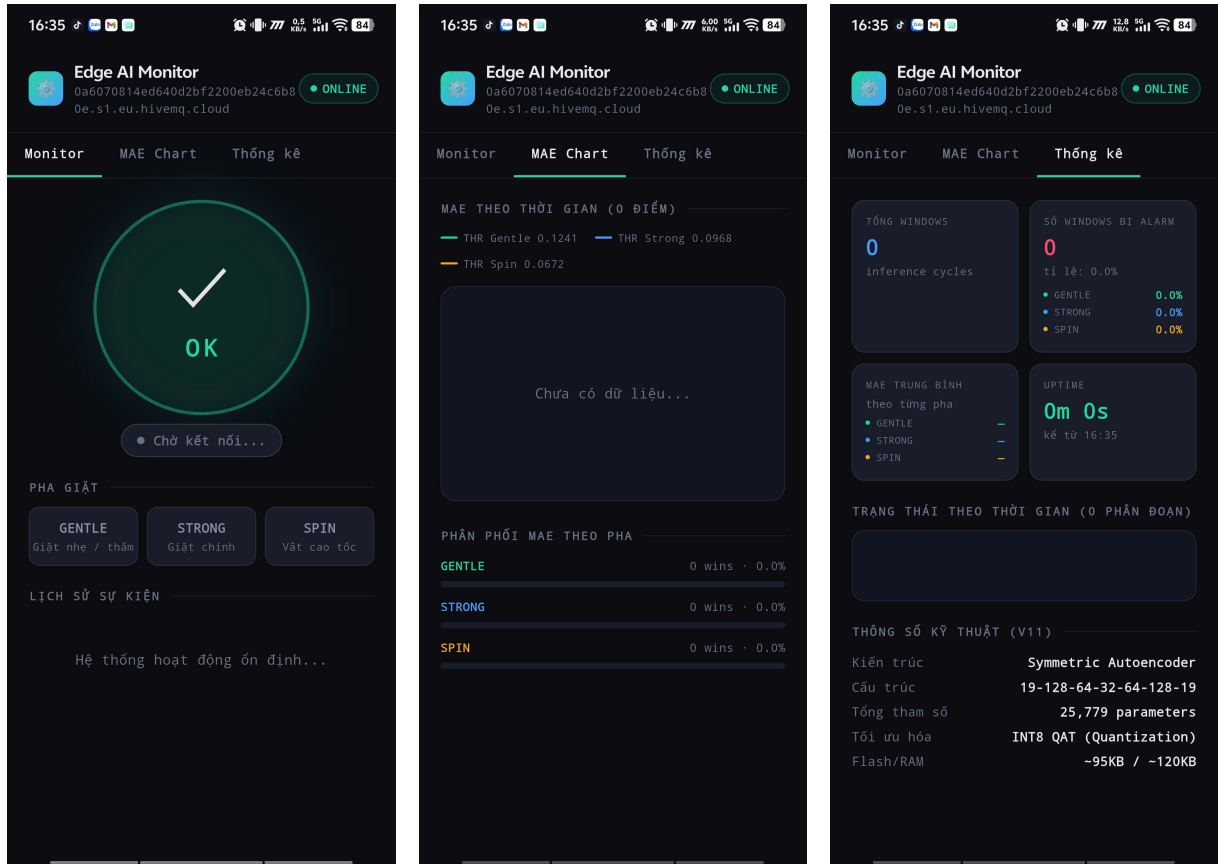
(b) Thông báo hiển thị tại Trung tâm thông báo khi thiết bị đang khóa màn hình.

Hình 4.7

Thông báo đẩy thực tế trên Android khi hệ thống kích hoạt ALARM — "Phát hiện rung lắc mạnh (MAE: 0.2009). Kiểm tra ngay!". Thông báo được gửi trong vòng **2,0 giây** kể từ khi vi điều khiển phát hiện bất thường.

4.5.3 Giao diện người dùng và trực quan hóa dữ liệu

Giao diện sử dụng chế độ tối hiện đại, tổ chức thành ba phần chức năng như tại Hình 4.8.



(a) Màn hình Giám sát

(b) Biểu đồ MAE

(c) Thống kê và Thông số

Hình 4.8

Giao diện ứng dụng di động Flutter: (a) Vòng tròn trạng thái động; (b) Trực quan hóa sai số MAE cùng các đường ngưỡng τ tương ứng; (c) Bảng thống kê nhật ký vận hành và thông số mô hình thực tế.

Màn hình Giám sát (Monitor) hiển thị vòng tròn trạng thái động thay đổi màu sắc: xanh lá ứng với trạng thái bình thường (OK), vàng khi sai số tiệm cận ngưỡng (HIGH), đỏ khi kích hoạt cảnh báo (ALARM). Cơ chế này giúp người dùng nhận biết tình trạng máy trong một cái nhìn mà không cần đọc số liệu kỹ thuật.

Biểu đồ MAE thời gian thực được dựng bằng thư viện `fl_chart`, thể hiện sai số tái tạo theo thời gian. Đường ngưỡng τ tương ứng với pha vận hành hiện tại được vẽ chồng lên dạng đường đứt nét làm tham chiếu. Quan sát xu hướng MAE trước khi chạm ngưỡng có giá trị thực tiễn: người dùng có thể nhận diện sớm dấu hiệu bất thường đang tích lũy và kiểm tra thiết bị trước khi lỗi trở nên nghiêm trọng.

Màn hình Thống kê và Nhật ký tổng hợp số lượng cảnh báo theo từng

pha vận hành (GENTLE, STRONG, SPIN), hỗ trợ người dùng đánh giá xu hướng hao mòn của thiết bị theo các chế độ giặt khác nhau trong dài hạn.

Lớp ứng dụng di động này hoàn thiện vòng phản hồi của hệ thống AIoT và kết nối các thuật toán TinyML phức tạp với người dùng cuối, giúp các chỉ số kỹ thuật trở thành thông tin cảnh báo có ý nghĩa thực tiễn và dễ tiếp cận.

4.6 So sánh với các nghiên cứu liên quan

4.6.1 Giới thiệu hệ thống mốc đối sánh của Mansoureh Lord

Nghiên cứu của Mansoureh Lord (2021) [15] là luận văn thạc sĩ tiêu biểu tại Sabancı University, tập trung phát hiện lỗi mất cân bằng lồng giặt trên máy giặt cửa trên sử dụng cảm biến gia tốc ADXL345 và mô hình tự mã hóa đơn giản một lớp ẩn. Hệ thống đạt độ chính xác khoảng 94,3% ở pha vắt nhưng chưa tích hợp kênh âm thanh và chưa triển khai cơ chế chuyển đổi trạng thái tự động. Trong phần này, công trình của Mansoureh Lord được sử dụng làm hệ thống mốc đối sánh trực tiếp.

Nghiên cứu này được lựa chọn làm mốc đối sánh vì:

- Cùng bài toán phát hiện bất thường trên máy giặt
- Sử dụng phần cứng cảm biến tương tự (ADXL345)
- Áp dụng kiến trúc mạng tự mã hóa cho bài toán phát hiện bất thường
- Công bố kết quả định lượng rõ ràng

Để xác nhận giá trị đóng góp, hệ thống tự mã hóa ba trạng thái được so sánh với nghiên cứu của Mansoureh Lord (2021) [15] — công trình tiêu biểu ứng dụng TinyML phát hiện lỗi mất cân bằng lồng giặt.

4.6.2 Cải tiến về kiến trúc mạng và thực thi phần cứng

Sự khác biệt cốt lõi nằm ở chiến lược tối ưu hóa kiến trúc để cân bằng độ chính xác và tài nguyên:

- **Cấu trúc mạng và đặc trưng đầu vào:** Hệ thống mốc đối sánh sử dụng dữ liệu rung động thô đưa trực tiếp vào mạng tự mã hóa tối giản (1 lớp ẩn)

[15]. Cách tiếp cận này nhẹ nhưng dễ bị nhiễu do không qua lọc đặc trưng. Đề tài áp dụng trích xuất đặc trưng xử lý tín hiệu số trước khi nạp vào mạng tự mã hóa Dense đối xứng. Việc tinh gọn đầu vào giúp mô hình hội tụ tốt hơn, đạt độ trễ suy luận 6,4 ms trên phần cứng nhúng — hoàn toàn đáp ứng yêu cầu thời gian thực với chu kỳ lấy mẫu 1 Hz.

- **Kỹ thuật lượng tử hóa:** Hệ thống mốc đối sánh dừng lại ở mô hình Float32 hoặc lượng tử hóa sau huấn luyện. Đề tài áp dụng lượng tử hóa nhận thức huấn luyện, nhúng sai số lượng tử hóa vào quá trình lan truyền ngược để mô hình duy trì nhất quán khi triển khai trên INT8.

4.6.3 Khả năng thích nghi đa trạng thái và độ tin cậy

Hệ thống vượt mốc đối sánh về phạm vi ứng dụng:

- **Cơ chế ba trạng thái chuyên biệt:** Hệ thống mốc đối sánh tập trung vào pha vắt của máy giặt cửa trên [15]. Hệ thống đề xuất nhận diện tự động và chuyển mô hình phù hợp với từng pha vận hành (GENTLE, STRONG, SPIN), giảm báo động giả khi thiết bị chuyển pha.
- **Kết hợp đa phương thức:** Bổ sung kênh âm thanh kỹ thuật số cho phép nhận diện các ma sát cơ khí tần số cao — điều cảm biến gia tốc đơn lẻ không thể nhận diện. Thực nghiệm cho thấy kết hợp này nâng F1-Score ở pha SPIN lên **0,9967**, vượt mốc đối sánh.

4.6.4 Đối chiếu với các hướng nghiên cứu gần đây

Ngoài mốc đối sánh TinyML trực tiếp của Lord, đề án cũng được đặt cạnh các nghiên cứu học sâu không giám sát trên thiết bị gia dụng. Castangia và cộng sự (2022) [16] sử dụng VAE trên chữ ký công suất của máy rửa bát, máy giặt và máy sấy, đạt F1-Score lớn hơn 0,90; tuy nhiên dữ liệu đầu vào của công trình này thuộc miền công suất điện, chưa xử lý trực tiếp rung động hoặc âm thanh cơ khí và chưa hướng tới triển khai TinyML trên vi điều khiển. Shul và cộng sự (2023) [12] khai thác phổ tiếng ồn và thông tin vận hành nội bộ của máy giặt bằng mạng DNN tự giám sát, cải thiện ROC-AUC tối đa khoảng 1,4%; cách tiếp cận này cho thấy giá trị của tín hiệu âm thanh nhưng vẫn cần thông tin như tốc độ quay hoặc khối lượng đồ giặt từ bên trong máy.

Vì vậy, Bảng 4.6 chỉ giữ phần đối chiếu trực tiếp với nghiên cứu của Lord (2021) [15], do đây là mốc đối sánh gần nhất với đề án về bài toán, cảm biến và hướng triển khai TinyML. Cách trình bày này giúp tránh lặp bảng, đồng thời làm rõ đóng góp chính của hệ thống đề xuất.

Bảng 4.6

Đối chiếu các thông số kỹ thuật và hiệu năng với mốc đối sánh [15]

Chỉ số so sánh	Mốc đối sánh (M. Lord)	Hệ thống đề xuất
Đối tượng nghiên cứu	Máy giặt cửa trên	Máy giặt cửa trước
Đặc trưng đầu vào	Rung động thô (3 trục)	Âm thanh + rung động
Kiến trúc mô hình	Tự mã hóa Dense đơn giản	Tự mã hóa đối xứng ba trạng thái
Lượng tử hóa	Không chuyên sâu	INT8 toàn phần
Độ trễ suy luận	> 10 ms (ước tính)	6,4 ms
F1-Score (Pha SPIN)	$\approx 0,943$	0,9967
Hệ thống cảnh báo	Đèn LED tại thiết bị	Ứng dụng di động + đám mây thời gian thực

4.7 Thảo luận và phân tích các mặt hạn chế

Mặc dù hệ thống mạng nơ-ron tự mã hóa ba trạng thái đạt được các chỉ số ấn tượng, quá trình thực nghiệm trong môi trường thực tế vẫn lộ ra những rào cản kỹ thuật. Nhận diện các hạn chế này là bước cần thiết để hướng dẫn cải tiến trong tương lai.

4.7.1 Hiện tượng Concept Drift và sự lão hóa cơ khí

Máy giặt chịu ảnh hưởng mạnh từ sự lão hóa theo thời gian. Các linh kiện như lò xo treo, giảm chấn cao su hay dây curoa sẽ thay đổi đặc tính đàn hồi sau 2-3 năm sử dụng.

- **Sự dịch chuyển phân phối:** Khi thiết bị lâu năm, biên độ rung động bình thường tự động dịch chuyển lên dải năng lượng cao hơn. Do mô hình tự mã hóa là tĩnh, các ngưỡng τ xác lập từ máy mới có thể trở quá nhạy, tăng tỷ lệ báo động giả (False Positives).

- **Thách thức về huấn luyện trên thiết bị:** Để khắc phục, hệ thống cần tự học lại hoặc tinh chỉnh tại biên để thích ứng với máy cũ. Tuy nhiên, giới hạn SRAM (512 KB) và chu kỳ CPU không cho phép thực hiện lan truyền ngược trực tiếp trên Sseed Studio XIAO ESP32-S3. Giải pháp hiện tại vẫn phụ thuộc vào thu thập dữ liệu và cập nhật firmware qua OTA.

4.7.2 Độ tin cậy hạ tầng và điều kiện vật lý của cảm biến

Triển khai Trí tuệ nhân tạo (AI) tại biên trong môi trường dân dụng gặp các yếu tố bên ngoài:

- **Sự ổn định của kết nối không dây:** Mặc dù suy luận AI diễn ra hoàn toàn offline, nhưng cảnh báo gửi tới người dùng phụ thuộc vào Wi-Fi 2.4 GHz. Trong môi trường nhà ở với nhiều vật cản, rớt gói tin (Packet Loss) có thể trễ thông báo. Một cơ chế cảnh báo vật lý (như còi Buzzer) tại chỗ cần thiết để bảo đảm.
- **Tác động của nhiệt độ và độ ẩm:** Máy giặt vận hành trong môi trường độ ẩm cao và nhiệt độ biến thiên (khi giặt nước nóng). Các cảm biến MEMS như ADXL345 và INMP441 có thể bị sai lệch (Offset Drift) nếu không được bảo vệ bằng vỏ chống nước IP67. Sai lệch nhỏ trong cảm biến có thể làm thay đổi vector đặc trưng đầu vào, gây nhiễu cho mô hình.

4.7.3 Giới hạn của dữ liệu lỗi giả lập

Hệ thống được kiểm chứng chủ yếu trên các lỗi cơ khí tức thời (như lệch tải trọng lồng giặt). Các loại lỗi tiến triển chậm như mòn vòng bi (bearing wear) hay rò rỉ nước nhỏ đòi hỏi tập dữ liệu Run-to-failure kéo dài hàng trăm giờ — một nguồn lực vượt quy mô của đề án này. Do đó, khả năng tổng quát hóa của mô hình đối với các hư hỏng tiến triển chậm vẫn cần kiểm chứng thực địa dài hạn.

KẾT LUẬN

5.1 Những kết quả đạt được

Trải qua quá trình nghiên cứu và thực nghiệm, đề tài đã hoàn thành đầy đủ các mục tiêu đề ra với những kết quả trọng tâm sau:

- **Về mặt thuật toán:** Xây dựng thành công kiến trúc *mạng nơ-ron tự mã hóa Ba trạng thái* cho phép tối ưu hóa việc phát hiện bất thường theo từng pha vận hành vật lý của thiết bị. Việc kết hợp đặc trưng âm thanh (13 dải bộ lọc Mel) và rung động đã nâng cao đáng kể độ nhạy của hệ thống.
- **Về mặt triển khai:** Triển khai thành công mô hình học máy tại biên trên vi điều khiển Sseeed Studio XIAO ESP32-S3. Mô hình sau lượng tử hóa INT8 đạt hiệu suất cao với thời gian suy luận trung bình 6,44 ms, chiếm dụng tài nguyên thấp với tổng dung lượng 95,3 KB cho 3 mô hình mạng nơ-ron tự mã hóa.
- **Về mặt hệ thống:** Hoàn thiện giải pháp *từ biên đến đám mây* tích hợp giao thức MQTT, cho phép giám sát trạng thái thiết bị theo thời gian thực và cung cấp khả năng cảnh báo tự động thông qua hạ tầng Internet vạn vật (IoT).
- **Về mặt thực nghiệm:** Đạt được các chỉ số hiệu năng nhất quán trên ba pha vận hành, với F1-Score lần lượt là 0,9898 (GENTLE), 0,9892 (STRONG) và 0,9967 (SPIN). Pha vất đạt Recall bằng 1,0000, đồng thời hệ thống kiểm soát tốt tỷ lệ cảnh báo giả nhờ cơ chế lọc trễ thời gian.

Bảng 5.1

Tổng hợp các kết quả định lượng chính của đề án.

Nhóm đánh giá	Kết quả đạt được
Hiệu năng phát hiện	F1-Score: GENTLE 0,9898; STRONG 0,9892; SPIN 0,9967.
Độ nhạy cảnh báo	Recall: GENTLE 0,9960; STRONG 0,9982; SPIN 1,0000.
Độ trễ suy luận	Trung bình 6,439 ms trên 3.000 phép đo thực tế, mỗi mô hình đo 1.000 lần.
Dung lượng mô hình	95,3 KB cho toàn bộ 3 mô hình INT8.
Dung lượng firmware	1.254.517 bytes, xấp xỉ 1,20 MB, chiếm khoảng 37% bộ nhớ chương trình.
Kiến trúc hệ thống	Thiết bị biên ESP32-S3, suy luận TinyML, truyền cảnh báo qua MQTT và ứng dụng Flutter/Firebase.

5.2 Hạn chế của đề tài

Bên cạnh những kết quả đạt được, đề tài có một số hạn chế cần ghi nhận:

- **Dữ liệu lỗi giả lập:** Do các sự cố hỏng hóc tự nhiên hiếm gặp, dữ liệu bất thường chủ yếu sử dụng phương pháp giả lập lỗi. Điều này chưa phản ánh sự thoái hóa cơ khí tuyến tính như mòn vòng bi qua nhiều năm.
- **Ngưỡng cắt cứng (Hard-thresholding):** Sử dụng ngưỡng phương sai cố định để phân pha ba trạng thái có thể gây dao động trạng thái (hiệu ứng ping-pong) trong giai đoạn chuyển đổi giữa các chu kỳ giặt.
- **Sự phụ thuộc vào firmware tĩnh:** Hệ thống chưa có khả năng tự học và thích ứng (Học trên thiết bị) với sự thay đổi thiết bị theo thời gian. Cần cập nhật firmware từ xa khi các thông số cơ khí thay đổi đáng kể.
- **Chưa đánh giá tiêu thụ năng lượng dài hạn:** Đề án đã đo đặc độ trễ suy luận, dung lượng Flash và khả năng vận hành thời gian thực, nhưng chưa thực hiện phép đo dòng tiêu thụ hoặc công suất trung bình trong các phiên chạy nhiều ngày. Vì vậy, khả năng triển khai dưới dạng nút cảm biến chạy pin vẫn cần được kiểm chứng thêm.

5.3 Hướng phát triển tiếp theo

Để hoàn thiện hệ thống, các hướng nghiên cứu trong tương lai tập trung vào:

- **Triển khai Học trên thiết bị:** Tích hợp các thuật toán học máy để tinh chỉnh trọng số trên vi điều khiển, thích ứng với sự lão hóa thiết bị (Lệch khái niệm).
- **Mở rộng hệ sinh thái giám sát:** Áp dụng kiến trúc mạng nơ-ron tự mã hóa ba trạng thái cho các thiết bị gia dụng khác có động cơ quay như máy nén tủ lạnh, máy bơm nước hoặc điều hòa không khí.
- **Tích hợp Trí tuệ nhân tạo sinh ở tầng ứng dụng:** Kết hợp dữ liệu cảnh báo, lịch sử lỗi và thống kê vận hành từ Firebase với mô hình ngôn ngữ lớn ở phía máy chủ hoặc ứng dụng di động để tạo báo cáo bảo trì bằng ngôn ngữ tự nhiên. Hướng này chỉ đóng vai trò diễn giải và hỗ trợ người dùng, trong khi quyết định phát hiện bất thường vẫn được thực hiện tại thiết bị biên để giữ độ trễ thấp và giảm phụ thuộc mạng.
- **Tối ưu hóa quản lý năng lượng:** Nghiên cứu cơ chế đánh thức hệ thống kích hoạt bởi sự kiện (Khởi động theo sự kiện) để hướng tới giám sát không dây dùng pin dài hạn.

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Anh

- [1] Xubin Wang, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang và Weijia Jia (2025), “Empowering edge intelligence: A comprehensive survey on on-device ai models”. *ACM Computing Surveys*, tập 57, tr. 1–39, doi:10.1145/3724420.
- [2] Hamidreza Mohammadi (2020), “Early detection of imbalance in load and machine in front load washing machines by monitoring drum movement”. Luận văn Thạc sĩ, Sabanci University, Istanbul, Turkey.
- [3] Partha Pratim Ray (2022), “A review on tinyml: State-of-the-art and prospects”. *Journal of King Saud University - Computer and Information Sciences*, tập 34, số 4, tr. 1595–1623, doi:10.1016/j.jksuci.2021.11.019.
- [4] Mohammed Ayalew Belay, Sindre Stenen Blakseth, Adil Rasheed và Pierluigi Salvo Rossi (2023), “Unsupervised anomaly detection for iot-based multivariate time series: Existing solutions, performance analysis and future directions”. *Sensors*, tập 23, số 5, tr. 2844, doi:10.3390/s23052844.
- [5] Raghavendra Chalapathy và Sanjay Chawla (2019), “Deep learning for anomaly detection: A survey”. *arXiv preprint arXiv:1901.03407*.
- [6] Ebere Njor, Mohammad Ali Hasanpour, Jan Madsen và Xenofon Fafoutis (2024), “A holistic review of the tinyml stack for predictive maintenance”. *IEEE Access*, tập 12, tr. 184.861–184.882.

- [7] Mustafa Abdallah, Byung-Gun Joung, Wo Jae Lee, Charilaos Mousoulis, Nithin Raghunathan, Ali Shakouri, John W. Sutherland và Saurabh Bagchi (2023), “Anomaly detection and inter-sensor transfer learning on smart manufacturing datasets”. *Sensors*, tập 23, số 1, tr. 486, doi:10.3390/s23010486.
- [8] Taiwo Samuel Ajani, Agbotiname Lucky Imoize và Aderemi A. Atayero (2021), “An overview of machine learning within embedded and mobile devices—optimizations and applications”. *Sensors*, tập 21, số 13, tr. 4412, doi: 10.3390/s21134412.
- [9] Espressif Systems (2026), *ESP32-S3 Series Datasheet*. URL https://documentation.espressif.com/esp32-s3_datasheet_en.pdf. Version 2.2.
- [10] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam và Dmitry Kalenichenko (2018), “Quantization and training of neural networks for efficient integer-arithmetic-only inference”. Trong “Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)”, tr. 2704–2713, doi:10.1109/CVPR.2018.00286.
- [11] Pete Warden và Daniel Situnayake (2019), *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers* (O’Reilly Media), ISBN 9781492052043.
- [12] Yusun Shul, Wonjun Yi, Jihoon Choi, Dong Soo Kang và Jung Woo Choi (2023), “Noise-based self-supervised anomaly detection in washing machines using a deep neural network with operational information”. *Mechanical Systems and Signal Processing*, tập 189, tr. 110.102, doi:10.1016/j.ymssp.2023.110102.
- [13] Alan V. Oppenheim và Ronald W. Schaffer (2010), *Discrete-Time Signal Processing* (Pearson), 3 edition.
- [14] Uditha Muthumala, Yuxuan Zhang, Luciano Sebastian Martinez-Rau và Sebastian Bader (2024), “Comparison of tiny machine learning techniques for embedded acoustic emission analysis”. Trong “2024 IEEE 10th World Forum on Internet of Things (WF-IoT)”, tr. 444–449, doi:10.1109/WF-IoT62078.2024.10811219.
- [15] Mansoureh Lord (2021), “Tinyml anomaly detection”. Luận văn Thạc sĩ, California State University, Northridge.

- [16] Marco Castangia, Riccardo Sappa, Awet Abraha Girmay, Christian Camarda, Enrico Macii và Edoardo Patti (2022), “Anomaly detection on household appliances based on variational autoencoders”. *Sustainable Energy, Grids and Networks*, tập 32, tr. 100.823, doi:10.1016/j.segan.2022.100823.
- [17] TDK InvenSense (2014), *INMP441: Omnidirectional Microphone with Bottom Port and I2S Digital Output*.
- [18] Analog Devices (2015), *ADXL345: 3-Axis, Digital Accelerometer*.
- [19] Real-Time Engineers Ltd. (2021), *FreeRTOS Reference Manual*. V10.4.0.
- [20] Espressif Systems (2023), *ESP-DSP Library Programming Guide*.
- [21] Yoshua Bengio, Aaron Courville và Pascal Vincent (2013), “Representation learning: A review and new perspectives”. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, tập 35, số 8, tr. 1798–1828, doi:10.1109/TPAMI.2013.50.
- [22] Matteo Frustaci, Pasquale Pace, Gianluca Aloì và Giancarlo Fortino (2018), “Evaluating critical security issues of the iot world: Present and future challenges”. *IEEE Internet of Things Journal*, tập 5, số 4, tr. 2483–2495, doi: 10.1109/JIOT.2018.2844090.
- [23] Peter J. Rousseeuw và Annick M. Leroy (2005), *Robust Regression and Outlier Detection* (John Wiley & Sons).
- [24] Diederik P. Kingma và Jimmy Ba (2014), “Adam: A method for stochastic optimization”. *arXiv preprint arXiv:1412.6980*.
- [25] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, Urmish Thakker, Antonio Torrini, Peter Warden, Jay Cordaro, Giuseppe Di Guglielmo, Javier Duarte, Stephen Gibellini, Videet Parekh, Honson Tran, Nhan Tran, Niu Wenxu và Xu Xuesong (2021), “Mlperf tiny benchmark”. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, doi:10.48550/arXiv.2106.07597.

Website

- [26] IoT Insider (2025), “Global iot connections set to reach 21.9 billion by 2026 as enterprises rethink connectivity strategies”. URL <https://www.iotinsider.com/news/global-iot-connections-set-to-reach-21-9-billion-by-2026-as-enterprises-rethink-connectivity-strategies/>. Truy cập ngày: 17/05/2026.
- [27] Susi Wallner (2026), “Iot report 2026: 39 billion connected devices by 2030”. URL <https://www.startus-insights.com/innovators-guide/iot-report/>. Truy cập ngày: 17/05/2026.
- [28] Mordor Intelligence (2026), “Artificial intelligence of things (aiot) market size and research report 2031”. URL <https://www.mordorintelligence.com/industry-reports/artificial-iintelligence-of-things-market>. Truy cập ngày: 17/05/2026.
- [29] World Economic Forum (2026), “Why making ai sustainable by design is key to a greener future”. URL <https://www.weforum.org/stories/2026/02/designing-sustainable-ai-better-future/>. Truy cập ngày: 17/05/2026.
- [30] Ericsson (2026), “Rethinking massive iot for the 6g era”. URL <https://www.ericsson.com/en/blog/2026/3/rethinking-massive-iot-for-the-6g-era>. Truy cập ngày: 17/05/2026.
- [31] Boston Consulting Group và Qualcomm (2026), “6g: The network for the future of ai and immersive connectivity”. URL <https://web-assets.bcg.com/15/95/eaacdd6544c1b10c74b38a1afcb8/6g-the-network-for-the-future-of-ai-and-immersive-connectivity-report-feb-2026-edit-02-web.pdf>. Truy cập ngày: 17/05/2026.
- [32] Covalense Digital (2026), “Top six telecom trends 2026: Agentic ai, 6g, open ran, and more”. URL <https://covalensedigital.com/resources/blogs/top-telecom-trends-2026>. Truy cập ngày: 17/05/2026.

- [33] Device Authority (2026), “The state of iot identity security in 2026: Why machine identity is the new perimeter”. URL <https://deviceauthority.com/the-state-of-iot-identity-security-in-2026-why-machine-identity-is-the-new-perimeter/>. Truy cập ngày: 17/05/2026.
- [34] IoT Analytics (2026), “State of enterprise iot 2026: The shift from iot to autonomous connected operations”. URL <https://iot-analytics.com/state-of-enterprise-iot-from-iot-autonomous-connected-operations/>. Truy cập ngày: 17/05/2026.
- [35] Leandro Antonio Pazmino Ortiz, Alan Cuenca-Sanchez và Byron Loarte (2026), “Distributed intelligence in the artificial intelligence of things: A comprehensive review of architectures, applications, and challenges”. doi: 10.20944/preprints202604.0261.v1, URL <https://www.preprints.org/manuscript/202604.0261>. Truy cập ngày: 17/05/2026.
- [36] Google (2026), “Litert for microcontrollers”. URL <https://ai.google.dev/edge/litert/microcontrollers/overview>. Truy cập ngày: 17/05/2026.
- [37] Seeed Studio (2023), “Getting started with seeed studio xiao esp32s3 (sense)”. URL https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/. Truy cập ngày: 17/05/2026.
- [38] OASIS Open (2019), “Mqtt version 5.0”. URL <https://www.oasis-open.org/standard/mqtt-v5-0-os/>. Truy cập ngày: 17/05/2026.
- [39] HiveMQ (2026), “Mqtt essentials”. URL <https://www.hivemq.com/mqtt/>. Truy cập ngày: 17/05/2026.
- [40] Flutter (2026), “Building user interfaces with flutter”. URL <https://docs.flutter.dev/ui>. Truy cập ngày: 17/05/2026.
- [41] Firebase (2026), “Cloud firestore data model”. URL <https://firebase.google.com/docs/firestore/data-model>. Truy cập ngày: 17/05/2026.